

American University of Ras Al Khaimah

School of Engineering

SENIOR DESIGN PROJECT

TrackMed: An Autonomous Mobile Manipulator for Scheduled Medication Delivery in Hospitals

Submitted by:

Seif Elwalid Elsenousy Taha
Mohammed Yousef Ahmed Hajeeh Alblooshi
Salem Alnuaimi
Yousef Abdulla Yousef Abdulla Bin Salman

Supervised by: Dr. Arfan Ghani

Department of Computer Science and Engineering

Fall 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our supervisor, Dr. Arfan Ghani, for his guidance, patience and constructive feedback throughout both semesters of this Senior Design Project. His advice shaped the direction of the project and helped us turn a difficult setback into a complete piece of engineering work.

We thank the School of Engineering at the American University of Ras Al Khaimah for providing the robotic platform, laboratory space and computing resources used in this project, and the faculty members who reviewed our proposal and progress.

We also acknowledge Hiwonder for the documentation, course material and simulation packages supplied with the JetArm Pro. Their robot description and line-following examples made it possible to continue the project in simulation after the physical robot failed.

Finally, we thank our families and friends for their encouragement and support during the project.

Table of Contents

List of Figures **10**
List of Tables **13**
List of Abbreviations **15**
ABSTRACT **17**
1 INTRODUCTION **18**
1.1 Problem Statement *18*
1.2 Purpose of the Study *19*
1.3 Project and Design Objectives *19*
1.4 Intended Outcomes and Deliverables *21*
1.4.1 Mobile Robotic Platform with Line-Following Navigation *21*
1.4.2 Scheduled Dispensing and Delivery Management *21*
1.4.3 Robotic Arm Manipulation for Container Delivery *21*
1.4.4 Evaluation Platform, Documentation and Interface *22*
1.5 Change of Platform and Scope *22*
1.6 Report Organization *22*
2 BACKGROUND **24**
2.1 Existing Approaches/Applications *24*
2.1.1 Manual Administration *24*
2.1.2 Stationary Smart Pill Dispensers *24*
2.1.3 Hospital Logistics Robots *24*
2.1.4 Educational and Research Mobile Manipulators *25*
2.2 Enabling Technologies *26*
2.2.1 Robot Operating System 2 ..... *26*
2.2.2 Gazebo Simulation *26*
2.2.3 Mecanum-Wheel Mobile Bases *26*
2.2.4 Robot Arm Kinematics *26*
2.2.5 Vision-Based Line Following *27*
2.2.6 Simulation and the Sim-to-Real Gap *27*
2.3 Discussion *27*
3 PROJECT MANAGEMENT **29**
3.1 Tasks, Schedule, and Milestones *29*
3.1.1 Tasks *29*
3.1.2 Schedule *30*
3.1.3 Milestones *31*

3.1.4 Project Charter and its Components *33*
3.1.5 Purpose and Objectives *33*
3.1.6 Scope *33*
3.1.7 Key Stakeholders and Roles *34*
3.1.8 Budget and Resources *34*
3.1.9 Timeline *35*
3.1.10 Key Deliverables *35*
3.1.11 Assumptions and Constraints *36*
3.1.12 High-Level Risks *36*
3.1.13 Task Breakdown and Scheduling *36*
3.1.14 Work Breakdown Structure (WBS) *37*
3.2 Risk Management and Mitigation *38*
3.2.1 Risk Identification *38*
3.2.2 Risk Assessment *38*
3.2.3 Risk Mitigation Strategies *39*
3.2.4 Realised Risk: Failure of the Robot *40*
3.2.5 Continuous Risk Monitoring *41*
4 REQUIREMENTS ANALYSIS **42**
4.1 Functional & Non-Functional Requirements *42*
4.1.1 Functional Requirements *42*
4.1.2 Scheduling and Dispensing Logic *43*
4.2 Non-Functional Requirements *44*
4.2.1 Performance *45*
4.2.2 Reliability *45*
4.2.3 Scalability *45*
4.2.4 Usability *46*
4.2.5 Security *46*
4.2.6 Maintainability *46*
4.2.7 Interoperability *46*
4.2.8 Safety and Reproducibility *46*
4.3 User & System Requirements and Constraints *47*
4.3.1 User Requirements *47*
4.3.2 System Requirements *47*
4.3.3 Constraints *47*
4.3.4 Functional Standards *48*
4.3.5 Industry-Specific Standards *48*
4.3.6 Data Security Standards *49*
4.4 Domain Requirements *49*

4.4.1 Healthcare and Medical Device Regulations *49*
4.4.2 Patient Safety and Collision Avoidance *49*
4.4.3 Human-Robot Interaction (HRI) Protocols *50*
4.4.4 Health Data Privacy Laws *50*
5 SYSTEM ANALYSIS & DESIGN **51**
5.1 System Model *51*
5.2 Methodology *53*
5.3 Hardware Platform *54*
5.3.1 JetArm Pro Robotic Arm *54*
5.3.2 Mecanum-Wheel Chassis *54*
5.3.3 Medication Container *56*
5.4 Software Architecture *56*
5.4.1 Technology Stack *56*
5.4.2 ROS 2 Nodes and Topics *56*
5.5 Arm Kinematics and Pick-and-Place Design *58*
5.5.1 Forward Kinematics *58*
5.5.2 Closed-Form Inverse Kinematics *59*
5.5.3 Gripper and Grasp Height *61*
5.5.4 Grasp Strategy *61*
5.5.5 Pick-and-Place Motion Sequence *62*
5.6 Navigation Design *62*
5.6.1 Camera Geometry *62*
5.6.2 Line-Detection Algorithm *63*
5.6.3 Steering Control *64*
5.6.4 Mobile Base and Turn-Round Manoeuvre *64*
5.7 Environment Design *65*
5.7.1 Ward Layout and Path *65*
5.7.2 Patient Room *66*
5.7.3 Smart Bedside Table *67*
5.8 Mission Design *70*
5.8.1 Delivery Sequence *70*
5.8.2 Mission States *71*
5.8.3 Interaction Between Components *73*
5.9 Scheduler and Data Design *74*
5.9.1 Schedule File *74*
5.9.2 Delivery Log and Result Files *75*
5.10 Technical Choices *75*
5.10.1 Programming Language *75*

5.10.2 Development Environment *76*
5.10.3 Simulator and Rendering *76*
5.10.4 Data Storage *76*
5.11 UI/UX Design *76*
5.11.1 User Interface Considerations *76*
5.11.2 User Experience Considerations *76*
5.12 Hospital System Design *77*
5.12.1 Design Goals and Constraints *77*
5.12.2 Building Layout *77*
5.12.3 Medicine Storage Unit *79*
5.12.4 Line Network and Route Planning *81*
5.12.5 Junction Handling *81*
5.12.6 Cap Check *83*
5.12.7 Delivery Sequence *83*
5.12.8 Scheduling *84*
6 IMPLEMENTATION 86
6.1 Development Environment *86*
6.1.1 Setup *86*
6.1.2 Problems Solved *86*
6.2 Arm on a Fixed Base *88*
6.3 Medication Container Model *89*
6.4 Grasp Development *90*
6.5 Robot and World Models *93*
6.5.1 Mobile Robot Description *93*
6.5.2 Base Motion Model: from Velocity Control to Planar Joints *94*
6.5.3 Hospital World *94*
6.5.4 Launch File *96*
6.6 Arm Control Implementation *96*
6.7 Line Follower Implementation *97*
6.8 Mission Controller Implementation *97*
6.9 Scheduler Implementation *98*
6.10 Evaluation and Documentation Tools *98*
6.10.1 Automated Evaluation *98*
6.10.2 Screenshots and Video *98*
6.10.3 Version Control and Backup *99*
6.11 Patient Room and Smart Bedside Table *99*
6.11.1 Generating the Room *99*
6.11.2 Smart Table Model *102*

6.11.3 Table Controller Node *102*
6.11.4 Mission, Scoring and Video Changes *103*
6.11.5 Problems Solved *104*
6.12 Hospital System Implementation *104*
6.12.1 World Generation *105*
6.12.2 Pharmacy Set-Up *106*
6.12.3 Route Follower and Planner *106*
6.12.4 Cap Colours *107*
6.12.5 Fine Line-Up at the Slot *107*
6.12.6 Robust Arm Wait *108*
6.12.7 Demonstration Runner *108*
6.12.8 Problems Solved *109*
7 TESTING AND RESULTS 111
7.1 Test Strategy *111*
7.2 Offline Tests *111*
7.2.1 Kinematics, Reach and Gripper *112*
7.2.2 Line Follower in Closed Loop *112*
7.2.3 Effect of the Camera Frame Rate *113*
7.3 Subsystem Tests in Gazebo *113*
7.3.1 Base Motion *113*
7.3.2 Simulation Speed *114*
7.3.3 Arm and Camera *115*
7.3.4 Line Following *115*
7.3.5 Pick and Place on the Mobile Robot *117*
7.4 Scheduled End-to-End Delivery *117*
7.5 Full-Mission Evaluation *117*
7.5.1 Method *117*
7.5.2 Results *118*
7.5.3 Failure Analysis *119*
7.6 Patient Room and Smart Table Tests *120*
7.6.1 Stand-Alone Table Test *120*
7.6.2 Scheduled Missions in the Patient Room *121*
7.6.3 Effect on the Simulation Speed *124*
7.7 Hospital System Tests *125*
7.7.1 Offline Tests *125*
7.7.2 Subsystem Tests in Gazebo *126*
7.7.3 Complete Delivery to the Farthest Room *127*
7.7.4 Demonstration of Four Scheduled Deliveries *127*

7.7.5 Placement Accuracy and Simulation Speed *131*
7.8 Requirement Verification *133*
7.9 Discussion *135*
7.9.1 Objectives *135*
7.9.2 Comparison with the Proposal *135*
7.9.3 Limitations *136*
8 IMPACT OF THE ENGINEERING SOLUTION **138**
8.1 Economic Impact *138*
8.2 Societal Impact *138*
8.3 Global Significance *139*
8.4 Environmental Impact *139*
8.5 Ethical Considerations *139*
8.5.1 Patient Safety and Accountability *140*
8.5.2 Privacy *140*
8.5.3 Human Contact and Employment *140*
8.5.4 Honest Reporting *140*
8.6 Other Issues *140*
8.7 Challenges and Future Trends *141*
9 CONCLUSIONS AND FUTURE WORK **142**
9.1 Summary of the Design Solution *142*
9.2 Key Challenges and Problem-Solving Approaches *143*
9.3 Acquired Technical Skills and Lessons Learned *144*
9.4 Future Work and Recommendations *144*
REFERENCES **146**
APPENDICES **149**
Appendix A: Software Repository *149*
Appendix B: User Guide *151*
B.1 Installation *151*
B.2 Running a Single Delivery *151*
B.3 Running the Scheduler *151*
B.4 Running an Evaluation *151*
B.5 Useful Commands *152*
B.6 Troubleshooting *152*
Appendix C: Selected Source Code *153*
C.1 Arm Kinematics and Pick-and-Place Planner (jetarm_pick_place.py, excerpt) *153*
C.2 Line Follower (line_follower.py) *155*
C.3 World Generator (make_hospital_world.py) *158*

C.4 Medication Container Model (medicine_container.sdf) *162*
C.5 Wrist Camera in the Robot Description (excerpt) *163*
C.6 Smart Bedside Table Controller (smart_table.py, excerpt) *163*
C.7 Hospital Route Planner (route_planner.py, excerpt) *165*
Appendix D: Evaluation Data *166*

List of Figures

Figure 1.1: Illustration of manual medication rounds and the bedside delivery gap (AI-generated illustration).....19
Figure 2.1: The TrackMed robot (JetArm Pro on the mecanum chassis) at the patient's bedside in the simulated ward.....28
Figure 3.1: Gantt chart of the original proposal plan (Spring 2026), weeks 1-18.....32
Figure 3.2: Gantt chart of the original proposal plan (Spring 2026), weeks 19-30.....33
Figure 3.3: Incremental development stages in the second semester, as actually carried out.....35
Figure 3.4: Work breakdown structure of the TrackMed project.....37
Figure 3.5: Risk matrix with the updated risk register; the realised risk R6 is shown in bold.....39
Figure 5.1: TrackMed system architecture: every mission step waits for feedback from the robot 52
Figure 5.2: Users of TrackMed and the services the system provides.....53
Figure 5.3: Hardware block diagram of the JetArm Pro on the mecanum chassis.....55
Figure 5.4: Estimated chassis footprint used in the robot model, top view.....55
Figure 5.5: Software layers of the TrackMed system.....56
Figure 5.6: ROS 2 programs, Gazebo systems and the topics between them.....57
Figure 5.7: Planar kinematic chain of the arm in the ready pose and in the grasp pose.....59
Figure 5.8: Reachable region of the tool point with the gripper pointing down.....60
Figure 5.9: Inverse-kinematics solutions across the reachable band at the grasp height.....60
Figure 5.10: Gripper finger gap versus gripper joint angle.....61
Figure 5.11: Wrist-camera geometry with the arm in the ready pose.....63
Figure 5.12: Line-following processing pipeline; a frame with red covering more than 15 % of the strips in two consecutive frames stops the robot.....63
Figure 5.13: Image strips and detected line centres (yellow) on a straight section and entering the curve.....64
Figure 5.14: Turn-round manoeuvre at the patient stop.....65
Figure 5.15: Top view of the simulated ward and patient room, with the start pose, the patient stop and the turning circle.....66
Figure 5.16: Side view of the smart bedside table beside the bed: (a) lowered to the robot's placing height, (b) raised above the mattress for the patient.....68
Figure 5.17: Planned lift profile of the platform and the measured lift times; the platform passes the mattress height after about 12.5 s.....69
Figure 5.18: State machine of the smart bedside table.....70
Figure 5.19: Delivery mission flow with its safety checks.....71
Figure 5.20: State machine of a delivery mission.....72
Figure 5.21: Sequence diagram of one delivery.....73

Figure 5.22: Scheduler processing loop.....74
Figure 5.23: Top view of the hospital system: corridor, pharmacy, four patient rooms and the line network, with the clearance checks.....78
Figure 5.24: The simulated hospital in Gazebo, with the corridor, the pharmacy and the four furnished patient rooms.....79
Figure 5.25: The medicine storage unit: six medicine containers with different caps on the pick counter, with the robot lined up at a slot.....80
Figure 5.26: The line network as a graph, with the length of each edge.....81
Figure 5.27: Handling of markers by the route follower.....82
Figure 5.28: Processing loop of the hospital scheduler.....85
Figure 6.1: Gazebo's sample world rendered with the OGRE 1 renderer after the blank-view fix 87
Figure 6.2: Physics test: a sphere falls under gravity and comes to rest beside the arm.....88
Figure 6.3: The JetArm Pro model on a fixed base in Gazebo, all joints at zero.....89
Figure 6.4: The medication container spawned beside the arm.....89
Figure 6.5: First grasp attempt: the container held near the cap and tilted.....90
Figure 6.6: Side view of the gripper at the grasp pose without a container.....91
Figure 6.7: Second attempt: the fingers grip only the top rim of the container.....91
Figure 6.8: Final grasp height: the pads hold the container around its middle.....92
Figure 6.9: Assisted grasp: the container carried in the gripper.....93
Figure 6.10: Assisted grasp: the container released at the drop-off position, standing upright…93
Figure 6.11: The generated hospital world in Gazebo: path, stop markers, medicine station, bedside table and bed.....95
Figure 6.12: The robot at its start position on the home marker, beside the medicine station.....96
Figure 6.13: Automated evaluation pipeline for one run; the pipeline repeats for each run.....98
Figure 6.14: The furnished patient room loaded in the Gazebo graphical interface.....100
Figure 6.15: The patient room from the overview camera, with the robot at its start position and the smart table lowered beside the bed.....101
Figure 6.16: The same view with the smart table raised, presenting the container beside the bed 101
Figure 6.17: The raised smart table seen across the bed from the patient's side, with the container at bed height.....103
Figure 6.18: The corridor seen from a room door, with the blue line, a green junction marker and a room's branch line.....105
Figure 6.19: Room 1, with the stop marker and the smart bedside table lowered beside the bed 106
Figure 6.20: The six caps as seen by the wrist camera during the cap test, all classified correctly 107

Figure 6.21: Frame of the demonstration video at its end: the robot parked at home and all four tables raised, with the caption and the wrist-camera inset.....109

Figure 7.1: Offline closed-loop test: robot paths and distance from the path centre.....112

Figure 7.2: Effect of the camera frame rate on the offline line-following test.....113

Figure 7.3: Measured base speed for a 0.08 m/s command.....114

Figure 7.4: Real-time factor of the simulation in different configurations.....115

Figure 7.5: The robot following the blue path towards the curve.....116

Figure 7.6: The robot stopped at the patient's marker beside the bedside table.....116

Figure 7.7: Outcome of each evaluation run and placement error of the successful runs.....118

Figure 7.8: Cause of the failures in evaluation 1 and the fix.....119

Figure 7.9: Close-up of the smart table with the container: (a) lowered after the place, (b) raised beside the mattress.....121

Figure 7.10: Scheduled mission in the patient room, part 1: (a) arm to the ready pose, (b) pick at the medicine station, (c) following the line, (d) arrival at the table, (e) place on the lowered table, (f) the table lifting the medicine (platform at 0.41 m).....123

Figure 7.11: Scheduled mission in the patient room, part 2: (a) table raised beside the bed, (b) turning round, (c) following the line home, (d) parked at the start.....124

Figure 7.12: Real-time factor of the original world and of the world with the patient room and the smart table.....125

Figure 7.13: Timeline of the demonstration on the hospital clock: the phases of each delivery, the due times and the lateness.....128

Figure 7.14: First delivery of the demonstration: (a) line-up at slot S1, (b) cap check (white), (c) pick, (d) right turn at J1, (e) the table in Room 1 lifting the medicine, (f) the robot parked at home after all four deliveries, with all tables raised.....129

Figure 7.15: Junction turns in the demonstration: (a) right at J1 to Room 1, (b) right at J2 to Room 3, (c) left at J2 to Room 4, (d) left at J1 to Room 2.....130

Figure 7.16: The smart tables of (a) Room 1, (b) Room 3, (c) Room 4 and (d) Room 2 lifting the delivered medicine.....131

Figure 7.17: Placement error of every successful scored delivery: both evaluations, the first scheduled delivery, the patient-room missions, the first hospital delivery before the fine line-up and both demonstration attempts.....132

Figure 7.18: Real-time factor of the hospital system and of the single-room world, measured alternately.....132

List of Tables

Table 2.1: Technologies available in automated medication delivery....25
Table 3.1: Milestones assigned per week....31
Table 3.2: Detailed budget breakdown for the TrackMed system....34
Table 3.3: Work breakdown structure....37
Table 3.4: Risk calculation table....39
Table 3.5: Risk mitigation strategies and outcomes....40
Table 4.1: Functional requirements....43
Table 4.2: Non-functional requirements....45
Table 4.3: Relevant standards....49
Table 5.1: Kinematic parameters of the JetArm Pro (from the manufacturer's URDF)....54
Table 5.2: Main ROS 2 topics used by TrackMed....58
Table 5.3: Fields of the delivery log (deliveries.csv)....75
Table 5.4: Inventory of the medicine storage unit....80
Table 5.5: Routes from the pharmacy to each room and back home, as planned by the route planner....81
Table 5.6: Classification of the cap colours (OpenCV HSV, hue 0-180, saturation and value 0-255)....83
Table 6.1: Environment problems and their solutions....87
Table 6.2: Problems met while building the patient room and the smart table....104
Table 6.3: Problems met while building the hospital system....110
Table 7.1: Test levels....111
Table 7.2: Results of the offline line-following test....113
Table 7.3: Line-following runs in Gazebo....115
Table 7.4: Result of the scheduled delivery....117
Table 7.5: Results of the two 5-run evaluations....118
Table 7.6: Results of the stand-alone smart table test....121
Table 7.7: Results of the two scheduled missions in the patient room....122
Table 7.8: Offline tests of the hospital system....126
Table 7.9: Gazebo tests of the hospital system....127
Table 7.10: Results of the demonstration (attempt 2)....128
Table 7.11: Junction events of the demonstration, with the simulated time at which each marker was seen (s)....128
Table 7.12: Requirements verification....134
Table A.1: Contents of the TrackMed repository....150
Table B.1: Useful commands....152

Table D.1: Per-run evaluation data.....167
Table D.2: Missions in the patient room with the smart table.....167
Table D.3: Deliveries of the hospital-system demonstration.....168

List of Abbreviations

LIST OF SYMBOLS

q1 ... q5	Arm joint angles (rad)
L2, L3	Lengths of the second and third arm links (m)
LT	Distance from the wrist joint to the tool point (m)
θ	Deflection angle of the floor line in the camera image (rad)
v, ω	Forward velocity (m/s) and angular velocity (rad/s) of the chassis
Kp, Kd	Proportional and derivative gains of the steering controller

LIST OF ABBREVIATIONS

AED	United Arab Emirates dirham
CSV	Comma-separated values
DART	Dynamic Animation and Robotics Toolkit (Gazebo physics engine)
DOF	Degree of freedom
ECG	Electrocardiogram
FR / NFR	Functional / non-functional requirement
HRI	Human-robot interaction
HSV	Hue, saturation, value (colour space)
I²C	Inter-integrated circuit (serial bus)
IK	Inverse kinematics
IV	Intravenous
JSON	JavaScript Object Notation
LiDAR	Light detection and ranging
OGRE	Object-Oriented Graphics Rendering Engine
PD	Proportional-derivative (controller)
PID	Proportional-integral-derivative (controller)
RGB	Red, green, blue (colour image)
ROS 2	Robot Operating System 2
RTF	Real-time factor (simulated time divided by wall-clock time)
SDF	Simulation Description Format
SDP	Senior Design Project
SLAM	Simultaneous localisation and mapping
TLS	Transport Layer Security
UAE	United Arab Emirates
URDF	Unified Robot Description Format
VM	Virtual machine
WBS	Work breakdown structure
WHO	World Health Organization

YAML	YAML Ain't Markup Language (configuration file format)

ABSTRACT

Patients on fixed medication schedules, particularly elderly and mobility-impaired patients, depend on nurses to bring each dose to the bedside, and nurses spend over an hour of every shift on medication administration. This project presents TrackMed, a mobile robot that, at a scheduled time, collects a medication container, follows a marked floor path to the patient, places the container on a smart bedside table that lifts it to the patient in bed, and returns to its starting point.

The system combines a Hiwonder JetArm Pro arm (five joints and a gripper) with a mecanum-wheel chassis. It navigates by colour line following with the arm's wrist camera, ported from the manufacturer's algorithm, grasps with a closed-form inverse-kinematics solution derived from the arm's URDF model, and confirms every mission step against feedback before continuing, stopping when a check fails. A scheduler decides from the medication schedule and the clock which medicine to fetch and where to take it, and logs every result.

The physical robot failed during development, so the system was implemented and evaluated in a ROS 2 Humble and Gazebo Fortress simulation built from the manufacturer's robot model. In a single-room ward, a first 5-run evaluation succeeded in 2 runs; the failures were traced to lost attach and release commands, and once these were confirmed and retried, 5 of 5 runs succeeded, with a mean placement error of 0.19 mm and a cycle of about 160 simulated seconds. A furnished patient room followed, with a smart table that lifts the container from 0.10 m to 0.80 m in 18.6 s (2 of 2 missions). Finally, a small hospital with a corridor, a storage unit for six medicines and four patient rooms was built: the robot checks each container's cap colour, plans its route with Dijkstra's algorithm and handles the junctions. In a demonstration of four scheduled doses, run after an earlier attempt had failed and been fixed, all four were delivered to the right rooms, each in 181 to 239 simulated seconds and placed within 0.6 to 2.0 mm of the target.

The results show, in simulation, that a low-cost educational manipulator on a mobile base can automate bedside medication delivery in a structured environment. Because the simulation has no sensor noise and the samples are small, validation on the physical robot, friction grasping and obstacle detection remain necessary before clinical testing.

Keywords: assistive robotics, medication delivery, smart bedside table, hospital logistics, route planning, mobile manipulation, line following, inverse kinematics, ROS 2, Gazebo simulation, healthcare automation

Source code, videos and documentation: https://github.com/Seif-Taha12903478/trackmed-medication-delivery-robot

1 INTRODUCTION

1.1 Problem Statement

Patients who require strict medication schedules, particularly the elderly, the chronically ill and patients with limited mobility, often fail to take the prescribed dose at the correct time. The World Health Organization (WHO) reports that adherence to long-term therapies for chronic diseases averages only 50 % in developed countries, and is lower still in developing countries [1]. Medication errors are a separate but related problem: WHO estimates their global cost at US$ 42 billion a year, almost 1 % of total global health expenditure, and in 2017 it launched a global challenge to halve severe, avoidable medication-related harm [2].

In hospitals and care homes, bringing each dose to the patient is a nursing task that competes with direct care. A 36-hospital time-and-motion study found that medical-surgical nurses spent 72 minutes of each shift, 17.2 % of their nursing-practice time, on medication administration [3]. At the same time, the WHO estimates a global shortage of 5.9 million nurses [4], and by 2030 one in six people in the world will be aged 60 years or over [5]. More patients will need help with medication while fewer staff are available to give it.

Existing solutions address only part of this problem. Stationary smart pill dispensers remind the patient and release the correct dose, but the patient must still get up and reach the device. Hospital delivery robots move medicines between the pharmacy and the wards, but they stop at the nurses' station, and the final step to the patient's bedside is still done by a person. There is therefore a need for a system that both follows a medication schedule and physically places the correct dose within the patient's reach, without a person in the loop for routine deliveries. Figure 1.1 illustrates this gap between manual medication rounds and delivery to the bedside.



Figure 1.1: Illustration of manual medication rounds and the bedside delivery gap (AI-generated illustration)

1.2 Purpose of the Study

The purpose of this study is to design, implement and evaluate **TrackMed**, a mobile robot that automates scheduled bedside medication delivery. At the time given in a medication schedule, the robot collects a medication container from a medicine station, travels to the patient along a marked path, places the container upright on the bedside table and returns to its starting point, ready for the next delivery.

The study uses a Hiwonder JetArm Pro, an educational robotic arm with a depth camera that is sold as six-degree-of-freedom (6-DOF): it has five revolute joints plus a gripper. The arm is mounted on a Hiwonder mecanum-wheel chassis. The aim is to show that a low-cost, openly programmable manipulator on a mobile base can carry out the complete "last metre" of bedside delivery, and to measure how reliably and accurately it does so. The study also shows how the work can continue, with its objectives intact, when the physical robot becomes unavailable: the full system was implemented and evaluated in a simulation built from the manufacturer's own robot model.

1.3 Project and Design Objectives

The primary objective of TrackMed is a working, measured demonstration of scheduled bedside medication delivery by a mobile manipulator. This objective is divided into eight design objectives, each of which is verified in Chapter 7:

1. **O1 Navigation.** Drive from the medicine station to the patient's bedside and back along a marked floor path, and stop within reach of the bedside table.
2. **O2 Manipulation.** Pick a medication container from the station and place it upright on the bedside table using the arm.
3. **O3 Scheduling.** Start each delivery automatically at the time given in a schedule of patients and medications, and record the outcome of every delivery.
4. **O4 Safety and verification.** Detect a failed grasp or release and stop safely, instead of continuing the mission with the container in the wrong place.
5. **O5 Evaluation.** Measure success rate, placement accuracy, container tilt and cycle time over repeated, independent runs.
6. **O6 Transferability.** Structure the software so that it can move from simulation to the physical robot with changes limited to the hardware interfaces.
7. **O7 Patient access.** Present the delivered medication within reach of a patient lying in bed, without changing the robot's low placing height, by a bedside table that detects the container and lifts it.
8. **O8 Hospital-scale operation.** Serve several patient rooms with different medicines: choose the medicine due from a schedule and the current time, check the container's cap against the inventory, fetch it from a medicine storage unit and navigate to the right room.

The design is centred on the JetArm Pro and the mecanum chassis supplied with it. Navigation uses colour line following with the arm's wrist camera, which the manufacturer supports on this chassis [6]. Line following was chosen over LiDAR-based mapping because the platform has no LiDAR, and because a marked route is predictable and easy to verify in a ward where the delivery path is fixed. Manipulation uses a closed-form inverse-kinematics solution derived from the arm's geometry, so that every motion can be computed and checked in advance.

Safety and reliability objectives shape every part of the design. The robot moves slowly (0.08 m/s near markers, junctions and furniture, and at most 0.12 m/s on long, clear corridor straights), never rotates in place close to furniture, and checks the real state of the container after each critical step. Scalability and usability are addressed by keeping the schedule in a simple, human-readable file and by logging every delivery, so that a caregiver can see what was delivered, when, and whether it succeeded.

1.4 Intended Outcomes and Deliverables

The primary intended outcome of the project is a functional mobile robotic system that delivers medication to a patient's bedside at scheduled times, with its performance measured. The system integrates the JetArm Pro arm and depth camera with a mecanum chassis, line-following navigation, inverse-kinematics-based pick-and-place and a scheduler. Because the physical robot failed during development, the deliverables were completed and evaluated in a high-fidelity simulation that uses the manufacturer's robot model; the software is structured so that it can be moved to the repaired or replaced robot.

The key deliverables are the complete simulated system, its source code under version control, the evaluation results, a demonstration video and this report. Each deliverable is described below.

1.4.1 Mobile Robotic Platform with Line-Following Navigation

A key deliverable is a model of the JetArm Pro mounted on the mecanum chassis that can travel from the medicine station to the patient's bedside and back. The robot follows a blue floor line using the arm's wrist camera, steers with a proportional-derivative controller, stops at red floor markers and turns round beside the bedside table without touching it. In the final hospital system the same robot travels from a pharmacy to any of four patient rooms over a network of lines, choosing its route at marked junctions. The mecanum wheels allow the base to move sideways, which is used to keep clear of furniture when turning.

1.4.2 Scheduled Dispensing and Delivery Management

The scheduling deliverable reads a medication schedule listing each patient, medicine, bed and delivery time, starts the delivery mission when each entry falls due, and records the result. Each delivery is written to a log with the due time, start and finish times, success or failure, placement error and the number of attempts each gripper command needed. The proposal's three-compartment cup holder was replaced by a medicine station from which the arm picks the prepared container, which keeps the chassis free and lets one design serve any number of doses.

1.4.3 Robotic Arm Manipulation for Container Delivery

The manipulation deliverable picks the container from the medicine station and places it upright on the bedside table. Joint angles are computed from the arm's geometry with a closed-form inverse-kinematics solver, and descents and lifts are split into several waypoints so that the gripper moves nearly vertically. Every grasp and release is confirmed, and the container's true position is checked before the robot moves on. Because the arm places at 0.10 m, the height of its own base plate, while a patient in bed reaches at about mattress height, the container is placed on a smart bedside table. A

contact sensor in the table's platform detects the container, and once the robot's arm has withdrawn, a telescopic column lifts the platform to 0.80 m, beside the patient's hand.

1.4.4 Evaluation Platform, Documentation and Interface

The evaluation deliverables are a hospital-ward simulation with a furnished patient room (hospital bed with side rails, IV pole, patient monitor, visitor chair and the smart bedside table), a hospital-system simulation with a corridor, a medicine storage unit holding six medicines and four such patient rooms connected by a network of marked lines, and an automated test script that runs complete missions from a fresh start, scores each run from the simulator's ground truth and writes the results to files. It is accompanied by demonstration videos, including one of four scheduled deliveries to the four rooms, the public source code repository (https://github.com/Seif-Taha12903478/trackmed-medication-delivery-robot) and this report. The mobile application and notification interface proposed in the first semester are specified as future work (Section 9.4), with the delivery log already providing the data such an interface would display.

1.5 Change of Platform and Scope

The Spring 2026 proposal described a JetRover-based robot with LiDAR mapping and three cups carried on the chassis. During the second semester the project was built on the platform that was actually available: the JetArm Pro with its mecanum chassis. This changed the navigation method from LiDAR mapping to camera line following, and the dispensing method from three on-board cups to one container collected from a medicine station.

Midway through development the physical robot failed and could not be repaired within the project timeline. This risk had been identified in the proposal's risk register (mechanical failure of the arm or components, Table 3.4). The team responded by moving implementation and testing into Gazebo, using the manufacturer's URDF model of the arm, the same controllers and the same camera placement as the hardware. Section 3.2 describes this decision in detail. All six objectives were kept; only physical validation moved to future work. A seventh objective, presenting the medication within the patient's reach with a smart bedside table, was added at the end of the semester once the delivery itself had been evaluated. At the supervisor's request the system was then extended into a small hospital system with four patient rooms, a medicine storage unit and a scheduler that chooses the medicine and the room by itself, which added an eighth objective.

1.6 Report Organization

Chapter 2 reviews existing approaches to medication management and delivery and the enabling technologies used in this project. Chapter 3 presents project management: tasks, schedule,

milestones, budget and risk management, including the hardware failure. Chapter 4 specifies the functional and non-functional requirements, constraints and standards. Chapter 5 presents the system analysis and design, and Chapter 6 its implementation. Chapter 7 describes the tests and results and verifies each requirement. Chapter 8 discusses the economic, societal, environmental and ethical impact, and Chapter 9 concludes the report and outlines future work. The appendices contain the repository structure, a user guide, selected source code and the evaluation data.

2 BACKGROUND

2.1 Existing Approaches/Applications

Automated support for medication management has grown with the rise of smart devices and the pressure on healthcare staff. Existing approaches fall into four groups: manual administration by nurses and caregivers, stationary smart dispensers, hospital logistics robots, and research and educational mobile manipulators. Each solves part of the problem, but none combines time-based dispensing, travel to the patient and placement at the bedside in one low-cost system.

2.1.1 Manual Administration

In most hospitals and care homes, nurses prepare doses from a medication cart or medicine room and carry them to each patient at scheduled times. This approach is flexible and keeps a clinician at the bedside, which is important for assessing the patient and confirming that the dose is taken. However, it is labour-intensive. Medical-surgical nurses spend about 72 minutes per shift on medication administration [3], and in home-care settings, where continuous supervision is not available, doses are easily missed. With a projected global shortage of 5.9 million nurses [4], the time spent walking doses to the bedside is time taken from direct care.

2.1.2 Stationary Smart Pill Dispensers

Stationary dispensers store several medications, alert the patient when a dose is due and release it automatically. The Hero dispenser, for example, holds up to a 90-day supply of 10 medications, signals with a tone and a flashing light when a dose is due, releases it at the press of a button and sends missed-dose alerts to caregivers through a companion app [7]. Devices of this kind improve adherence for patients who can reach them, and their connectivity gives caregivers remote oversight. They remain stationary, however: a patient who is bedridden, recovering from surgery or unsteady on their feet must still get to the device, or rely on someone to bring the dose.

2.1.3 Hospital Logistics Robots

Autonomous mobile robots already carry medicines and supplies in many hospitals. Aethon's TUG transports medication, meals, linen and equipment through hospitals; by early 2014 over 400 TUG robots were operating in U.S. hospitals, making more than 50,000 deliveries a week [8]. Diligent Robotics' Moxi performs last-mile pharmacy deliveries, including restricted and controlled medications, with time-stamped proof of delivery; by July 2025 it had completed over 300,000 pharmacy deliveries and over 1.1 million deliveries in total across more than 30 U.S. health systems [9]. A survey of robots in healthcare lists such hospital robots alongside care, assistive and rehabilitation robots, and notes their role in reducing the routine workload of staff [10].

These robots deliver to wards, units and nurses' stations rather than to the patient's bedside table. They are also expensive, purpose-built platforms that require integration with hospital infrastructure, which puts them out of reach of small clinics, care homes and home care.

2.1.4 Educational and Research Mobile Manipulators

Low-cost educational platforms combine a small arm, a camera and open software. The Hiwonder JetArm Pro used in this project has an arm sold as 6-DOF (five revolute joints plus a gripper), an Orbbec Gemini depth camera and an NVIDIA Jetson computer running ROS 2 [11]. It can be mounted on a mecanum-wheel chassis, for which the manufacturer provides target-tracking and line-following programs [6]. Such platforms are affordable and fully programmable, but they are designed for teaching and are not developed or validated for healthcare tasks.

Approach	Description	Example	Reaches the bedside	Main limitation
Manual administration	Nurses carry doses to each patient at scheduled times	Medication cart rounds	Yes, by a person	Labour-intensive; 72 min per shift [3]
Stationary smart dispenser	Fixed device stores and releases doses with alerts	Hero [7]	No	Patient must walk to the device
Hospital logistics robot	Autonomous robot transports supplies between departments	TUG [8]	No, stops at wards and stations	Last metre still done by staff; high cost
Pharmacy delivery robot	Robot delivers medications to units with proof of delivery	Moxi [9]	No, hands over to staff	Purpose-built and expensive
Educational mobile manipulator	Arm, camera and open software on a wheeled base	JetArm on mecanum chassis [6]	Possible	Not designed for healthcare use
TrackMed (this project)	Scheduled pick from a pharmacy, route to the right room, place on a lifting bedside table	JetArm Pro + mecanum chassis	Yes	Structured environment; validated in simulation

Table 2.1: Technologies available in automated medication delivery

Table 2.1 shows a clear gap. Stationary dispensers manage the schedule but cannot move, and hospital robots move but leave the final hand-over to a person. TrackMed combines the schedule-keeping of a dispenser with the mobility of a delivery robot and the manipulation of an arm, using an affordable platform.

2.2 Enabling Technologies

2.2.1 Robot Operating System 2

The Robot Operating System 2 (ROS 2) is the standard open-source middleware for robot software. A ROS 2 system is a set of programs called nodes that exchange typed messages over named topics, call services, and run long actions. ROS 2 is built on the Data Distribution Service standard and adds support for real-time behaviour, security and multi-robot systems; it is used in research, education and commercial products [12]. TrackMed uses ROS 2 Humble, the long-term-support release on which the JetArm Pro software and its simulation packages are based.

The `ros2_control` framework connects ROS 2 controllers to hardware or to a simulator through a common interface [13]. TrackMed uses its joint-trajectory controllers for the arm and gripper, so the same commands drive the simulated arm and the real servos.

2.2.2 Gazebo Simulation

Gazebo is an open-source robot simulator that combines a physics engine, sensor simulation and 3-D rendering, and integrates closely with ROS [14]. A robot is described in URDF or SDF, with its links, joints, inertia, collision shapes and visual meshes, and plugins add behaviour such as controllers, cameras or joint locks. TrackMed uses Gazebo Fortress (version 6), which pairs with ROS 2 Humble. Two Gazebo systems are central to the design: the Sensors system, which renders the wrist camera's images, and the DetachableJoint system, which creates and removes a fixed joint between two models on command and publishes its state [15].

2.2.3 Mecanum-Wheel Mobile Bases

A mecanum wheel carries rollers mounted at 45° to its axis. When four such wheels are driven at suitable speeds, the base can move forward, sideways and rotate independently, which is called omnidirectional or holonomic motion. Each wheel rolls without slipping in its driving direction and slides freely along its rollers, and the chassis velocity (forward, sideways and yaw) maps linearly to the four wheel speeds [16]. For TrackMed, sideways motion is valuable because the robot can step away from a table before turning, instead of sweeping its corners across it.

2.2.4 Robot Arm Kinematics

Forward kinematics computes the position and orientation of a robot's tool from its joint angles; inverse kinematics (IK) solves the reverse problem. For general arms, IK is solved numerically, but arms whose shoulder, elbow and wrist joints are parallel, like the JetArm, reduce to a planar two-link problem that has a closed-form solution [16]. A closed-form solution is fast, deterministic and

returns every valid configuration, which makes it easy to choose the one furthest from the joint limits.

2.2.5 Vision-Based Line Following

Line following is a classical navigation method for mobile robots and automated guided vehicles: the robot detects a marked path with a sensor and steers to keep it centred. With a camera, the path is separated from the floor by colour thresholding and its position in the image gives a steering error, which a PID controller converts into a turn rate. Camera line following needs no map and no expensive sensors, and it makes the route explicit and easy to inspect. OpenCV, the open-source computer-vision library used in this project, provides the colour-space conversion, thresholding, morphology and contour tools it needs [17]. Hiwonder's own line-following program for the JetArm on the mecanum chassis uses exactly this approach: three regions of interest near the bottom of the image, a weighted centre of the line, and a PID steering controller [6].

2.2.6 Simulation and the Sim-to-Real Gap

Developing robot software in simulation is fast, safe and repeatable, but the simulated world is never identical to the real one. Differences in friction, sensor noise, lighting, actuator dynamics and dimensions cause a controller that works in simulation to perform worse on hardware; this is known as the sim-to-real gap [18]. Common ways to reduce it include using the manufacturer's accurate robot models, keeping the same controllers and interfaces in simulation and on hardware, adding noise and variation to the simulation, and validating on hardware as early as possible. TrackMed applies the first two of these and lists the others as future work.

2.3 Discussion

The review shows that the individual building blocks of bedside medication delivery already exist and are well proven: schedules and reminders in smart dispensers, autonomous transport in hospital robots, and manipulation and navigation on educational platforms. What is missing is their combination in a low-cost system that completes the final metre to the patient's table.

TrackMed's design follows from this gap and from the platform available. Instead of requiring the patient to approach a fixed device, the robot brings the dose to the patient, which improves accessibility for patients with limited mobility. Instead of a purpose-built hospital robot, it uses an open educational arm and chassis (Figure 2.1), which lowers the cost of trying the idea in small clinics, care homes and teaching labs. Line following provides a predictable, inspectable route that suits a ward where the path from the medicine room to each bed is fixed.

Several challenges were identified at the design stage. First, a mobile manipulator in a patient's room must never continue a task in an unsafe state, for example by driving off while still holding a container it should have released; the design therefore confirms every step against feedback. Second, placing a small container on a table requires accurate arm motion and a reliable grasp. Third, scheduling must be dependable and transparent, so that caregivers can see what happened to every dose. Finally, because the project was validated in simulation, the sim-to-real gap must be stated openly and addressed in future work.

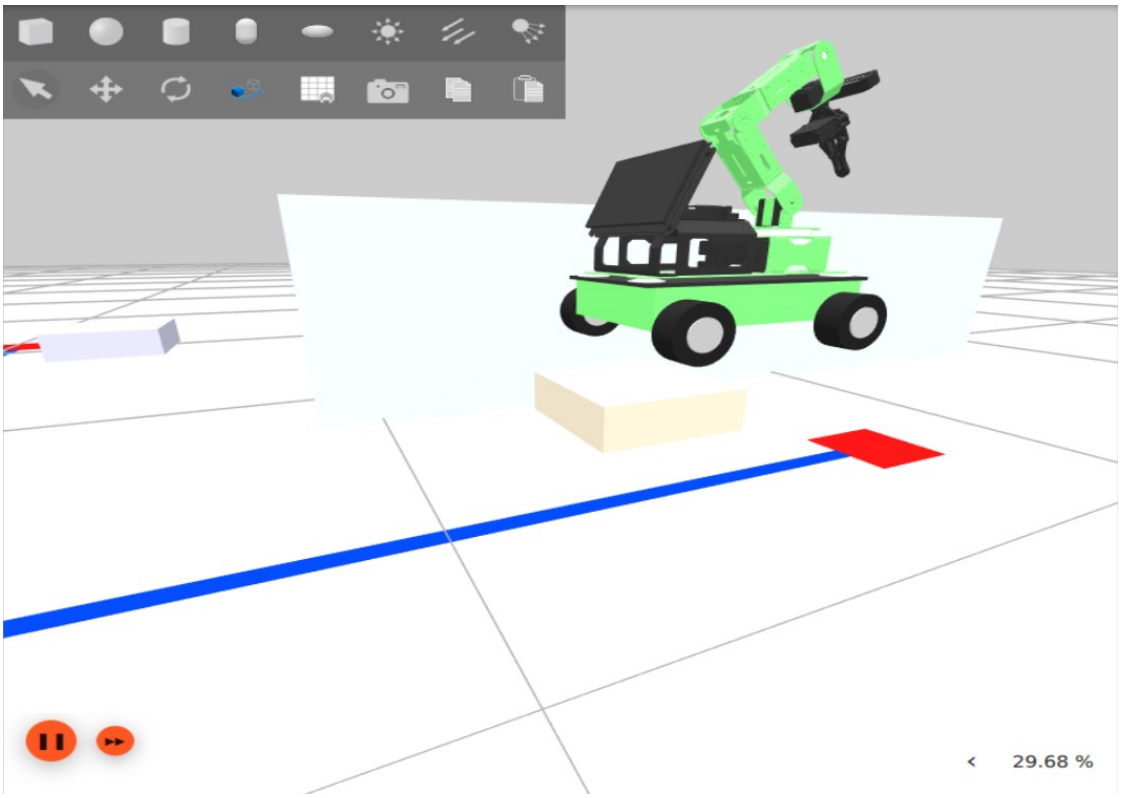


Figure 2.1: The TrackMed robot (JetArm Pro on the mecanum chassis) at the patient's bedside in the simulated ward

In a broader context, TrackMed aligns with the shift toward smart healthcare and assistive robotics. It addresses medication adherence and nursing workload, and it lays a foundation for further automation such as delivery to several beds, patient interaction and integration with hospital information systems. The following chapter presents the project management plan, including the schedule, budget and risk management for developing and testing the TrackMed system.

3 PROJECT MANAGEMENT

Effective project management was essential for TrackMed, which combines mobile robotics, manipulation, computer vision and scheduling in one system. Structured planning was needed to make sure each subsystem was developed, integrated and validated within the two-semester timeline, and structured risk management proved decisive when the physical robot failed midway through development. This chapter presents the plan, how it was carried out, the budget, and the risks with their mitigation.

Managing TrackMed required more than technical work. Every stage, from platform selection and environment setup to inverse kinematics, line following, mission control and evaluation, depended on earlier stages and on the availability of the hardware and computing resources. The team therefore planned the work in phases with clear milestones, reviewed progress against them regularly, and re-planned when the hardware situation changed.

3.1 Tasks, Schedule, and Milestones

The project follows a 30-week timeline consisting of research, design, integration, software development, testing, evaluation and documentation. Early tasks covered requirements and component selection; later tasks covered navigation, manipulation, scheduling and their integration; and the final weeks covered testing, evaluation and this report. Key milestones were requirements completion, design approval, integration, validation of navigation and dispensing, pilot evaluation, and final submission.

3.1.1 Tasks

The development of TrackMed was divided into seven major tasks, each addressing a specific part of the overall objective of autonomous bedside medication delivery:

1. **Project initiation and research.** Review existing medication-management systems and healthcare robots, identify requirements, define the scope, and select the platform, software and communication approach. Assess the feasibility of combining navigation, manipulation and scheduling on the selected hardware.
2. **System design.** Develop the system architecture, define the data flow between the scheduler, mission controller and robot, design the navigation method and the pick-and-place motions, and define the delivery sequence and its checks.

3. **System development.** Configure the robot software and simulation environment, build the robot and world models, implement the arm control and inverse kinematics, the line follower and base motions, the mission controller and the scheduler.
4. **System integration.** Combine navigation, manipulation and scheduling into a complete delivery mission and verify that the components exchange data correctly and in time.
5. **System testing.** Test each subsystem and then the complete mission: navigation accuracy, base speed, arm accuracy, grasp and release, and the full delivery cycle.
6. **Deployment and evaluation.** Run complete scheduled deliveries in the hospital-ward environment and evaluate reliability, accuracy and cycle time over repeated independent runs.
7. **Documentation and final presentation.** Prepare the technical documentation, user guide, demonstration video and this report.

These tasks together ensure that TrackMed is developed systematically, from requirements to a measured, documented system.

3.1.2 Schedule

The schedule allocated time to each phase so that each subsystem could be validated before the next depended on it:

- **Project initiation and requirements analysis** (weeks 1-3)
- **System design and architecture** (weeks 4-7)
- **Hardware integration and robot configuration** (weeks 8-14)
- **Software development and scheduling implementation** (weeks 15-22)
- **System testing and validation** (weeks 23-26)
- **Deployment and evaluation** (weeks 27-29)
- **Final documentation and presentation** (week 30)

The integration phase received particular attention, because navigation, manipulation and scheduling must work together for a delivery to succeed. After the robot failed during the hardware-integration phase, the remaining phases were carried out in simulation without changing their order or their milestones (Section 3.2).

3.1.3 Milestones

Table 3.1 lists the seven planned phases with their start and end weeks, durations and milestones, as in the proposal's schedule. It also records the outcome of each phase, including where the hardware failure changed how its milestone was met.

Task	Start	End	Length	Milestone	Outcome
Project initiation and requirements	Week 1	Week 3	3 weeks	Completion of system requirements	Completed in the Spring 2026 proposal
System design and architecture	Week 4	Week 7	4 weeks	Architecture and navigation design approved	Approved; revised for the JetArm Pro and mecanum chassis
Hardware integration and robot configuration	Week 8	Week 14	7 weeks	Robot integrated and configured	Robot failed; integration completed in simulation
Software development and scheduling	Week 15	Week 22	8 weeks	Delivery mission and scheduler implemented	Completed in simulation (Chapter 6)
System testing and validation	Week 23	Week 26	4 weeks	Navigation and dispensing validated	Validated in simulation (Chapter 7)
Deployment and evaluation	Week 27	Week 29	3 weeks	Pilot testing completed	Two 5-run evaluations, patient-room missions and the four-room hospital demonstration
Final documentation and presentation	Week 30	Week 30	1 week	Final SDP submission and demonstration	This report, videos and repository

Table 3.1: Milestones assigned per week

1. **Completion of system requirements (week 3).** The project-initiation phase ends with all functional and non-functional requirements identified and documented, covering navigation, manipulation, scheduling and safety.
2. **Architecture and navigation design approved (week 7).** The system architecture, the navigation method and the pick-and-place approach are finalised and approved by the supervisor.
3. **Robot integrated and configured (week 14).** The arm, chassis, camera and software are integrated. This milestone was met in simulation after the hardware failure.
4. **Delivery mission and scheduler implemented (week 22).** The scheduler can start a complete delivery mission.
5. **Navigation and dispensing validated (week 26).** Navigation accuracy and reliable pick-and-place are verified under controlled conditions.
6. **Pilot testing completed (week 29).** Complete scheduled deliveries are run and evaluated over repeated independent runs.

7. **Final submission and demonstration (week 30).** The report, repository and demonstration video are submitted and presented.

The Gantt charts in Figures 3.1 and 3.2 are the plan of the Spring 2026 proposal, kept here as the baseline against which the project was managed. They show the same seven phases and week ranges as Table 3.1, but some of their tasks belong to the proposal's design and were dropped: the LiDAR and camera set-up and SLAM navigation (replaced by camera line following, Section 1.5), the ESP32 IoT wiring and IoT scheduling module (scheduling runs on the robot's computer), cup selection (replaced by a medicine station), and the mobile-app interface and the collection of caregiver feedback (future work, Section 9.4). What was actually carried out in each phase is given in the Outcome column of Table 3.1, and the development stages of the second semester in Figure 3.3.

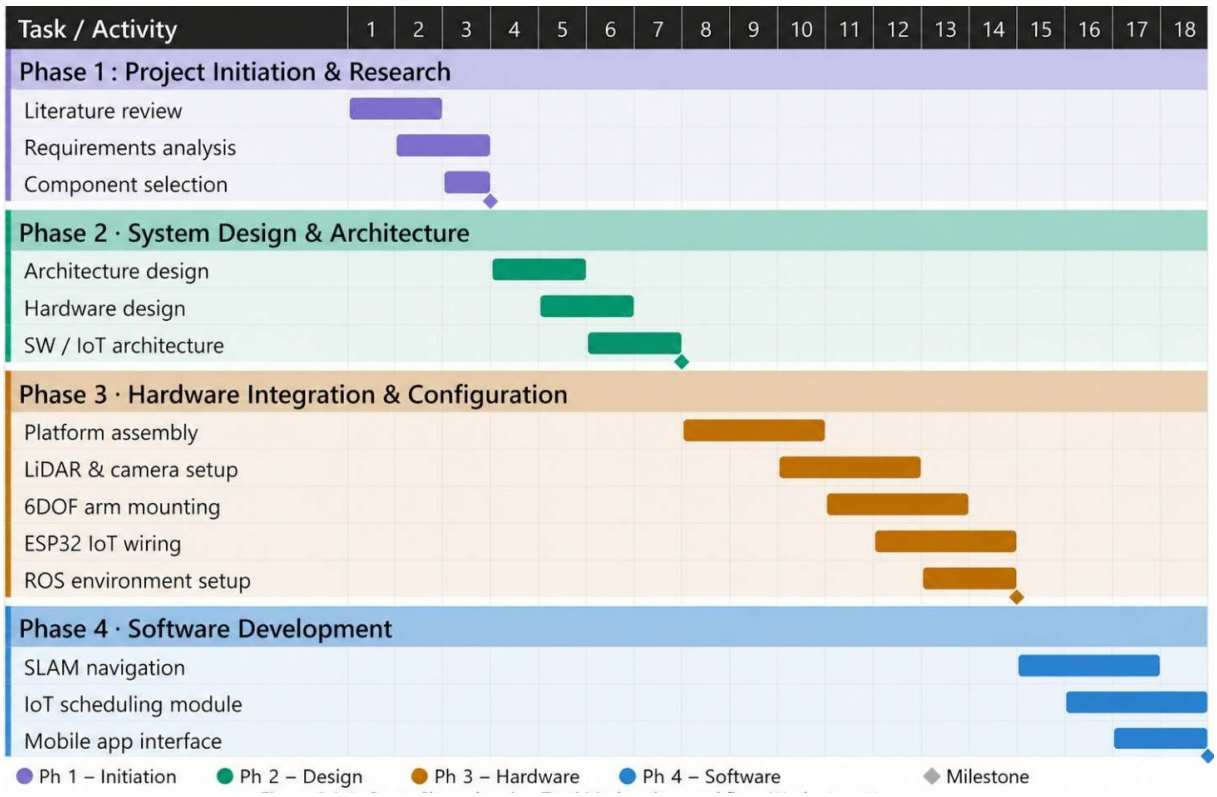


Figure 3.1: Gantt chart of the original proposal plan (Spring 2026), weeks 1-18

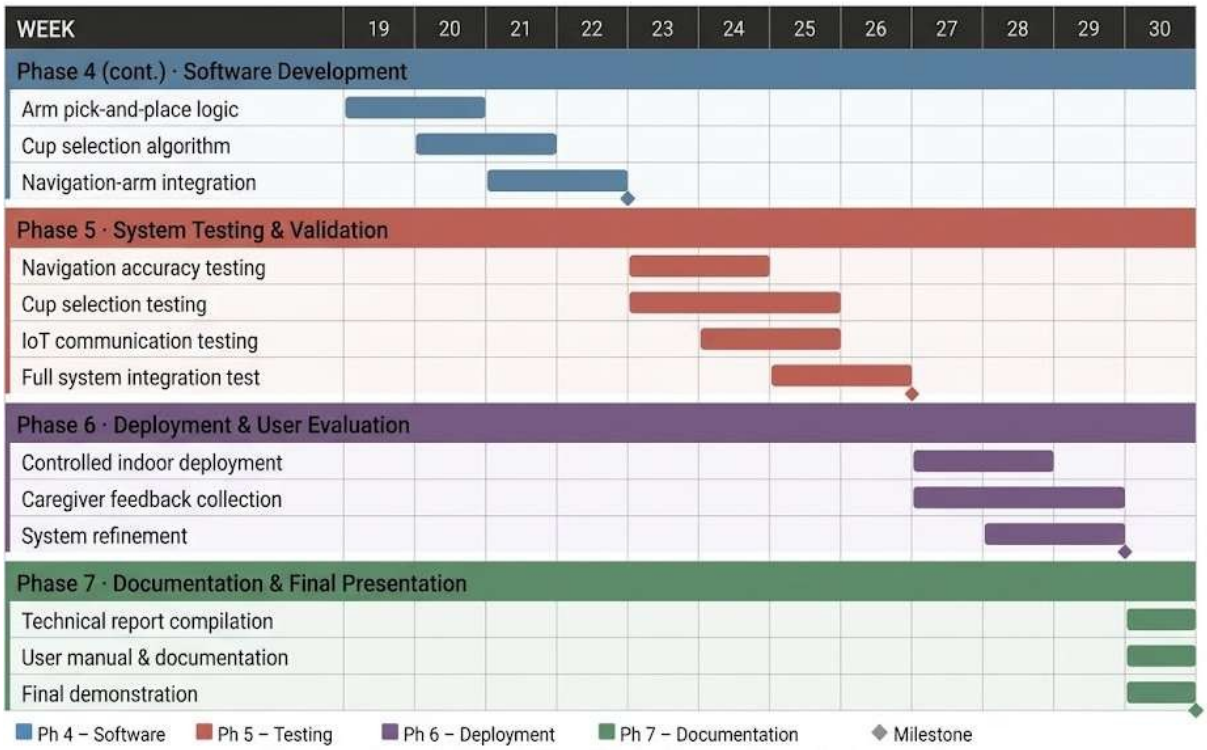


Figure 3.2: Gantt chart of the original proposal plan (Spring 2026), weeks 19-30

3.1.4 Project Charter and its Components

The project charter formally authorises the TrackMed project and sets out its vision, approach and expected outcomes. It records the agreement between the team and the supervisor on what will be delivered, by when, with which resources and under which assumptions, and it identifies the main risks. The sections below summarise each component of the charter.

3.1.5 Purpose and Objectives

The purpose of TrackMed is to develop a mobile robot that automates scheduled bedside medication delivery, improving the timeliness of doses and reducing the time caregivers spend carrying them. The objectives are the eight design objectives O1-O8 of Section 1.3: navigation, manipulation, scheduling, safety and verification, evaluation and transferability to hardware, which were set in the proposal, and patient access and hospital-scale operation, which were added during the second semester (Section 1.5).

3.1.6 Scope

The project covers the design, implementation and evaluation of a mobile manipulator that collects a medication container from a medicine station, follows a marked path to a patient's bedside, places the container on the bedside table and returns, at the times given in a schedule. It includes the robot

and environment models, arm control, navigation, mission control, scheduling, logging and an automated evaluation. It excludes clinical trials, preparation and verification of the medication itself, integration with hospital information systems, multi-robot coordination and large-scale deployment. After the hardware failure, the scope of the evaluation was limited to a high-fidelity simulation.

3.1.7 Key Stakeholders and Roles

The project involves the following stakeholders:

- **Patients (end users):** receive their doses at the bedside on time.
- **Nurses and caregivers:** prepare the containers, maintain the schedule and review the delivery log.
- **Project team:** responsible for design, implementation, integration, testing and documentation. All four team members contributed equally to the project.
- **Project supervisor:** reviews progress and design decisions, provides technical guidance and approves milestones.
- **School of Engineering:** provides the platform, laboratory and computing resources.

3.1.8 Budget and Resources

The budget is dominated by the robotic platform. All software used is open source or available free of charge, so the remaining costs are small items for testing. Table 3.2 lists the actual costs.

Category	Component / service	Details	Cost (AED)
Robotic platform	Hiwonder JetArm Pro (Advanced kit) on the mecanum-wheel chassis	Arm with five joints and a gripper (sold as 6-DOF), Orbbec Gemini depth camera, Jetson controller, four encoder motors and motor driver	4,334.69
Shipping	Robot delivery	Shipping of the robot kit	183.43
Software	Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress	Operating system, middleware, simulator	0
Software	Python 3, OpenCV, NumPy	Programming language and libraries	0
Software	VMware Workstation Pro	Virtual machine host	0
Version control	Git and GitHub (public repository)	Version history, off-site backup and publication	0
Total			**4,518.12**

Table 3.2: Detailed budget breakdown for the TrackMed system

The proposal budgeted AED 6,925, mainly for a JetRover platform, an ESP32 microcontroller, a Wi-Fi module, a cup holder and cloud services. The ESP32, Wi-Fi module, cup holder and cloud services were not needed in the final design, because scheduling runs on the robot's own computer and the container is collected from a medicine station. The actual cost of AED 4,518.12 was therefore AED 2,406.88 below the proposal's budget.

3.1.9 Timeline

The project runs over a 30-week period. The first weeks were used for research and requirements, followed by design, integration and software development, then testing, evaluation and documentation. In the second semester, the development work was organised in incremental stages, each tested and committed to version control before the next began (Figure 3.3). The seventh stage, the furnished patient room with the smart bedside table, was built on a separate branch after the 5-run evaluation and merged once it had passed its tests. The hospital system, requested by the supervisor after the patient room had been demonstrated, followed as an eighth stage on its own branch, with a written plan approved before any of it was built.

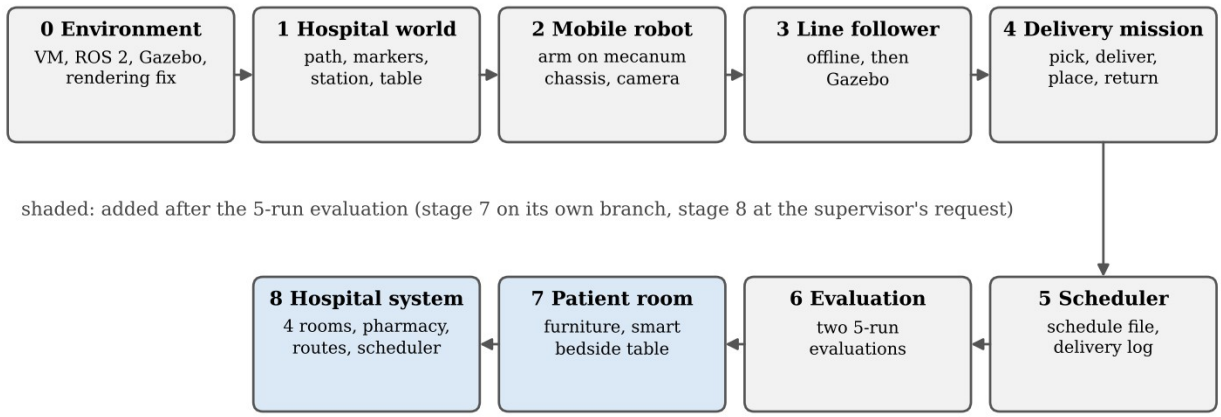


Figure 3.3: Incremental development stages in the second semester, as actually carried out

3.1.10 Key Deliverables

The tangible outputs of the project are:

- A mobile manipulator model (JetArm Pro on the mecanum chassis) with a working wrist camera, in a simulated hospital ward with a furnished patient room
- A smart bedside table that detects the delivered container and lifts it to the patient

- A hospital-system simulation with four patient rooms, a medicine storage unit with six medicines, a line network with junctions, a route planner and a scheduler that chooses the medicine and the room, with a demonstration video of four deliveries
- Line-following navigation with stop markers and a safe turn-round manoeuvre
- Inverse-kinematics-based pick-and-place with confirmed grasp and release
- A mission controller, a medication scheduler and a delivery log
- An automated evaluation tool and the evaluation results
- Source code in a version-controlled repository, a demonstration video, a user guide and this report

3.1.11 Assumptions and Constraints

The design assumes a structured indoor environment with a clearly marked floor path, fixed positions for the medicine station and the bedside table, and no obstacles on the path during a delivery. It assumes that the medication container has been prepared correctly by a nurse or pharmacist. The project was constrained by the hardware failure, which limited evaluation to simulation; by the computing resources available for simulation (a two-core virtual machine); by the unpublished dimensions of the chassis, which had to be estimated; and by the two-semester timeline.

3.1.12 High-Level Risks

The main risks arise from combining navigation, manipulation and scheduling near patients. Navigation may fail if the path is not detected or the robot overshoots its stop. Manipulation may fail if the arm misses the container, drops it or places it off the table, which affects patient safety directly. Hardware failure of the arm, servos or chassis may stop development, and differences between simulation and hardware may reduce performance after transfer. Section 3.2 assesses these risks and their mitigation.

3.1.13 Task Breakdown and Scheduling

The project was broken down into sequential phases, with dependencies managed between navigation, manipulation and scheduling. Parallel work was used where possible: for example, the arm's pick-and-place was developed on the fixed-base model while the hospital world and mobile base were being designed. Progress was reviewed regularly against the milestones, and each working stage was committed to version control so that any later change could be compared with, or rolled back to, a known good state. This also allowed a risky change to be developed on a separate branch while a long evaluation continued on the stable version.

3.1.14 Work Breakdown Structure (WBS)

The work breakdown structure divides the project into the same seven phases as the schedule, each with its own activities and outputs (Table 3.3 and Figure 3.4).

Phase	Tasks included
1. Requirements and research	Literature review, problem definition, scope, requirements, selection of platform and software
2. System design	Architecture, kinematics and grasp design, navigation design, delivery sequence and checks
3. Hardware integration and configuration	Robot set-up; after the failure: simulation environment, robot model on the mecanum chassis, wrist camera
4. Software development	Arm control and inverse kinematics, line follower and base motions, mission controller, scheduler, smart bedside table, route planner and junction handling
5. Testing and validation	Offline closed-loop tests, subsystem tests in simulation, integration tests
6. Deployment and evaluation	Scheduled deliveries, two 5-run evaluations, failure analysis and fixes, patient-room missions, four-room hospital demonstration
7. Documentation and presentation	Report, user guide, demonstration videos, public repository

Table 3.3: Work breakdown structure

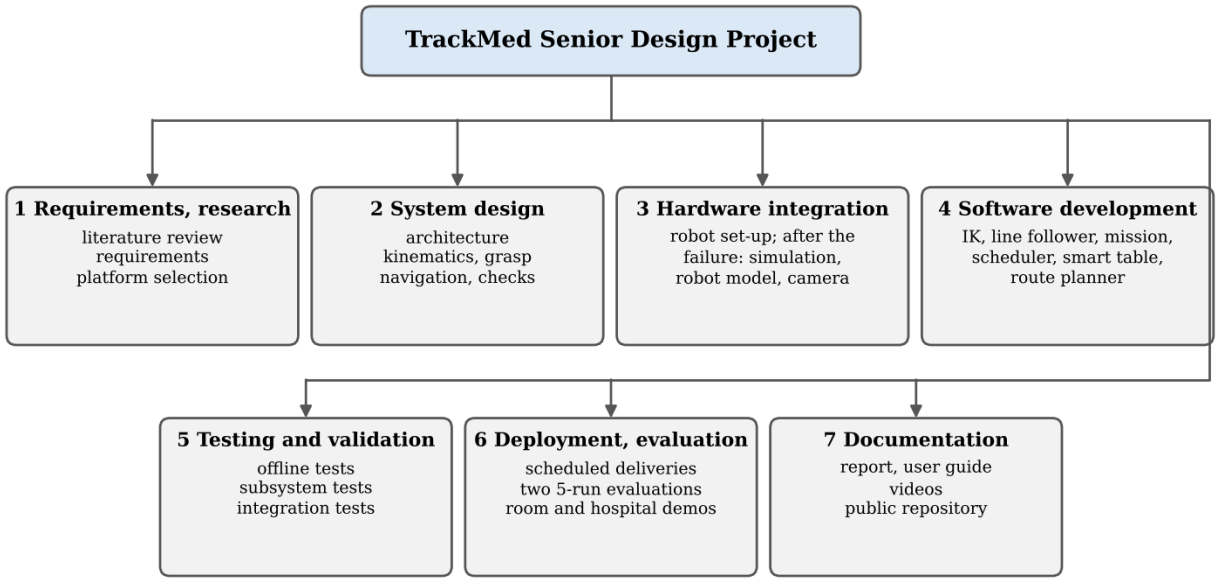


Figure 3.4: Work breakdown structure of the TrackMed project

Each phase was planned so that its outputs were the inputs of the next, which avoided rushed transitions between phases and made it clear what had to be finished before the next phase could start.

3.2 Risk Management and Mitigation

Risk management is critical for TrackMed, because failures may affect patient safety as well as the project itself. Potential risks include navigation errors, incorrect or failed delivery, hardware failure and differences between simulation and hardware. They were mitigated through the choice of a predictable navigation method, feedback-checked mission steps, simulation built from the manufacturer's robot model, and continuous testing. This section identifies and assesses the risks, describes their mitigation, and reports which risks occurred.

3.2.1 Risk Identification

Technical risks include navigation failures, where the robot does not detect the path or stops in the wrong place; manipulation failures, where the arm misses, drops or misplaces the container; and integration failures between the scheduler, mission controller and robot. Operational risks include hardware failure of the arm, servos, chassis or computer, limited computing power for simulation, and limited availability of team members. Safety risks include collisions with furniture or people and the robot continuing a task in an unsafe state. Finally, the change from hardware to simulation introduced the risk that simulation results would not transfer to the physical robot.

3.2.2 Risk Assessment

Each risk was rated for the probability that it would occur during the project and for its impact on the project, on a three-level scale. The combination gives the risk level used to prioritise mitigation (Table 3.4 and Figure 3.5).

Risk	Probability	Impact	Risk level
R1 Hardware integration issues (arm, camera, chassis, software)	High	High	High
R2 Navigation errors (path not detected, overshoot)	Medium	High	High
R3 Incorrect or failed delivery (missed grasp, dropped or misplaced container)	Medium	High	High
R4 Data privacy of schedules and delivery logs	Medium	Medium	Medium
R5 Wireless connectivity in the operating environment	High	Medium	High
R6 Mechanical or electrical failure of the robot	Low	High	Medium
R7 Budget limitations for hardware and maintenance	Medium	Medium	Medium
R8 Environmental conditions affecting the camera	Low	Medium	Low
R9 Team member unavailability	Low	Medium	Low
R10 Simulation too slow for repeated testing (added)	High	Medium	High
R11 Simulation results do not transfer to hardware (added)	Medium	High	High

Table 3.4: Risk calculation table

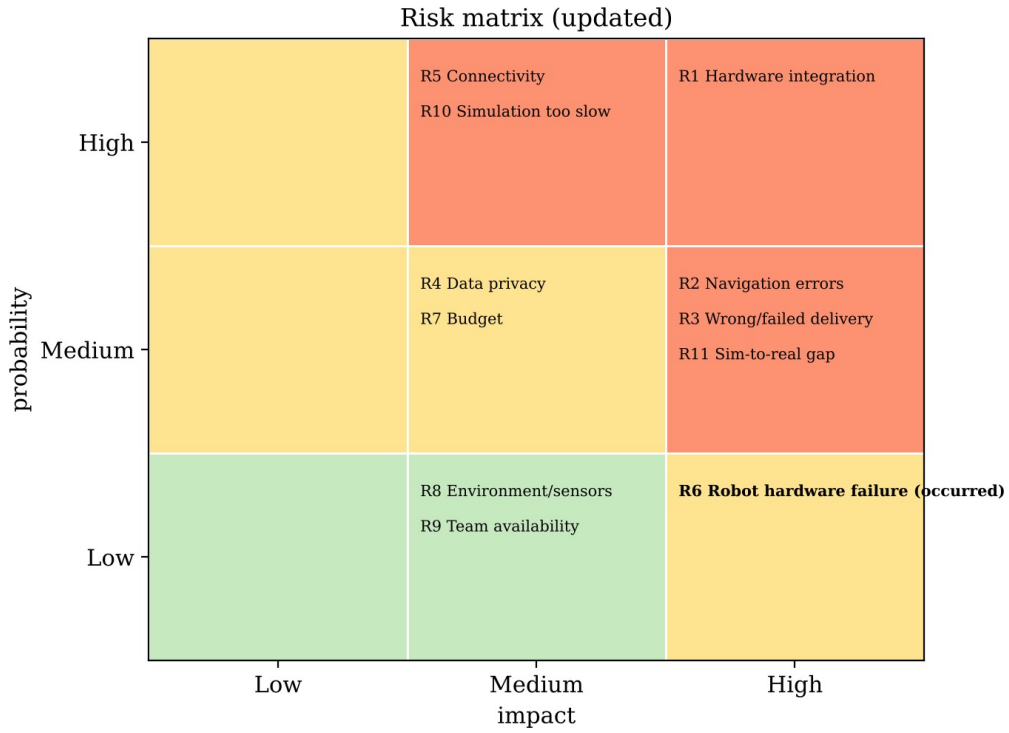


Figure 3.5: Risk matrix with the updated risk register; the realised risk R6 is shown in bold

3.2.3 Risk Mitigation Strategies

Table 3.5 lists the mitigation strategy planned or applied for each risk, and the outcome during the project.

Risk	Mitigation strategy	Outcome
R1 Hardware integration	Use the manufacturer's supported configuration and its robot description; integrate one subsystem at a time	Integration completed in simulation
R2 Navigation errors	Use the manufacturer's tested line-following method; test the follower offline in closed loop before Gazebo; stop on a red marker confirmed in two frames	Every stop within 2.6 cm of its planned point (target 5 cm, Table 7.12)
R3 Failed delivery	Confirm every attach and release; check the container's real position at the five points of Section 5.8.1; stop safely on failure	Occurred in evaluation 1; fixed for evaluation 2
R4 Data privacy	Store schedules and logs locally; keep patient identifiers minimal; plan encryption and access control for a networked version	No patient data used
R5 Connectivity	Run the scheduler on the robot's computer so deliveries do not depend on a network	Network not required

Risk	Mitigation strategy	Outcome
R6 Robot failure	Develop in simulation built from the manufacturer's URDF, with the same controllers and camera placement	Occurred; mitigated by full simulation
R7 Budget	Use open-source software throughout; drop items not required by the final design	Within budget
R8 Environment	Choose high-contrast path colours; use a colour space robust to brightness (HSV)	Path detected reliably
R9 Team availability	Keep documentation and notes in the repository	No impact
R10 Slow simulation	Run headless, reduce the camera to 4 Hz, model the base kinematically, use an assisted grasp	One delivery in 31-58 min of wall-clock time in the single room, 67-103 min in the hospital demonstration
R11 Sim-to-real gap	Use the manufacturer's models and controllers; state the limitations; validate on hardware as future work	Open; addressed in Section 9.4

Table 3.5: Risk mitigation strategies and outcomes

3.2.4 Realised Risk: Failure of the Robot

The proposal rated mechanical failure of the robot (R6) as a low-probability, high-impact risk. This risk occurred during the hardware-integration phase: the robot failed and could not be repaired or replaced within the project timeline. Because the failure happened before navigation and delivery could be tested, continuing on hardware would have meant losing the testing, evaluation and pilot milestones.

The team evaluated three options: waiting for a repair, switching to a different robot, or continuing in simulation. Waiting carried no guarantee of a repair in time, and a different robot would have required a new design. Continuing in simulation preserved the design and all objectives except physical validation. Hiwonder supplies a complete URDF model of the JetArm Pro, its ROS 2 controllers and Gazebo packages, so the simulated arm has the same geometry, joint limits, controllers and camera placement as the hardware. The team therefore moved the remaining development and all testing into Gazebo, modelled the mecanum chassis from the manufacturer's documentation, and kept the software interfaces identical to those of the real robot, so that the code can be moved back to hardware later (objective O6).

This decision kept the project on schedule. Its cost is the sim-to-real gap (R11): results obtained in simulation must be confirmed on hardware before any clinical use, as discussed in Sections 7.9.3 and 9.4.

3.2.5 Continuous Risk Monitoring

Risk management was an ongoing activity rather than a single assessment. Risks were reviewed at each milestone and whenever a test revealed a new failure mode, and the register was updated with new risks such as R10 and R11. The automated evaluation itself served as a risk-monitoring tool: its first run revealed the lost-command failures behind R3, which were then fixed and re-evaluated.

4 REQUIREMENTS ANALYSIS

The requirements analysis defines what TrackMed must do and how well it must do it, so that the design can be checked against measurable targets. Functionally, the system must start deliveries at scheduled times, pick the container at the medicine station, travel to the patient, place the container at the bedside, return and record the outcome. Non-functionally, it must be accurate, reliable, safe around people and furniture, reproducible and maintainable. This chapter states each requirement with an identifier and a target value; Chapter 7 verifies each one.

4.1 Functional & Non-Functional Requirements

Functional requirements state what the system does: the services it provides and how it responds to inputs and failures. Non-functional requirements state the qualities the system must have, such as performance, reliability, safety and maintainability. Each requirement below has an identifier, a measurable target and a verification method, which makes it testable and traceable from design through to results.

4.1.1 Functional Requirements

TrackMed manages and carries out scheduled medication deliveries. Table 4.1 lists its functional requirements.

ID	Requirement	Target	Verification
FR-1	The system shall start a delivery automatically at the time given in the schedule	Within 1 minute of the due time	Scheduler test (Section 7.4)
FR-2	The robot shall pick the medication container from the medicine station	Container rises with the gripper	Pose check after the lift
FR-3	The robot shall follow the marked path to the patient and stop at the stop marker	Stop within 5 cm of the planned stop point	Line-following tests (Section 7.3)
FR-4	The robot shall place the container on the bedside table	Within 30 mm of the target, tilted less than 10°	Full-mission evaluation (Section 7.5)
FR-5	The robot shall return home and park at its start position	Within 5 cm and 5° of the start pose	Full-mission evaluation
FR-6	The system shall detect a failed grasp or release and stop safely	No mission continues with the container in the wrong place	Five container checks (Section 5.8.1)
FR-7	The system shall record every delivery	Patient, medicine, due time, outcome, placement error, attempts	Delivery log inspection
FR-8	The robot shall stop if the path is lost	Stop within 2 s of losing the line	Line-follower logic

ID	Requirement	Target	Verification
FR-9	The system shall support daily and one-off delivery times for several entries	Daily "HH:MM" and dated entries	Scheduler test
FR-10	The system shall run deliveries that fall due together one after another	No overlapping missions	Scheduler design
FR-11	The bedside table shall detect the placed container and lift it to the patient's reach	Lift to 0.80 m within 30 s; container still on the platform and upright (< 10°)	Table test and missions in the patient room (Section 7.6)
FR-12	The bedside table shall not move while the gripper is above it or still holding the container	Lift only after a steady contact of 2 s, a released weld and the arm at home	Table test (Section 7.6.1)
FR-13	The bedside table shall lower again when the container has been taken	Lower after 3 s without contact	Table test
FR-14	The system shall choose the medicine due, and a slot holding it, from the medication schedule and the current time	Right container delivered	Hospital system demonstration (Section 7.7.4)
FR-15	The robot shall reach the right one of four patient rooms by a planned route	Planned junction actions executed	Route tests and demonstration (Section 7.7)
FR-16	The robot shall check the container's cap against the inventory before picking it	Wrong cap stops the delivery before the pick	Cap test (Section 7.7.2)
FR-17	The scheduler shall queue doses that are due together, earliest due first, then shortest route, and log each dose's lateness	Queue order and lateness in the log	Demonstration
FR-18	The scheduler shall skip a medicine that is out of stock, log it and continue	Logged, next dose run	Unit tests
FR-19	Picking one medicine shall not disturb the others	Other containers moved < 1 mm	Pick test and demonstration
FR-20	The robot shall stop if a marker is missing or does not match the plan	Stop with an error	Route simulator fault cases

Table 4.1: Functional requirements

4.1.2 Scheduling and Dispensing Logic

The scheduling and dispensing logic links the medication schedule to the robot's physical actions. Schedules are written by a caregiver in a schedule file, one entry per delivery, giving the patient, the medicine, the bed and the time. A time of the form "HH:MM" repeats daily, while a date and time is a one-off delivery. The scheduler computes the next due time of every entry, waits until the earliest one falls due, and then starts the delivery mission with a label that identifies the patient and medicine.

When a delivery starts, the mission controller runs a fixed sequence: it confirms that the container is resting at the medicine station, picks it, carries it to the patient, places it on the bedside table, returns and parks. Each step waits for feedback from the robot before the next begins, and the container's real position is checked after the critical steps. If a check fails, the mission stops the robot in a safe state and reports the failure; it never continues on the assumption that a step has worked.

At the end of each delivery, the scheduler records the outcome in the delivery log: the due time, start and finish times, patient, medicine, bed, success or failure, the failed step and error message, the placement error, the container tilt, the number of attempts each gripper command needed, and the simulated and wall-clock durations. Because the scheduler runs on the robot's own computer, deliveries do not depend on a network connection; a networked caregiver interface can be added later on top of the same log (Section 9.4).

4.2 Non-Functional Requirements

Table 4.2 summarises the non-functional requirements with their targets; the subsections explain each group.

ID	Quality	Requirement and target	Verification
NFR-1	Performance	Single-room ward: complete delivery and return in at most 180 simulated seconds	Full-mission evaluation
NFR-2	Reliability	At least 4 of 5 independent runs succeed	Full-mission evaluation
NFR-3	Performance, safety	Drive at 0.08 m/s, the speed used on the hardware [6], near markers, junctions and furniture; at most 0.12 m/s on long, clear corridor straights	Speed test
NFR-4	Safety	Never rotate in place within 0.27 m of a table	Turn-round manoeuvre design and test
NFR-5	Maintainability, interoperability	Hardware-specific code limited to the arm, gripper and velocity interfaces	Code structure (Appendix A)
NFR-6	Reproducibility	Every evaluation run starts from a fresh simulation; all code under version control	Evaluation tool, Git history
NFR-7	Usability	Schedule editable as a plain text file; log readable in a spreadsheet	Inspection
NFR-8	Safety	Arm motions stay at least 0.1 rad inside every joint limit	IK solver check
NFR-9	Safety, usability	Table moves smoothly (at most 40 mm/s, 40 mm/s²) so the container does not slide or tip	Lift profile and container pose after the lift
NFR-10	Performance	Room furniture reduces the real-time factor by at most 10 %	Interleaved before/after measurements (Section 7.6.3)

ID	Quality	Requirement and target	Verification
NFR-11	Performance	The hospital system runs at a real-time factor of at least 7 %, measured headless with the robot idle as in Section 7.3.2	Interleaved measurements (Section 7.7.5)
NFR-12	Performance	The longest delivery route is at most about 6 m one way	Layout and route planner

Table 4.2: Non-functional requirements

4.2.1 Performance

The robot must complete a delivery and return within a predictable time so that scheduled doses arrive on time. In the simulated ward, where the robot drives about 2.96 m each way along a 3.46 m line, the target is a complete cycle in at most 180 simulated seconds (NFR-1). This target was set for the single-room ward; the hospital system, whose routes are up to 5.54 m one way, was given no separate cycle-time target, and its cycle times are reported in Section 7.7. The robot drives at 0.08 m/s on straight sections, the same speed used by the manufacturer's line follower on the real chassis, and slows in sharp turns. In the hospital corridor it drives at up to 0.12 m/s on long, clear straights, but returns to 0.08 m/s at least 0.8 m before every marker and junction (NFR-3). Arm motions are given a duration of a few seconds each, and every step waits for the robot to report completion, so the cycle time reflects real motion rather than fixed delays.

4.2.2 Reliability

A delivery robot is only useful if deliveries succeed consistently. The target is at least four successful deliveries in five independent runs (NFR-2), where a run succeeds only if the container stands upright on the bedside table within 30 mm of the target. Reliability is supported by confirming every gripper command, checking the container's real position, and stopping safely rather than continuing after a failure. Every evaluation run starts from a fresh simulation, so that a run cannot inherit a state left by the previous one (NFR-6).

4.2.3 Scalability

The design supports more patients and deliveries without structural change: the schedule holds any number of entries, the mission takes the pick and place positions as parameters, and the world is generated from a script in which beds and stations can be added. The hospital system of Section 5.12 uses this to serve four rooms from one storage unit; its size is bounded by the longest route (NFR-12), because the simulation's speed limits the wall-clock time of a delivery. Several robots, several wards and integration with hospital systems would require coordination and a central server, which are outside the scope of this project.

4.2.4 Usability

Caregivers must be able to set up and check deliveries without programming. The schedule is a short plain-text file with one entry per delivery, and the delivery log is a comma-separated file that opens in any spreadsheet (NFR-7). A graphical caregiver interface with notifications is specified as future work.

4.2.5 Security

The prototype stores schedules and logs locally on the robot's computer and uses no network services, which limits exposure. A deployed version that exchanges data with a server or mobile application must encrypt all communication (for example with TLS), authenticate users, restrict who can change schedules or command the robot, and keep an audit trail, because a tampered schedule is a patient-safety risk as well as a privacy risk. These are requirements for a deployed version; Table 4.2 sets no security requirement for the prototype, which holds no patient data beyond a label.

4.2.6 Maintainability

The software is divided into modules with a single responsibility each: inverse kinematics and arm control, line following, base motion, gripper-command confirmation, mission control, scheduling and evaluation. Modules communicate through ROS 2 topics with standard message types, so a module can be changed or replaced without affecting the others. All code is under version control with a commit for every working milestone, and a project notes file records design decisions and known issues.

4.2.7 Interoperability

TrackMed uses standard ROS 2 interfaces throughout: joint trajectories for the arm and gripper, velocity commands and odometry for the base, and standard image messages for the camera. These are the same interfaces the JetArm Pro uses on hardware, which is what allows the software to move from simulation to the robot with changes limited to the hardware interfaces (NFR-5). Results are written in open formats (CSV and JSON) that other tools can read.

4.2.8 Safety and Reproducibility

The safety requirements (NFR-3, NFR-4, NFR-8 and NFR-9) limit the robot's speed near objects, keep its turning circle clear of every table, keep the arm away from its joint limits and keep the smart table's motion gentle; Section 4.4.2 explains how they protect a patient. The reproducibility requirement (NFR-6) makes every evaluation run start from a fresh simulation, with all code and results under version control, so that any reported result can be traced to the code that produced it.

4.3 User & System Requirements and Constraints

Users need a dependable, transparent delivery service that requires little effort; the system needs accurate navigation and manipulation and a reliable link between the schedule and the robot. The main constraints are the capabilities of the platform, the structured environment the method requires, the computing power available for simulation, and the regulations that apply to healthcare devices and data.

4.3.1 User Requirements

The system serves patients, nurses and caregivers. Caregivers need to enter and change delivery times easily and to see afterwards what was delivered, when, and whether it succeeded, including any failure that needs their attention. Patients, particularly those with limited mobility, need the dose placed within reach at the right time without having to move to a device. Both groups need to trust the system, which requires that it either completes a delivery correctly or stops and reports the problem; a silent failure, such as driving away still holding the container, is not acceptable.

4.3.2 System Requirements

The system requires an arm with at least five joints and a gripper able to hold a small medication container, a camera that can see the floor ahead of the robot, and a mobile base able to move forward, sideways and rotate. On the software side it requires ROS 2 Humble with controllers for the arm, gripper and base, a vision pipeline for line detection, a mission controller, a scheduler and persistent storage for logs. For development and evaluation in simulation it requires Gazebo Fortress with the Sensors and DetachableJoint systems, and a computer able to render the wrist camera.

4.3.3 Constraints

Several technical, operational and project constraints shaped the design:

- **Platform capabilities.** The JetArm Pro's reach with the gripper pointing down is about 0.14-0.24 m at table height, its gripper opens to about 53 mm, and its joints are limited to ±2.09 rad. Pick and place positions, table heights and the container size were chosen within these limits.
- **Hardware availability.** The robot failed during development, so implementation and evaluation were carried out in simulation; physical validation is future work.
- **Computing resources.** The simulation runs in a virtual machine with two processor cores. Rendering the wrist camera slows the simulation to about 7-11 % of real time with the robot idle and about 4-9 % during a mission, so one delivery takes 31-58 minutes of wall-clock

time in the single room and 67-103 minutes in the hospital system, and the number of evaluation runs is limited.

- **Unpublished dimensions.** The manufacturer does not publish the chassis dimensions, so they were estimated from installation photographs (±15 %).
- **Structured environment.** Line following requires a marked path and fixed station and table positions; the prototype assumes no people or obstacles on the path.
- **Medication handling.** The container is assumed to have been prepared and checked by a nurse or pharmacist; the robot does not verify its contents. In the hospital system it checks only that the container's cap colour matches the inventory (Section 5.12.6).
- **Time and scope.** The work had to fit within two semesters, so features such as the caregiver application and obstacle detection were scoped as future work. Deliveries to several patients, first scoped out as well, were added at the end of the project in the hospital system (Section 5.12).

These constraints define the boundaries within which the results of Chapter 7 are valid, and they point to the main areas for future work.

4.3.4 Functional Standards

The system follows good practice for autonomous systems: each delivery step has a defined precondition and postcondition, each postcondition is checked against the robot's real state, and any failed check leads to a safe stop and a logged error. Navigation stops at a confirmed marker, and arm motions are computed and checked against joint limits before they are sent. These practices make every delivery repeatable, verifiable and fail-safe: a fault stops the robot rather than being worked around.

4.3.5 Industry-Specific Standards

Table 4.3 lists the standards relevant to a robot of this kind and how each applies to TrackMed.

Standard	Scope	Relevance to TrackMed
ISO 13482:2014 [19]	Safety requirements for personal care robots: mobile servant, physical assistant and person carrier robots	TrackMed is a mobile servant robot; the standard's hazards (collision, crushing by the arm, unexpected motion) motivate the low speed, the clearance rule and the safe stops
IEC 60601-1:2005 [20]	General requirements for basic safety and essential performance of medical electrical equipment	Applies if the robot is classified as medical electrical equipment; guides the electrical safety of a hardware version
ISO 13485:2016 [21]	Quality management systems for medical devices	Required for a commercial version regulated as a medical device

Table 4.3: Relevant standards

4.3.6 Data Security Standards

TrackMed handles data that identify patients and their medication: schedules and delivery logs. A deployed version must protect this data in transit and at rest, restrict access to authorised caregivers, keep only the data needed for the service, and keep an audit trail of who changed a schedule and when. In the prototype, data are stored only on the robot's computer, no patient identifiers beyond a label are used, and no data leave the machine.

4.4 Domain Requirements

TrackMed operates in the healthcare domain, where patient safety, data protection and human-robot interaction impose requirements beyond those of a general-purpose robot. The robot must work near patients who may be frail, in rooms with beds and furniture, and it handles data about patients' medication.

4.4.1 Healthcare and Medical Device Regulations

A robot that takes part in medication administration may be regulated as a medical device, depending on its intended use and the jurisdiction. A commercial version would therefore need a quality management system such as ISO 13485 [21], electrical safety assessment against IEC 60601-1 if it is classed as medical electrical equipment [20], and safety assessment as a personal care robot under ISO 13482 [19]. The prototype is a research system and is not intended for clinical use; responsibility for the correct medication remains with the clinician who prepares it.

4.4.2 Patient Safety and Collision Avoidance

The robot moves at 0.08 m/s on straight sections and slower in turns, and at up to 0.12 m/s only on long, clear corridor straights, at least 0.8 m before any marker or junction. It stops if it loses the path. It never rotates in place within 0.27 m of a table: at the bedside it first moves sideways away from the table, then turns, then moves back onto the path. The arm's motions are computed in advance and kept away from joint limits, and the arm is folded into its ready pose while the robot drives. Detection of people and obstacles on the path is specified as future work, using the depth camera.

The smart bedside table is a second moving machine beside the patient and follows the same rule: it moves only when it is safe to do so. It lifts only when the container has rested on its platform for 2 seconds, the gripper has released it and the arm has returned to its folded pose, so the platform can never strike the gripper (FR-12). It moves at no more than 40 mm/s with gentle acceleration (NFR-

9), and the side rail of the bed on the table side covers only the head section and ends below the raised platform, so the patient can reach the medication without climbing over a rail.

4.4.3 Human-Robot Interaction (HRI) Protocols

The robot follows a fixed, visible route marked on the floor, which makes its behaviour predictable for patients and staff. It places the container on the bedside table rather than handing it to the patient, which avoids contact between the arm and the patient. Every delivery is logged, and a failure leaves the robot stopped and the error recorded for a caregiver to review. A deployed version should add visual and audio signals before the arm moves, an emergency stop button, and a confirmation by the patient or caregiver that the dose was received.

4.4.4 Health Data Privacy Laws

In the United Arab Emirates, health data processed using information and communication technology are governed by Federal Law No. 2 of 2019, which requires them to be kept confidential, stored inside the country unless an exception is approved, and retained for at least 25 years [22]. Other personal data fall under the Personal Data Protection Law, Federal Decree-Law No. 45 of 2021, which took effect on 2 January 2022 [23]. A deployed TrackMed system that keeps schedules and logs about named patients must comply with both, which affects where data are stored, who can access them and how long they are kept.

5 SYSTEM ANALYSIS & DESIGN

This chapter presents the design of TrackMed: the system model and architecture, the hardware platform, the software stack, and the design of each subsystem (arm kinematics and grasping, vision-based navigation, the mobile base, the patient room and smart bedside table, mission control and scheduling), and finally the extension into a four-room hospital system (Section 5.12). Each design decision is linked to the requirements of Chapter 4.

5.1 System Model

TrackMed is modelled as a set of cooperating subsystems around a central mission controller (Figure 5.1). The scheduler decides when a delivery starts. The mission controller runs the delivery as a sequence of eleven steps (fifteen in the hospital system, Section 5.12.7) and calls three subsystems in turn: the arm controller for picking and placing, the weld control for attaching and releasing the container, and navigation for following the path and turning round. Each subsystem acts on the robot and reports back, and the mission controller moves to the next step only after the report confirms the previous one. A fourth subsystem, the smart bedside table, is not part of the robot: it has its own sensor and controller, decides for itself when it is safe to lift the container, and reports its state, which the mission controller waits for before the robot leaves the bedside (Section 5.7.3).

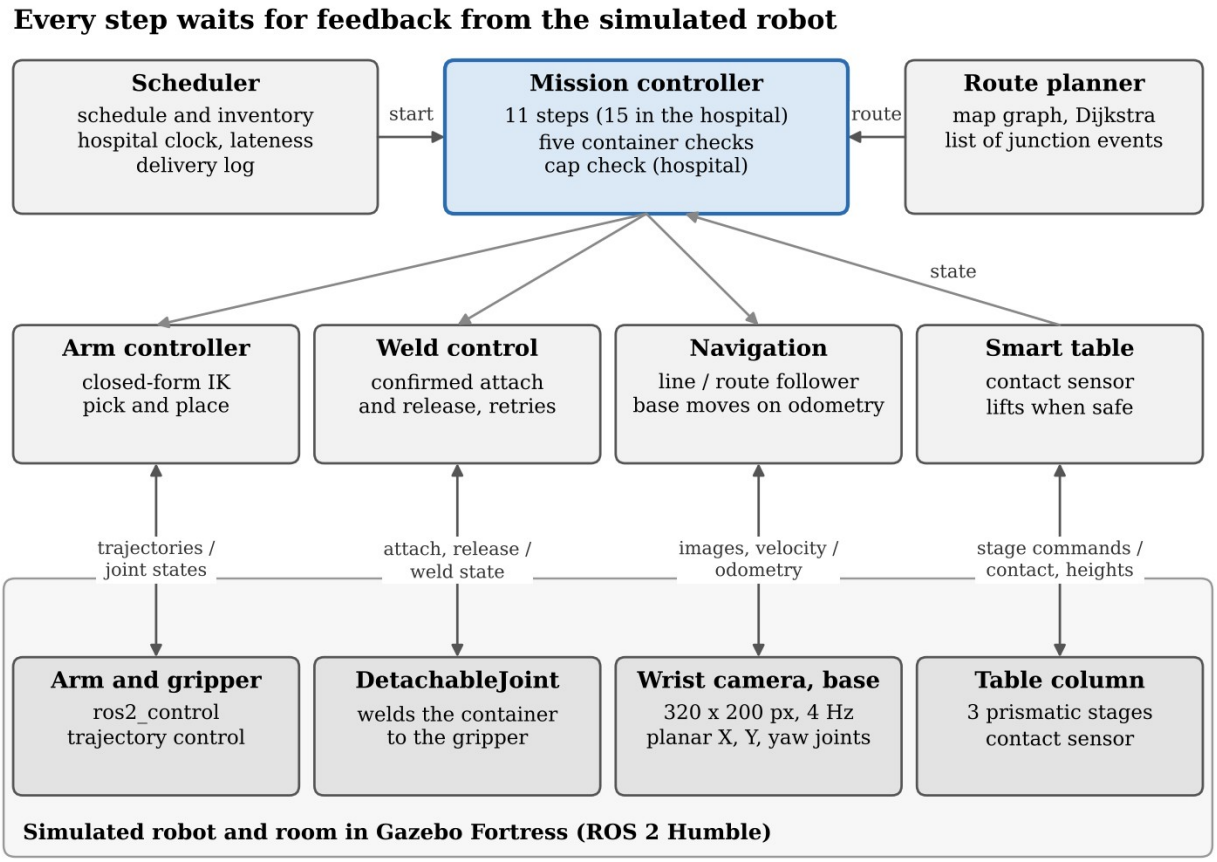


Figure 5.1: TrackMed system architecture: every mission step waits for feedback from the robot

The users of the system and the services it provides are summarised in Figure 5.2. Nurses and caregivers prepare the container, edit the schedule and review the delivery log; the patient receives the dose; and the robot collects, carries, places and returns, or stops safely and reports a fault.

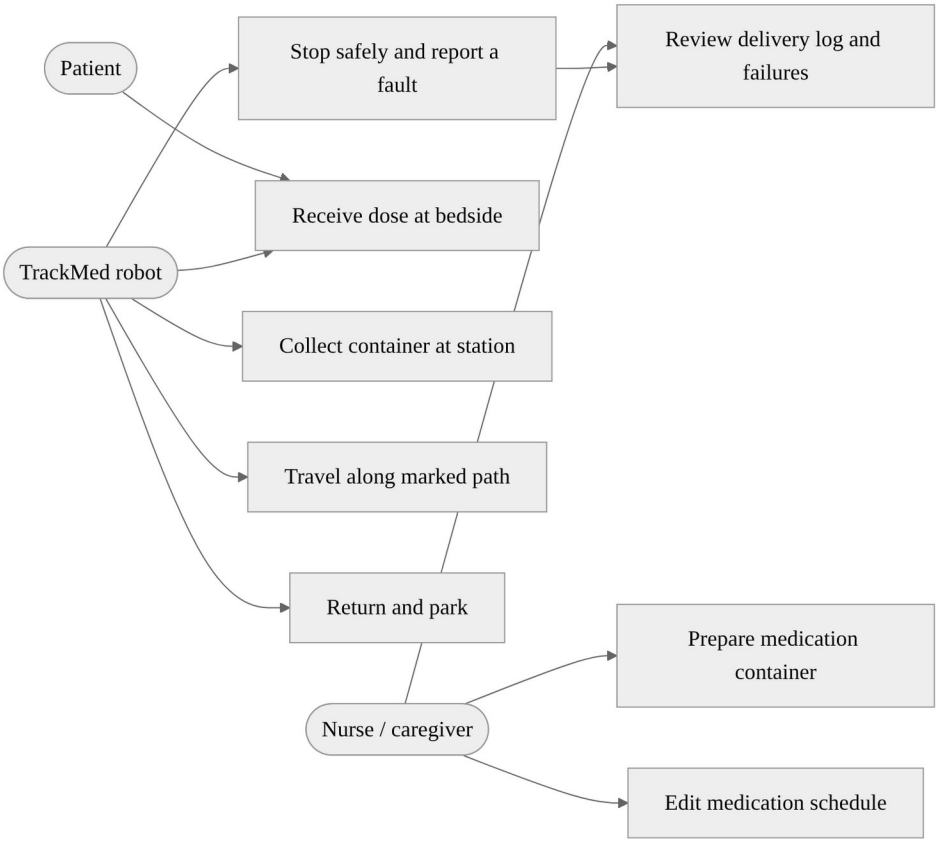


Figure 5.2: Users of TrackMed and the services the system provides

5.2 Methodology

The project followed an incremental, test-driven methodology. The system was built in stages, and each stage was tested on its own before the next was added: first the arm on a fixed base, then the hospital world, the mobile robot and its camera, line following, the complete mission, scheduling, automated evaluation, the patient room with the smart bedside table and finally the hospital system (Figure 3.3). Algorithms were first checked offline, against models built from the same geometry, before being run in the simulator, which allowed errors to be found quickly and cheaply.

Three principles guided the design:

- **Feedback before progress.** No step is assumed to have succeeded. Arm motions wait for the joint states to reach the target, base motions are closed on odometry, gripper commands wait for a confirmation, and the container's real position is checked after the critical steps.
- **Same interfaces as the hardware.** The simulated robot uses the manufacturer's URDF model, controllers and topics, so the mission code does not depend on whether it runs on the simulator or the robot.

- **Measure in the right time base.** Because the simulation runs much slower than real time, all waits, timeouts and rates are measured in simulated time, from the time stamps of the robot's messages.

5.3 Hardware Platform

5.3.1 JetArm Pro Robotic Arm

The JetArm Pro (Advanced version) is a desktop robotic arm, sold as six-degree-of-freedom, with five revolute joints and a servo-driven gripper (the gripper is counted as the sixth degree of freedom), an Orbbec Gemini depth camera and an NVIDIA Jetson computer running ROS 2 [11]. Its geometry, taken from the manufacturer's URDF model, is summarised in Table 5.1. Joint 1 rotates the arm about the vertical axis; joints 2, 3 and 4 are parallel and move the arm in a vertical plane; joint 5 rotates the gripper about its own axis. The gripper is driven by one joint (`r_joint`), and five further finger joints follow it as mimic joints.

Joint	Axis	Offset to next joint	Limits
joint1 (base yaw)	vertical (z)	0.054 m up to joint2	±2.09 rad
joint2 (shoulder)	horizontal (−y)	L2 = 0.134416 m	±2.09 rad
joint3 (elbow)	horizontal (−y)	L3 = 0.134445 m	±2.09 rad
joint4 (wrist pitch)	horizontal (−y)	LT = 0.134483 m to the tool point	±2.09 rad
joint5 (wrist roll)	tool axis (−z)	—	±2.09 rad
r_joint (gripper)	finger axis	five mimic finger joints	0 to 1.57 rad

Table 5.1: Kinematic parameters of the JetArm Pro (from the manufacturer's URDF)

The camera is mounted on the fourth link, above the gripper, so it always looks in the direction the gripper points. With the arm in its ready pose, the camera looks at the floor ahead of the robot, which is what makes it usable for line following.

5.3.2 Mecanum-Wheel Chassis

The arm is mounted on Hiwonder's mecanum-wheel chassis, which has four encoder motors connected to a four-channel motor driver: M1 front-left, M2 rear-left, M3 front-right and M4 rear-right [6]. The driver connects to the Jetson over I²C and to the arm's expansion board, and the arm is bolted to the chassis' top plate. Figure 5.3 shows the main hardware connections.

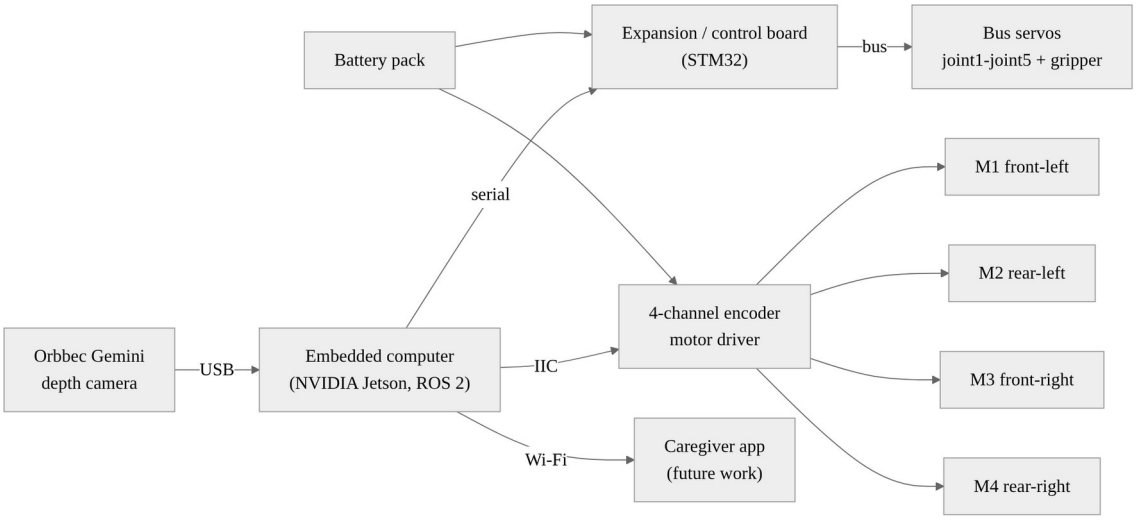


Figure 5.3: Hardware block diagram of the JetArm Pro on the mecanum chassis

The manufacturer does not publish the dimensions of the chassis. They were estimated from the installation photographs in the manufacturer's integration course, using the arm base, whose footprint of 0.32 m × 0.16 m is known exactly from its 3-D model, as a scale. The estimates are an overall length of about 0.32 m, a width of 0.26 m including the wheels, 80 mm wheels and an arm plate 0.10 m above the floor, with an uncertainty of about ±15 % (Figure 5.4). The plate height matters most, because it sets the camera height and the table height that the arm can reach.

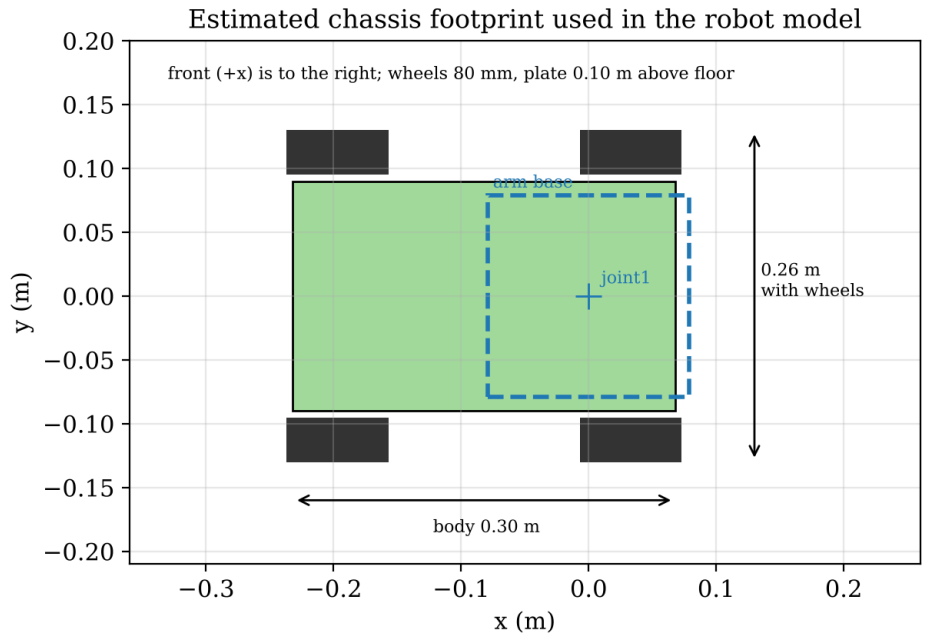


Figure 5.4: Estimated chassis footprint used in the robot model, top view

5.3.3 Medication Container

The container is modelled as a cylinder 30 mm in diameter and 60 mm tall with a mass of 30 g, with a blue cap, similar to a small pill bottle. Although the line detector looks for blue, the cap never disturbs it: during line following the container is either held in the gripper, inside the camera's near clipping distance, or left behind the robot on the table. Its diameter is chosen to fit inside the open gripper with a margin of about 23 mm. Its collision surface has a high friction coefficient (1.5), no restitution and a stiff contact, so that it stands steadily and does not bounce or sink into the table.

5.4 Software Architecture

5.4.1 Technology Stack

Figure 5.5 shows the layers of the software. The operating system is Ubuntu 22.04 running in a VMware virtual machine; on it run Gazebo Fortress and ROS 2 Humble; and on top of ROS 2 run the TrackMed programs, written in Python.

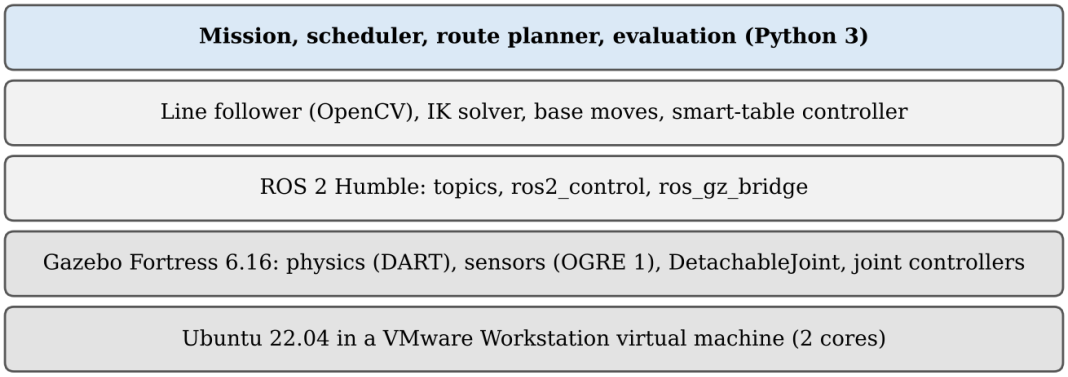


Figure 5.5: Software layers of the TrackMed system

5.4.2 ROS 2 Nodes and Topics

The programs communicate over ROS 2 topics; a bridge connects the ROS 2 topics to the corresponding Gazebo topics (Figure 5.6 and Table 5.2). The same topic names and message types are used by the JetArm Pro hardware, except for the base-joint topics, which replace the motor driver in simulation.

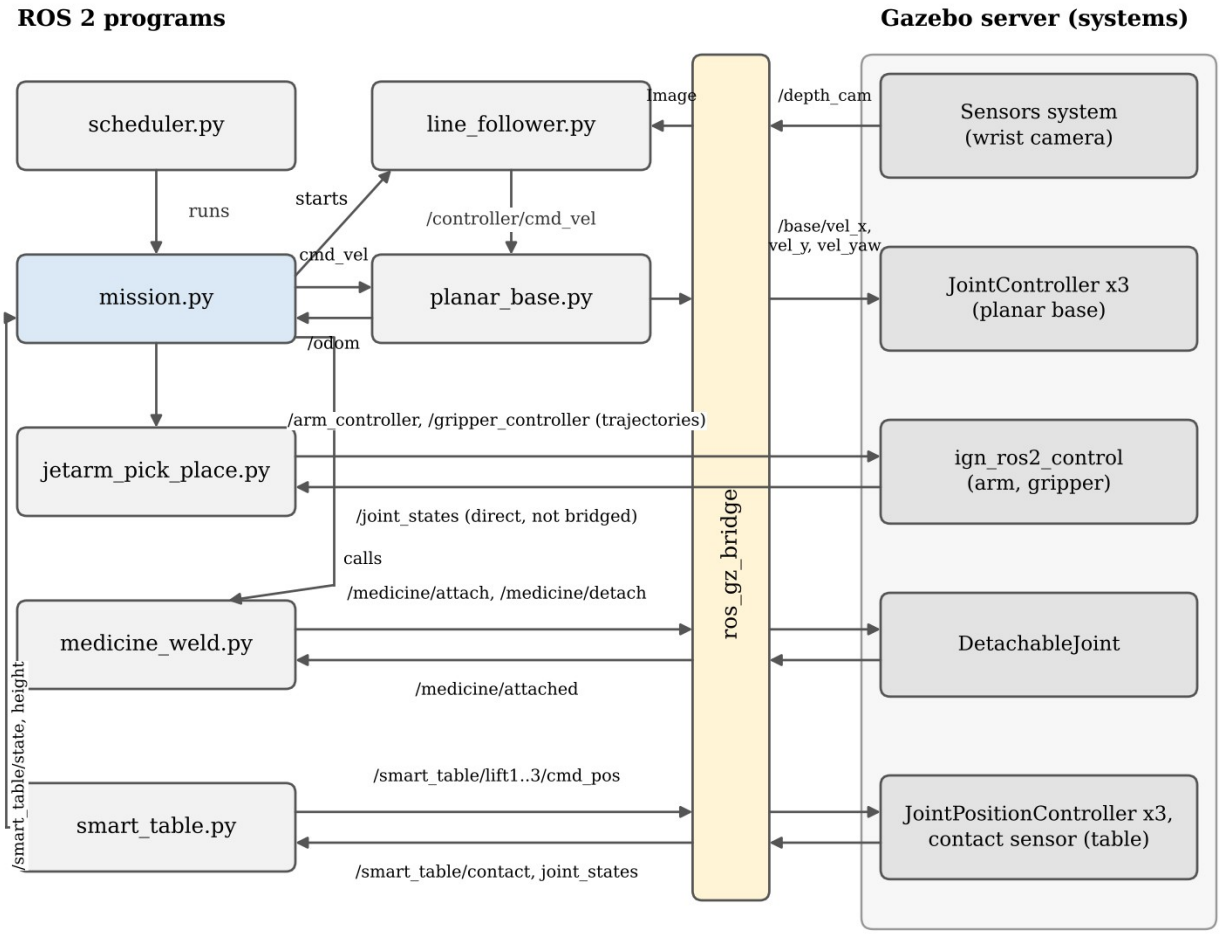


Figure 5.6: ROS 2 programs, Gazebo systems and the topics between them

Topic	Message type	Purpose
/arm_controller/joint_trajectory	trajectory_msgs/ JointTrajectory	Arm motion commands (joint1-joint5)
/gripper_controller/ joint_trajectory	trajectory_msgs/ JointTrajectory	Gripper commands (r_joint)
/joint_states	sensor_msgs/JointState	Measured joint positions of arm and gripper
/depth_cam/depth_cam	sensor_msgs/Image	Wrist-camera images (320 × 200, 4 Hz)
/controller/cmd_vel	geometry_msgs/Twist	Base velocity in the robot frame
/odom	nav_msgs/Odometry	Base position and heading
/base/vel_x, /base/vel_y, /base/vel_yaw	std_msgs/Float64	Velocities of the three planar base joints (simulation only)
/medicine/attach, /medicine/detach	std_msgs/Empty	Attach or release the container
/medicine/attached	std_msgs/String	Weld state reported by Gazebo
/smart_table/contact	ros_gz_interfaces/ Contacts	Contacts on the table platform (contact sensor)

Topic	Message type	Purpose
/smart_table/joint_states	sensor_msgs/JointState	Positions of the three lift stages
/smart_table/lift1..3/cmd_pos	std_msgs/Float64	Position set-points of the three lift stages
/smart_table/state	std_msgs/String	Table state: lowered, lifting, raised or lowering
/smart_table/height, /smart_table/contact_state	std_msgs/Float64, std_msgs/String	Platform height; container contact: none, touching or steady
/smart_table/raise, /smart_table/lower	std_srvs/Trigger	Manual raise and lower for testing

Table 5.2: Main ROS 2 topics used by TrackMed

5.5 Arm Kinematics and Pick-and-Place Design

The manufacturer's course on advanced arm control [24] drives the real arm through its own kinematics and servo interfaces. TrackMed instead derives a closed-form solution directly from the URDF model, so that every motion can be computed and checked against the joint limits before it is sent.

5.5.1 Forward Kinematics

Because joints 2, 3 and 4 are parallel, the arm's motion separates into a rotation about the vertical axis (joint 1) and motion in a vertical plane (joints 2-4). In that plane, let r be the horizontal distance from the joint-1 axis and z the height above the arm base. With the shoulder pivot at height Z2 = 0.104 m and the angles measured as in the URDF, the tool point is:

$$r = L2\cdot\sin(-q2) + L3\cdot\sin(-(q2+q3)) + LT\cdot\sin(-(q2+q3+q4)) \quad (5.1)$$

$$z = Z2 + L2\cdot\cos(-q2) + L3\cdot\cos(-(q2+q3)) + LT\cdot\cos(-(q2+q3+q4)) \quad (5.2)$$

and its position in the robot frame is $(x, y) = (r\cdot\cos q1, r\cdot\sin q1)$. The tool direction makes the angle $\varphi = -(q2 + q3 + q4)$ with the vertical. These equations were checked against the URDF: at the manufacturer's ready pose (0, 0.7309, −1.6127, −1.6591, 0) rad both give the tool point at (0.090, 0, 0.179) m, and with all joints at zero both give a tool point 0.507 m straight above the base (Figure 5.7).

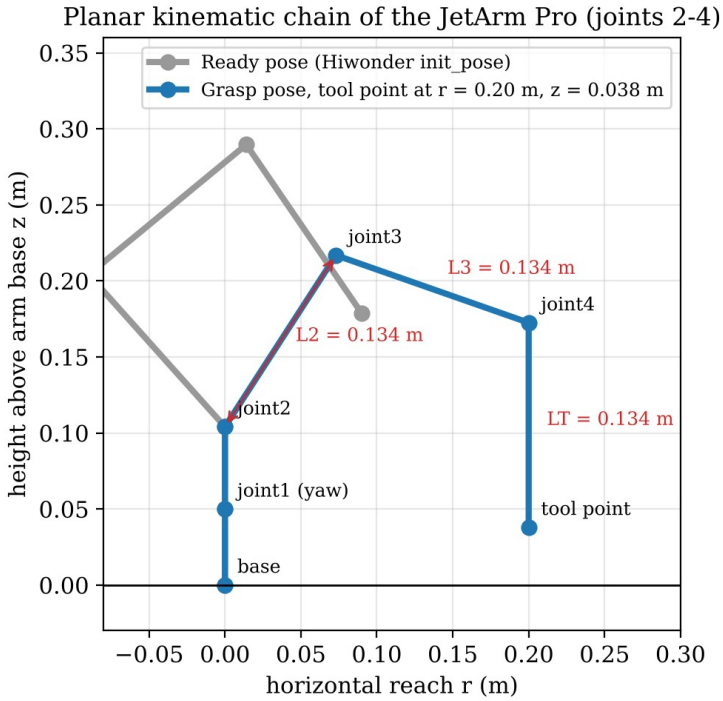


Figure 5.7: Planar kinematic chain of the arm in the ready pose and in the grasp pose

5.5.2 Closed-Form Inverse Kinematics

For a target tool point (x, y, z) with the gripper pointing straight down ($\varphi = \pi$), the base angle follows directly:

$$q1 = \operatorname{atan2}(y, x), \quad r = \sqrt{(x^2 + y^2)} \tag{5.3}$$

The wrist point lies LT above the tool point, so the problem reduces to placing the end of a two-link arm (L2, L3) at the wrist point (r, z + LT). With d the distance from the shoulder pivot to the wrist point, the law of cosines gives the angle between the first link and the line to the wrist:

$$d = \sqrt{(r^2 + (z + LT - Z2)^2)}, \quad \alpha = \arccos((d^2 + L2^2 - L3^2) / (2 \cdot d \cdot L2)) \tag{5.4}$$

The two signs of α give the "elbow-up" and "elbow-down" solutions; q2, q3 and q4 follow from the link directions, with q4 chosen so that the tool points down. The solver evaluates both solutions and, if neither is valid, tilts the tool outward in steps of 5° up to 60°. A solution is accepted only if every joint is at least 0.1 rad inside its limit (NFR-8) and if the forward kinematics of the solution reproduce the target within 1 mm. If no solution is accepted, the target is reported as out of reach.

Figure 5.8 shows the region the tool point can reach with the gripper pointing down, and Figure 5.9 the joint angles across the reachable band at grasp height. At table height the reach with the gripper vertical is about 0.14-0.24 m from the base axis. A reach of 0.20 m was chosen for both pick and place, near the middle of the band and away from the joint limits.

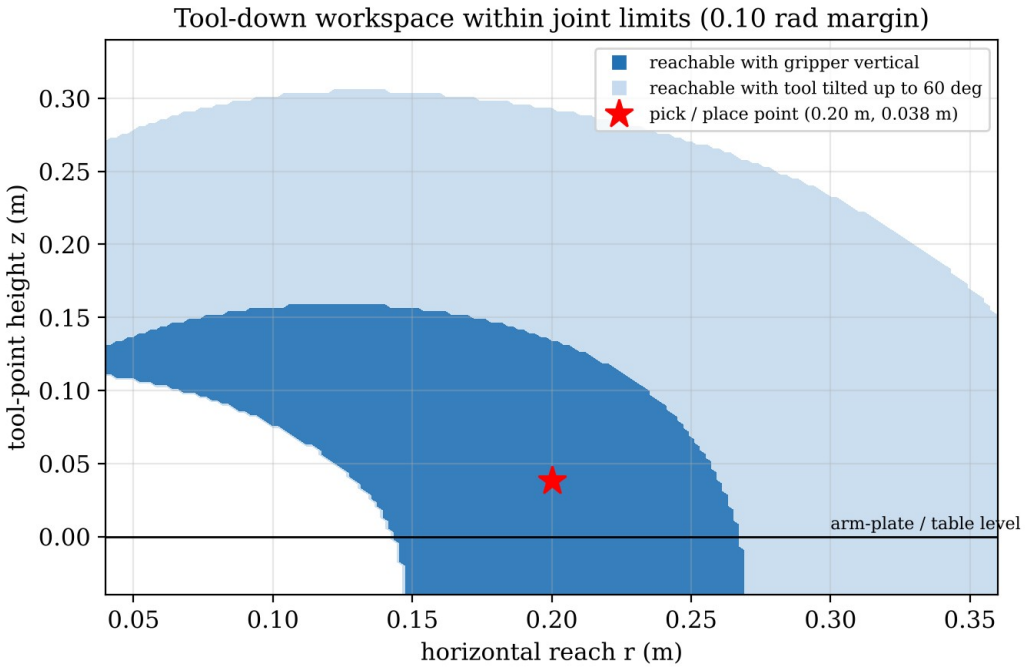


Figure 5.8: Reachable region of the tool point with the gripper pointing down

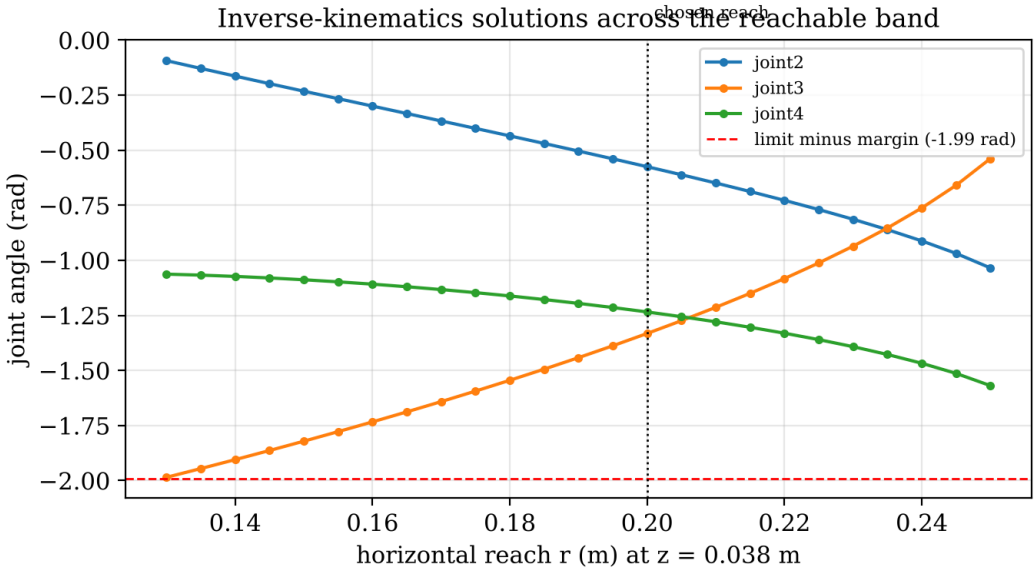


Figure 5.9: Inverse-kinematics solutions across the reachable band at the grasp height

5.5.3 Gripper and Grasp Height

The gripper's finger gap was computed from its 3-D meshes as a function of the gripper joint (Figure 5.10). It is closed at 0 rad, opens to about 31 mm at 0.6 rad and 53 mm at 1.4 rad. A 30 mm container therefore fits comfortably in the open gripper (1.4 rad), and the fingers touch it at about 0.59 rad.

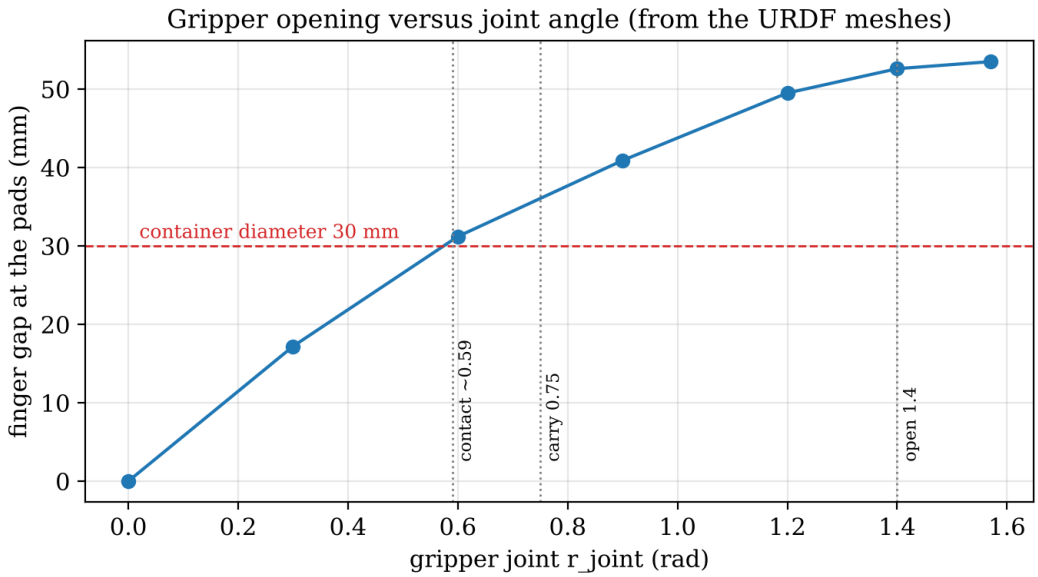


Figure 5.10: Gripper finger gap versus gripper joint angle

The fingers are curved: the flat pads close level with the tool point, while the claw tips curl up to 32 mm below it. If the gripper grasps too high, the pads hold only the rim of the container; if too low, the claw tips hit the table. The grasp height was set so that the tool point is 38 mm above the surface on which the container stands, which places the pads at the container's middle with the claw tips about 6 mm above the surface. Section 6.4 describes how this value was found experimentally.

5.5.4 Grasp Strategy

A purely friction-based grasp, in which the fingers squeeze the container, held the container correctly but slowed the simulation to about 0.3 % of real time. The six position-controlled finger joints pressing on a 30 g object create many stiff contacts that are expensive to solve. TrackMed therefore uses an assisted grasp: the fingers close around the container to 0.75 rad, just short of contact, and Gazebo's DetachableJoint system then creates a fixed joint between the container and the gripper for the carry [15]. Releasing removes the joint, and the container drops the last 7-8 mm onto the table under normal physics, so the placement result still depends on the simulated contact between container and table.

Because the attach and release commands are critical to the delivery, each one is confirmed: the command is sent through a persistent publisher, the controller waits for the DetachableJoint system to report the new state, and it resends the command up to five times if no confirmation arrives within two simulated seconds. Independently of this confirmation, the mission checks the container's real position five times during a delivery (Section 5.8.1): at the station before the pick, after the lift, after the placement, after the arm has withdrawn, and on the raised platform before the robot drives away.

5.5.5 Pick-and-Place Motion Sequence

A pick consists of five arm and gripper motions: open the gripper to 1.4 rad, move above the container at a travel height of 0.14 m, descend to the grasp height, close to 0.75 rad and attach, then lift back to the travel height. The descent and the lift are divided into four waypoints each, computed by the inverse-kinematics solver, so that the gripper moves almost vertically and does not sweep sideways into the container. A place is the reverse: move above the target, descend to a release height 7 mm higher than the grasp height, release, open the gripper and retreat upward. Each motion waits until the measured joint positions are within 0.03 rad of the target.

5.6 Navigation Design

5.6.1 Camera Geometry

With the arm in its ready pose, the wrist camera is 0.396 m above the floor and its axis points 54.1° below the horizontal, as measured from the simulated robot's transforms (Figure 5.11). The bottom of the image sees the floor about 0.18 m ahead of the arm's base axis, and the centre of the image about 0.34 m ahead. The gripper and a carried container are below the camera's field of view, so the robot can carry the container in the gripper while following the line, and no separate holder on the chassis is needed. The camera's near clipping distance is set to 0.15 m so that parts of the gripper close to the lens are not rendered into the image.

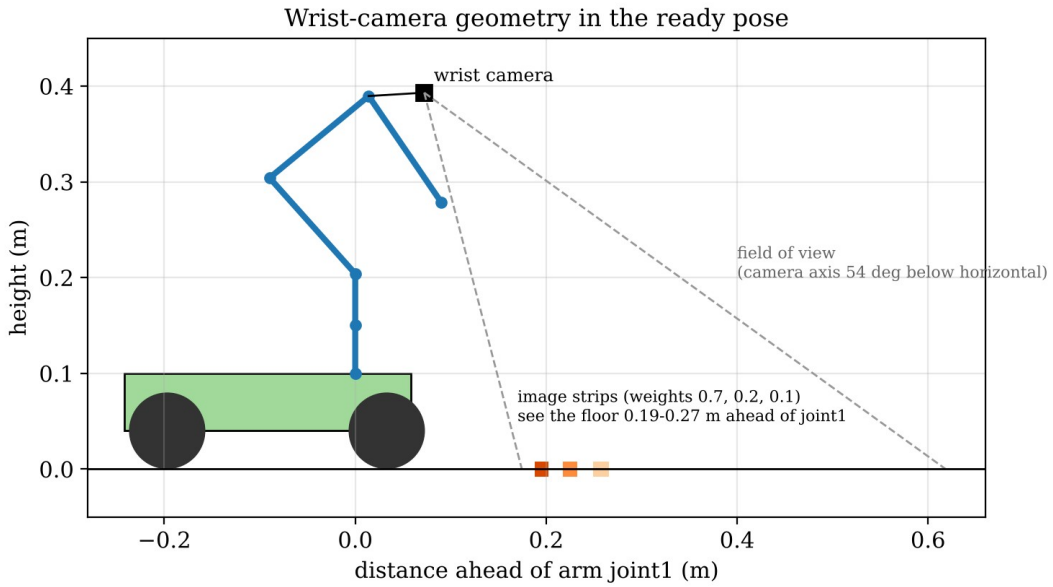


Figure 5.11: Wrist-camera geometry with the arm in the ready pose

5.6.2 Line-Detection Algorithm

The line follower is a port of the manufacturer's line-following program for this arm and chassis [6], adapted to the simulation. Figure 5.12 shows the processing pipeline. Each camera frame is blurred slightly and converted to the HSV colour space, which separates colour from brightness and makes the threshold robust to shading. Three horizontal strips near the bottom of the image are examined, with weights 0.7, 0.2 and 0.1 from the bottom up, exactly as in the manufacturer's code for this camera. In each strip, pixels within the blue range (hue 100-130, saturation at least 120, value at least 60) form a mask, which is cleaned by erosion and dilation; the largest blob's centre is taken as the line position in that strip.

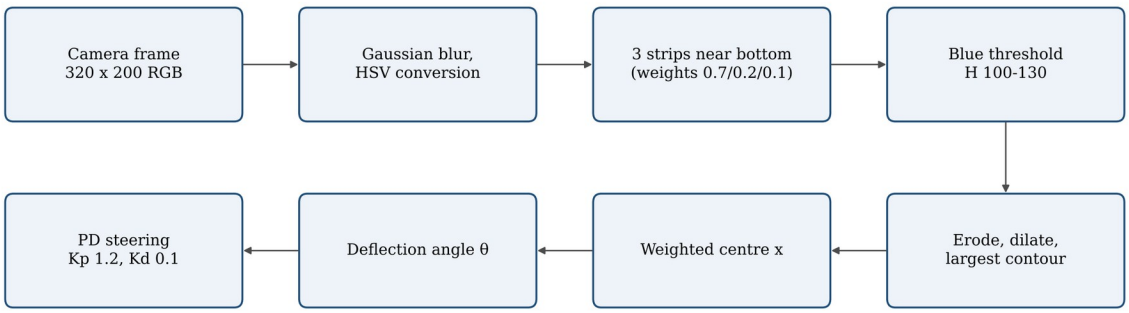


Figure 5.12: Line-following processing pipeline; a frame with red covering more than 15 % of the strips in two consecutive frames stops the robot

The weighted average of the strip centres, $\bar{x}$, gives the deflection angle of the line:

$$\theta = -\arctan\left((\bar{x} - w/2) / (h/2) \right) \quad (5.5)$$

where w and h are the image width and height. A positive θ means the line lies to the left and the robot must turn left. Figure 5.13 shows the three strips and the detected centres on a straight section and at the start of the curve.

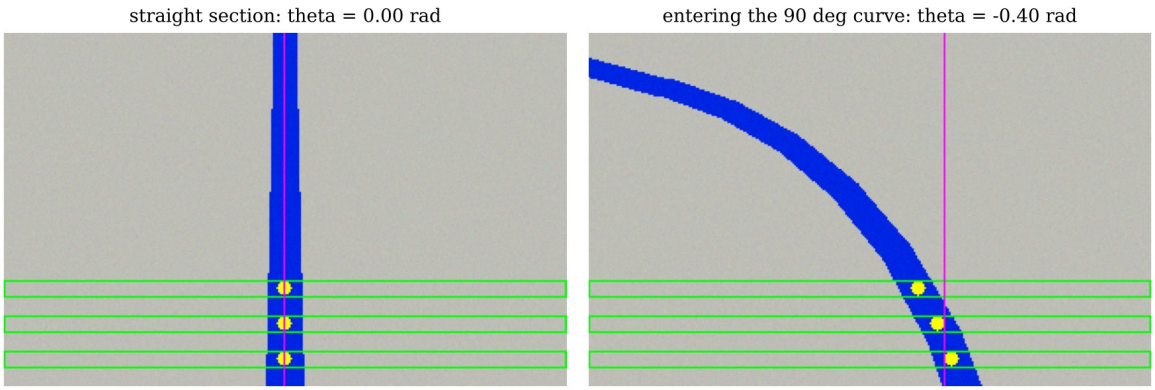


Figure 5.13: Image strips and detected line centres (yellow) on a straight section and entering the curve

5.6.3 Steering Control

A proportional-derivative (PD) controller converts the deflection angle into a turn rate, and the forward speed is reduced in sharp turns:

$$\omega = K_p \cdot \theta + K_d \cdot d\theta/dt, \text{ limited to } \pm 0.6 \text{ rad/s, with } K_p = 1.2 \text{ and } K_d = 0.1 \quad (5.6)$$

$$v = 0.08 \cdot \max(0.4,\ 1 - |\theta| / 0.6) \text{ m/s} \quad (5.7)$$

The derivative is computed from the time stamps of successive images, so the controller is correct at any frame rate and any real-time factor. The robot stops when red pixels cover more than 15 % of the strips in two consecutive frames, which marks a stop point; requiring two frames prevents a single noisy frame from stopping the robot. If no line is found for 2 simulated seconds, the robot stops and reports that the line was lost (FR-8).

5.6.4 Mobile Base and Turn-Round Manoeuvre

The mecanum base can move forward, sideways and rotate. Velocity commands are given in the robot's frame as a forward speed, a sideways speed and a turn rate, and the base reports its position

and heading as odometry. Closed-loop moves built on this interface are used for the turn-round at the patient's bed and for parking at the start: a sideways or forward move continues until odometry shows that the requested distance has been covered, and a rotation continues until the heading has changed by the requested angle. Each move stops if the robot moves away from its goal by more than 5 cm, which guards against a wrong goal.

A robot of this size sweeps a circle of about 0.27 m radius when it rotates in place, while at the bedside stop the table is only 4-5 cm from its side. The turn-round therefore has three steps (Figure 5.14): move 0.12 m sideways away from the table, rotate 180° using odometry, and move 0.12 m sideways back onto the line, now facing home (NFR-4).

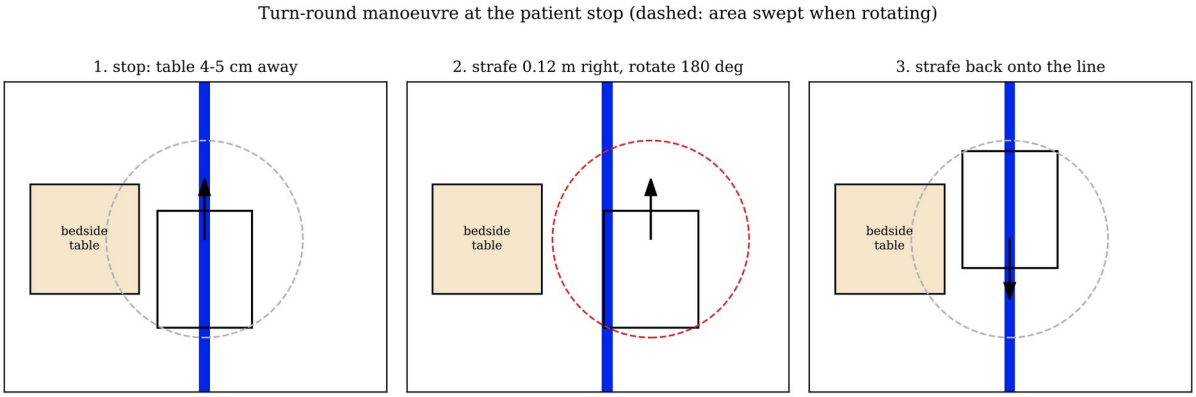


Figure 5.14: Turn-round manoeuvre at the patient stop

5.7 Environment Design

The simulated environment has two parts: the ward corridor with the marked path, in which the robot travels, and the patient room beside the patient stop, in which the delivery takes place. Both are generated by one script from a set of parameters, so the layout can be changed and regenerated consistently, and the script checks the layout before it writes the world file.

5.7.1 Ward Layout and Path

The layout is shown in Figure 5.15. The robot starts at the origin facing along the path, with the medicine station on its left. A 30 mm wide blue path runs 1.42 m straight, turns 90° to the left on a 0.5 m radius and runs 1.25 m to the patient. A 0.15 m red square marks each stop: one at the patient's end and one behind the start, used when the robot returns. The medicine station is 0.10 m high and the lowered bedside table platform is at the same height, level with the arm's base, so the same arm reach (0.20 m to the left) serves both pick and place.

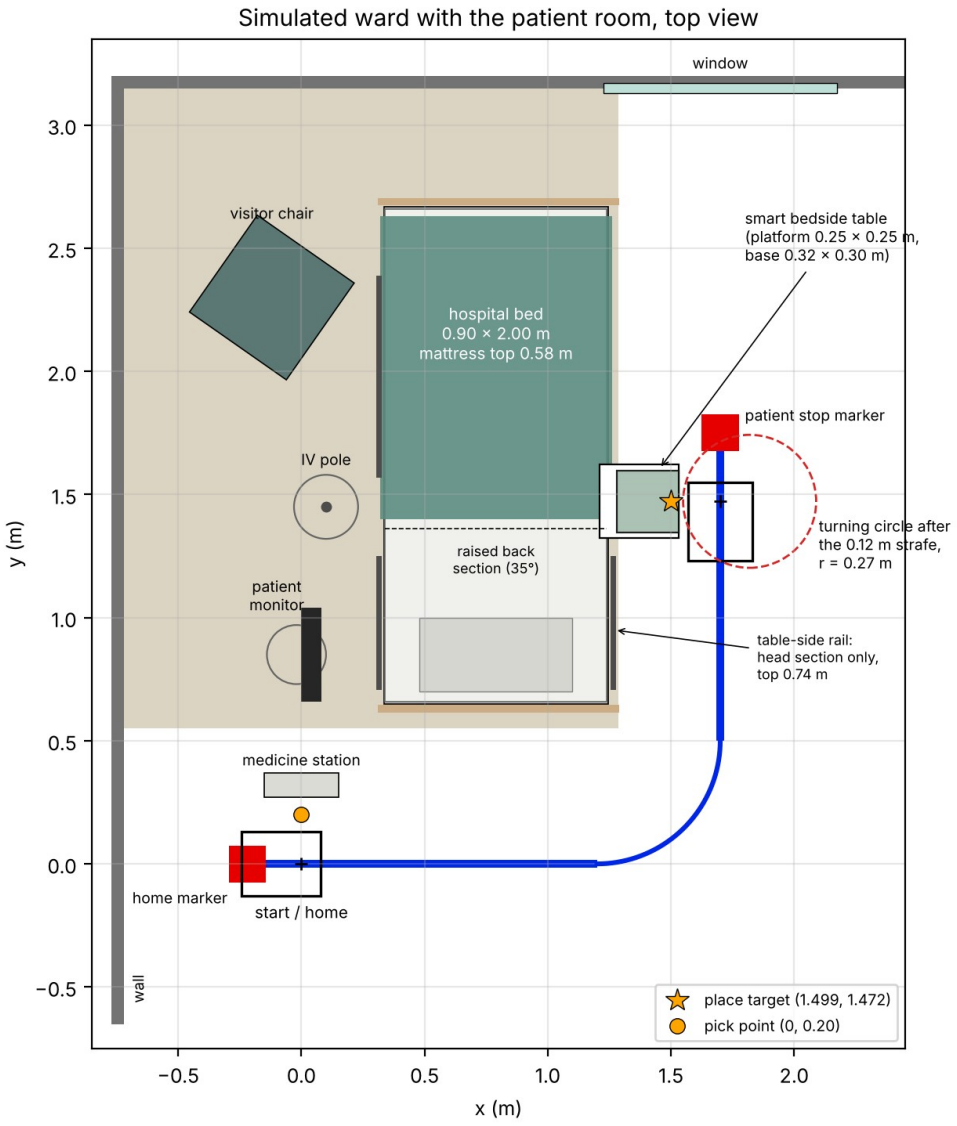


Figure 5.15: Top view of the simulated ward and patient room, with the start pose, the patient stop and the turning circle

5.7.2 Patient Room

The first version of the ward represented the bed and the bedside table by two plain blocks. For the final version the patient area was designed as a furnished hospital room, so that the demonstration shows the robot in the setting for which it is intended and so that the bedside geometry, which decides whether a patient can actually reach the medication, is realistic. The room occupies the area to the robot's left at the patient stop and contains:

- **Hospital bed.** A full-size bed, 0.90 × 2.00 m, with a wheeled base, a frame with two lift columns that looks height-adjustable, a white mattress whose top is 0.58 m above the floor,

a pillow on a back section raised by 35°, a teal blanket, a headboard and footboard with a patient chart, and split side rails. On the far side the bed has a head rail (top 0.88 m) and a foot rail (top 0.85 m). On the table side it has only a head rail, whose top is 0.74 m and which ends 8 cm before the table, so the patient can reach over or past it to the raised platform.

- **IV pole.** A steel pole 1.9 m high on a five-leg wheeled base, with a hook, an IV bag, a drip chamber and a drip line running to the bed.
- **Patient monitor.** A dark monitor housing on a wheeled stand near the head of the bed, whose screen shows an ECG trace with a heart rate of 72, a plethysmograph with an SpO2 of 98 %, a respiration trace and a blood-pressure reading.
- **Visitor chair.** An upholstered chair with armrests at the foot of the bed on the side away from the robot, turned towards the bed.
- **Room.** Two walls 2.0 m high with skirting, a window, a whiteboard and a wall clock, and a light beige floor area that distinguishes the room from the white corridor floor.
- **Smart bedside table.** The table that replaces the old bedside block, described in Section 5.7.3.

The room had to look realistic without disturbing the robot. Three rules were therefore built into the generator, which refuses to write a world that breaks them. First, no furniture may have a blue hue (hue 100-130 in OpenCV) or a red hue, so that nothing but the tape and the stop markers can be mistaken for the path or a stop; for the same reason the monitor's screen uses only green, yellow and white, and the medicine station, which had a faint lavender tint, was changed to neutral grey. Second, every item must stay at least 0.165 m from the centre of the path (half the robot's width plus a margin) and more than 0.285 m from the centres of the two turning circles. Third, all furniture is static and built from simple boxes, cylinders and spheres, with collision geometry only where something can touch it (the bed and the table), so that the furniture, which brings the world to 111 visual elements, costs as little simulation time as possible.

5.7.3 Smart Bedside Table

Purpose. The arm can place the container only at the height of its own base plate, 0.10 m above the floor, while a patient lying in bed reaches at about mattress height, 0.58 m. Rather than raising the robot, which would change its stability and its camera view, the table rises instead. The smart bedside table receives the container at the robot's height and then presents it at the patient's height.

Structure. The table (Figure 5.16) consists of three parts:

- a flat, matte white base 0.32 × 0.30 m and only 5 cm high, standing on the floor partly under the edge of the bed;
- a 25 × 25 cm light grey platform with a pale green sensor pad, whose top is at 0.10 m when the table is lowered and at 0.80 m when it is raised;
- a telescopic column of three nested steel tubes (radii 40, 32 and 24 mm) between the base and the platform.

When the table is lowered, the column is hidden: a 5 cm base cannot hold 0.70 m of tube, so the collapsed tubes extend below the floor, where they are invisible and have no collision geometry. A green status light on the base shows that the table is powered.

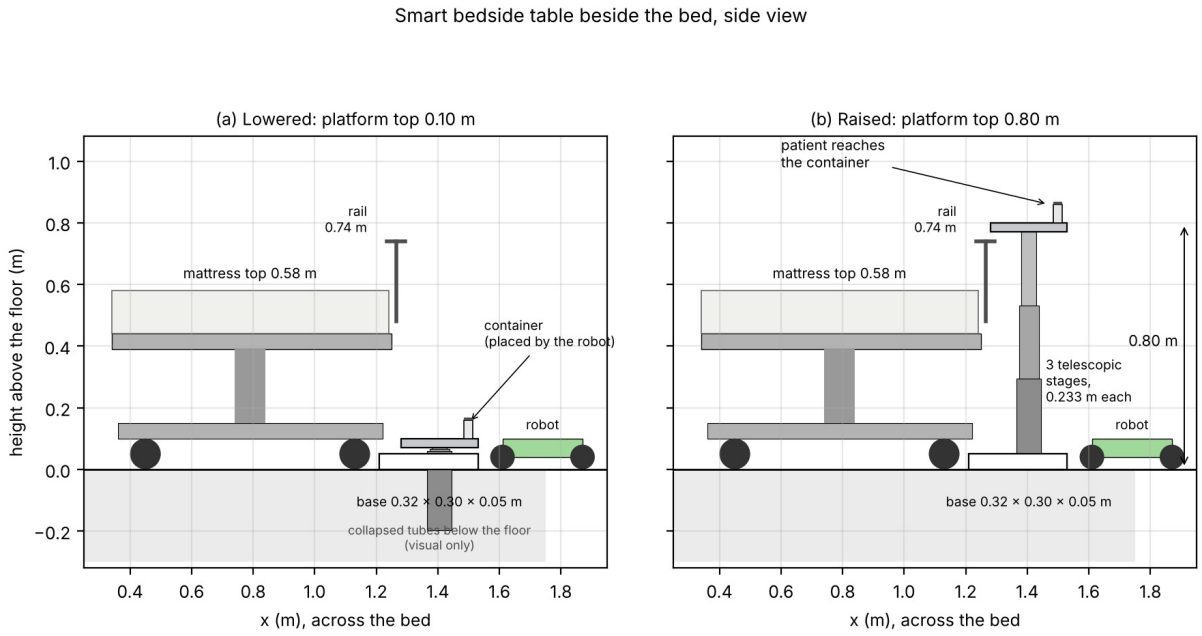


Figure 5.16: Side view of the smart bedside table beside the bed: (a) lowered to the robot's placing height, (b) raised above the mattress for the patient

Placement. The platform is centred on the place target along the robot's direction of travel, but not across it. The place target is fixed by the arm's reach, 0.20 m to the left of the arm's base at the patient stop. A 25 cm platform centred on it would reach into the robot's body by 1.4 cm and into its wheels by 5.4 cm. The platform's edge on the robot's side was therefore kept where the edge of the old table was, 4 cm from the wheels, which also keeps the table 0.289 m from the centre of the robot's turning circle (at least 0.27 m is required). The target lies 31 mm inside that edge, which leaves 16 mm between the side of the 30 mm container and the edge of the platform.

Drive. Each tube is mounted on its own vertical prismatic joint, and the three joints are connected in series (base, tube 1, tube 2, tube 3 with the platform). Every stage moves an equal share of the travel, 0.233 m, so the platform height is 0.10 m plus the sum of the three stage positions. Each joint is driven by a position controller in velocity mode, capped at 20 mm/s per stage. A controller node moves the platform along a trapezoidal profile at 40 mm/s with an acceleration of 40 mm/s² at the start and end (Figure 5.17), which lifts the full 0.70 m in a calculated 18.5 s and keeps the 30 g container from sliding. The lift times reported in Chapter 7 (18.4-18.7 s) are taken from the time stamps of the table's state messages, so they can differ from the calculated value by up to about 0.2 s.

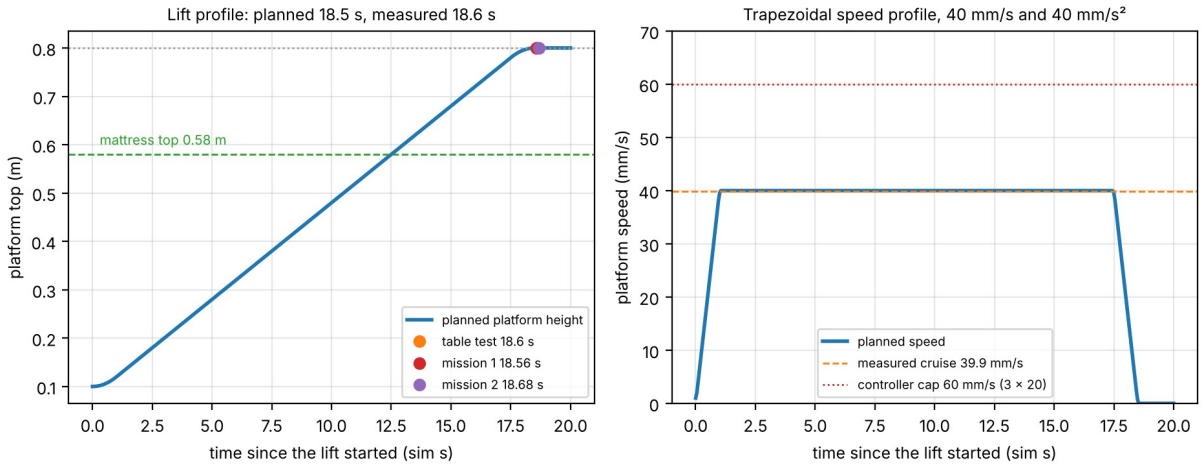


Figure 5.17: Planned lift profile of the platform and the measured lift times; the platform passes the mattress height after about 12.5 s

Sensing. A contact sensor on the platform's top surface reports every object that touches it, at 50 Hz. The controller treats the container as present while contact messages arrive less than 0.3 s apart, and as "steady" once the contact has lasted 2 s. The table also publishes the positions of its three stages, from which the controller computes the platform height.

Control logic. The controller is a state machine with four states (Figure 5.18). From *lowered* it starts *lifting* only when all three conditions hold: the container has been in steady contact with the platform for at least 2 s, the gripper has released it (the weld state reports "detached"), and the arm is back in its folded home pose, with every joint within 0.06 rad of it. The last condition guarantees that the gripper is never above the platform while it moves. Once the platform reaches 0.80 m the table is *raised*. When the patient takes the container and the platform has been free of contact for 3 s, the table starts *lowering* and becomes *lowered* again, ready for the next delivery. For testing, the table can also be raised and lowered by hand, but these commands are refused unless the arm is home or the gripper is more than 0.10 m from the platform.

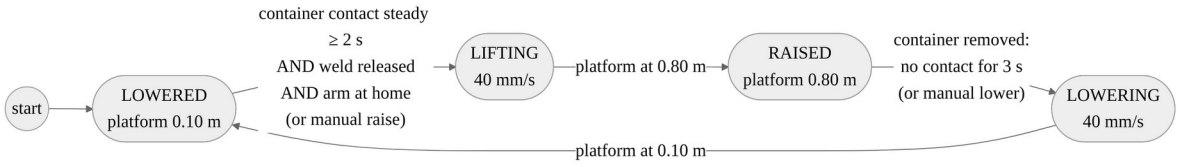


Figure 5.18: State machine of the smart bedside table

The table reports its state, the platform height and the contact state on three topics (Table 5.2), which the mission controller uses to wait for the lift and which are recorded in every result file.

5.8 Mission Design

5.8.1 Delivery Sequence

One delivery is a fixed sequence of eleven steps (Figure 5.19):

1. Place the container at the station and release any initial weld.
2. Move the arm to its ready pose.
3. Pick the container: open, approach, descend, close, attach and lift.
4. Return the arm to the ready pose while carrying the container.
5. Follow the line to the patient's stop marker.
6. Wait until the bedside table reports that it is lowered, then place the container on it: approach, descend, release, open and retreat.
7. Return the arm to the ready pose.
8. Wait while the table lifts the medicine to the patient, then check that the container is still on the raised platform.
9. Turn round: move sideways away from the table, rotate 180° and move back onto the line.
10. Follow the line to the home marker.
11. Park at the start position, ready for the next delivery.

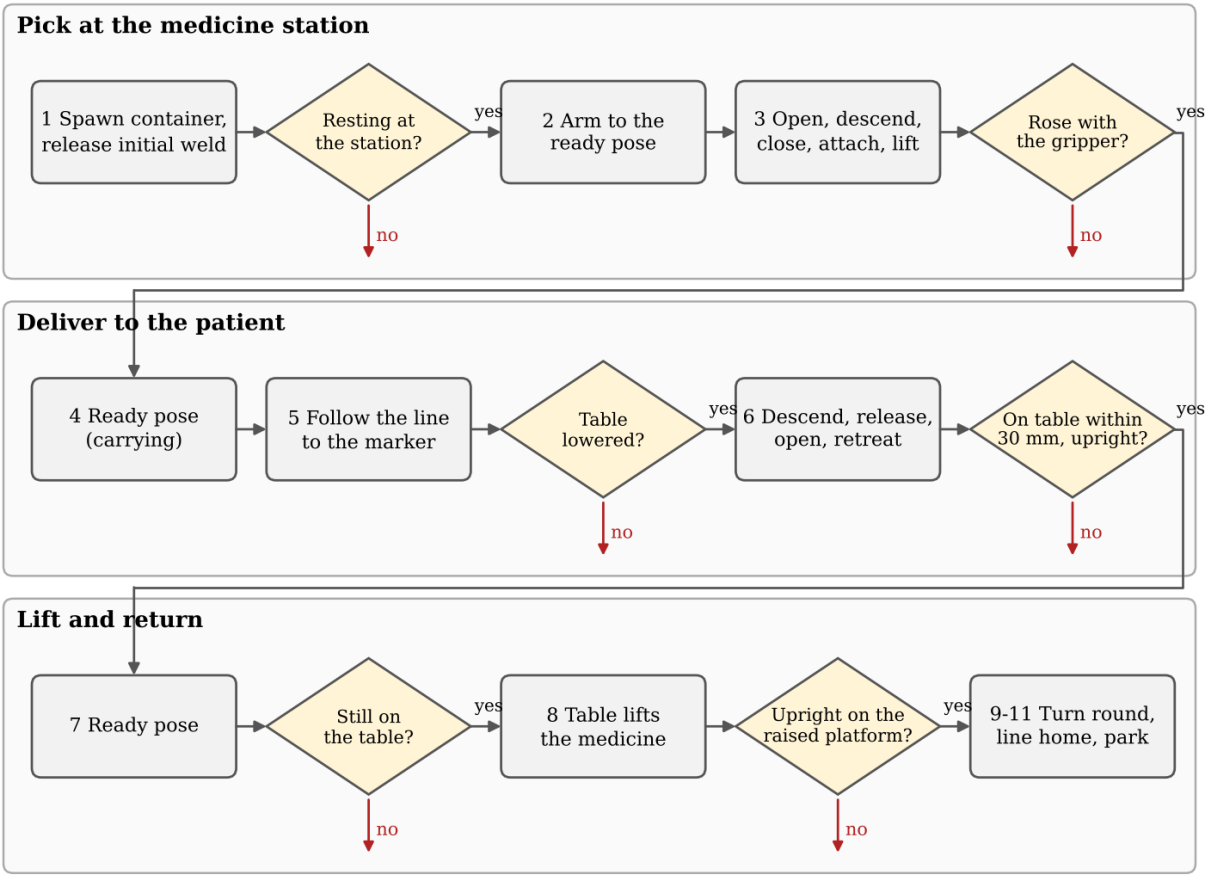


Figure 5.19: Delivery mission flow with its safety checks

Five checks guard the sequence. After step 1 the container must be resting at the station; after step 3 it must have risen with the gripper; after step 6 it must stand on the bedside table within 30 mm of the target; after step 7 it must still be there; and after step 8 it must stand upright on the raised platform. The placement is measured at the moment the table's contact sensor confirms the container, while the table is still lowered, so the placement error means the same as before the table was added. A failed check, a lost line or a timeout stops the robot and records the failed step and its error (FR-6). The lift is the one exception: if the table has not risen within 90 simulated seconds, the robot still returns home, because the container is safely on the table, and the result records that the table did not rise.

5.8.2 Mission States

The same sequence can be seen as a state machine (Figure 5.20). The robot is idle until a delivery falls due; it then passes through preparing, picking, carrying, travelling, placing, presenting (while

the table lifts the medicine), returning and parking, and returns to idle once the result is logged. Any failure leads to the fault state, in which the robot is stopped, the error is logged and the robot waits for a caregiver.

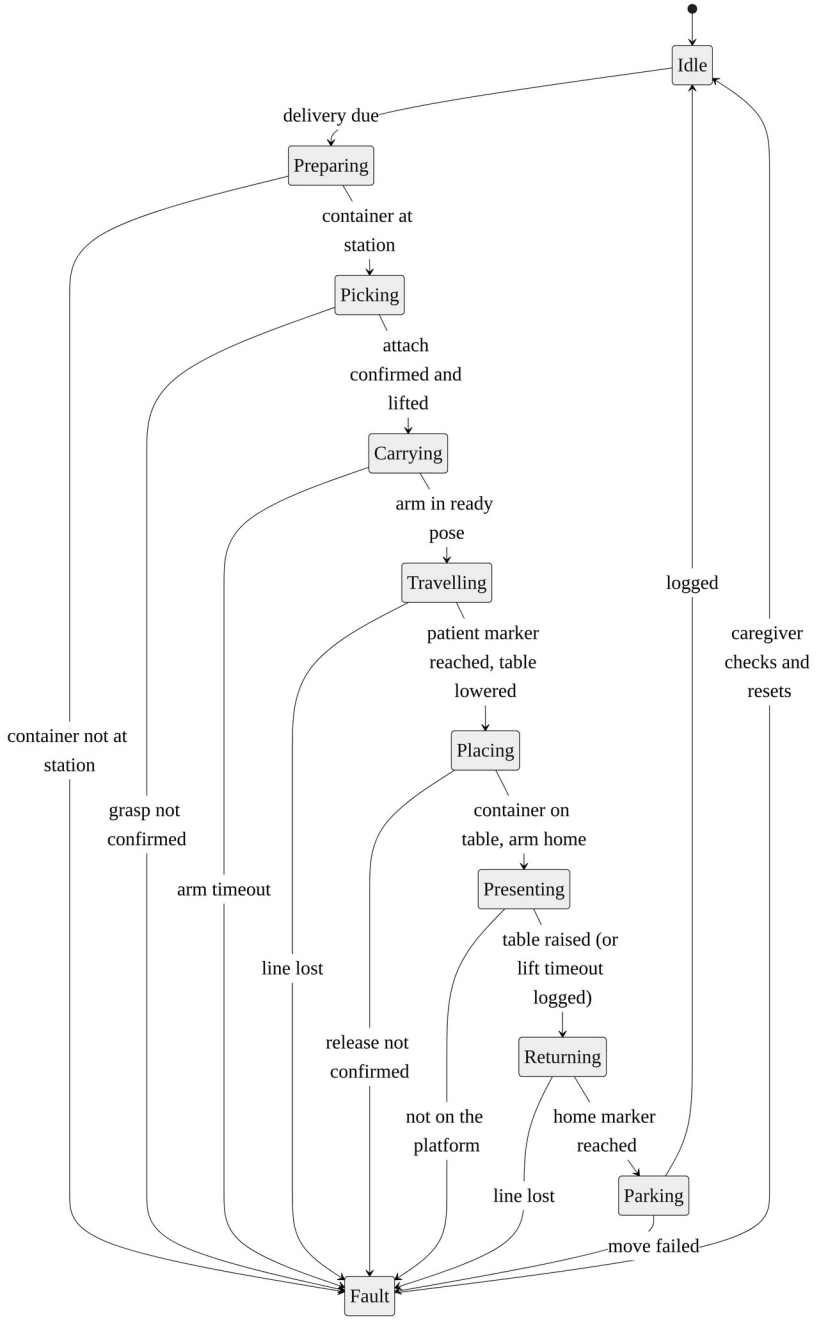


Figure 5.20: State machine of a delivery mission

5.8.3 Interaction Between Components

Figure 5.21 shows the sequence of messages between the scheduler, the mission controller, the arm and weld control, the line follower, the smart table and the robot during a delivery. Every command is followed by a confirmation from the robot before the mission continues.

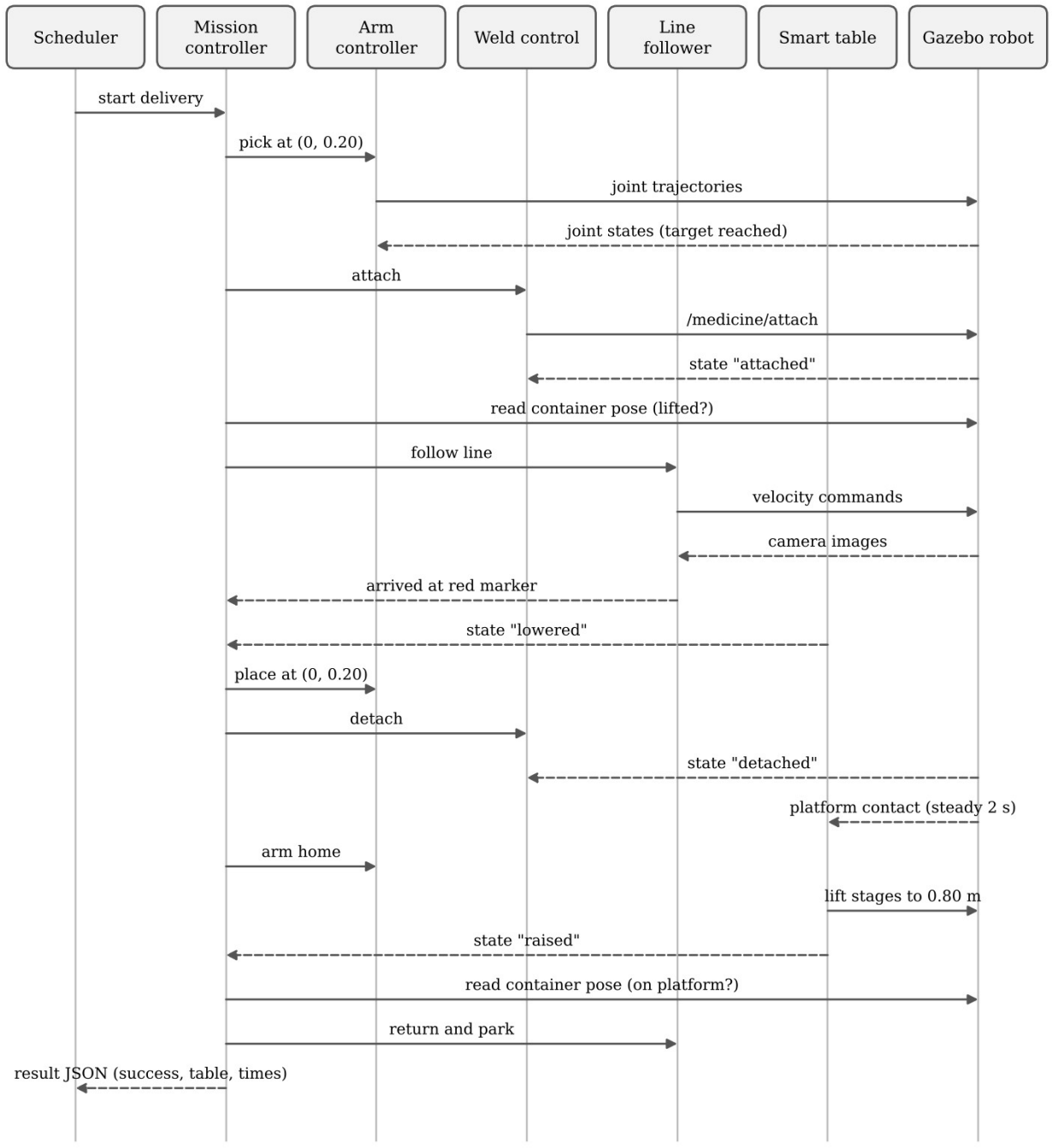


Figure 5.21: Sequence diagram of one delivery

5.9 Scheduler and Data Design

5.9.1 Schedule File

The schedule is a YAML file with a list of deliveries, each with a patient, a medicine, a bed and a time. A time of the form "HH:MM" repeats every day; a date and time is a one-off delivery. The listing below is an example that shows both forms; the schedule file in the repository uses a daily time for all three entries. For testing, the scheduler can run any entry a given number of minutes from now. Figure 5.22 shows how the scheduler processes the schedule.

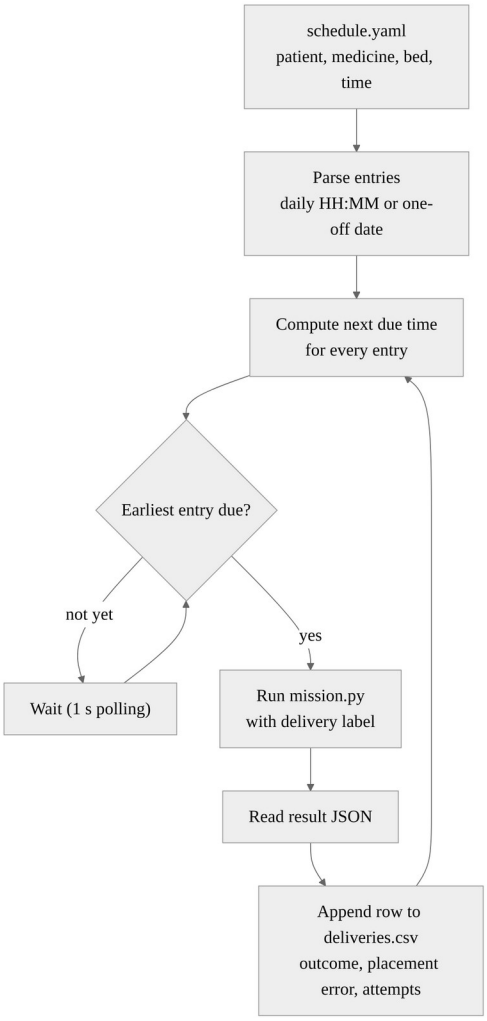


Figure 5.22: Scheduler processing loop

```
deliveries:
  - patient: "Patient A"
    medicine: "Paracetamol 500 mg"
    bed: 1
    time: "08:00"           # every day at 08:00
  - patient: "Patient A"
```

```
  medicine: "Amoxicillin 250 mg"
  bed: 1
  time: "14:00"
- patient: "Patient A"
  medicine: "Paracetamol 500 mg"
  bed: 1
  time: "2026-11-05 20:00"  # one-off delivery
```

5.9.2 Delivery Log and Result Files

Each mission writes a result file in JSON format with its outcome, the failed step if any, the container's and robot's final poses, the placement error, the container tilt, the number of attempts each gripper command needed, the record of the smart table (its states with their times, the lift time, the platform height and the container's pose after the lift), and the simulated and wall-clock durations. The scheduler appends a summary row to the delivery log, a CSV file with one row per delivery (Table 5.3). The evaluation tool reads the same result files, so a delivery is always scored the same way whether it was started by the scheduler or by an evaluation.

Field	Content
due, started, finished	Due time, start and finish time of the delivery
patient, medicine, bed	Delivery details from the schedule
success, failed_step, error	Outcome, the step that failed and its error message
placement_error_mm, at_target	Horizontal distance of the container from the target; within 30 mm
tilt_deg, on_table	Container tilt from vertical; container resting on the table
spawn_detach_attempts, pick_attach_attempts, place_detach_attempts	Attempts each gripper command needed before it was confirmed
weld_unconfirmed	Any gripper command that was never confirmed
table_raised, lift_time_s	Whether the smart table rose to the patient, and how long the lift took
container_on_platform_after_lift	Container centred on the raised platform, standing on it and upright
sim_time_s, wall_time_s	Simulated and wall-clock duration of the mission
result_json	Path of the full result file

Table 5.3: Fields of the delivery log (deliveries.csv)

5.10 Technical Choices

5.10.1 Programming Language

All TrackMed programs are written in Python 3 using the ROS 2 Python client library (rclpy). Python is supported by the manufacturer's own examples, has mature libraries for vision (OpenCV) and numerical work (NumPy), and is quick to change and test. The computational load is small

(one 320 × 200 image every 0.25 simulated seconds and a few closed-form calculations per motion), so Python's speed is not a limitation.

5.10.2 Development Environment

Development used Ubuntu 22.04 with ROS 2 Humble and Gazebo Fortress in a VMware Workstation virtual machine, the environment supplied by the manufacturer for its simulation course [25]. Claude, an AI assistant by Anthropic, helped with the code and with this report, under the team's direction: in the virtual machine (through Claude Code) it wrote and ran code and simulation tests, and it helped draft and check the report. The team set the goals and the plan, made the design decisions, reviewed the results and every change, and committed the work with Git; the commits made with the assistant's help carry its name as co-author.

5.10.3 Simulator and Rendering

Gazebo Fortress was chosen because it is the simulator paired with ROS 2 Humble and the one supported by the manufacturer's packages. In the VMware virtual machine, Gazebo's default renderer (OGRE 2) produced a blank view, although the virtual graphics adapter supports OpenGL 4.3. The older OGRE 1 renderer works and is used throughout, both for the graphical view and for rendering the camera images.

5.10.4 Data Storage

Schedules, results and logs are stored as plain YAML, JSON and CSV files on the robot's computer. These formats need no database server, can be read by people and by any spreadsheet or analysis tool, and are simple to back up and to place under version control.

5.11 UI/UX Design

5.11.1 User Interface Considerations

The prototype's interface for caregivers is the schedule file and the delivery log. Entries are short and self-explanatory, and the log can be opened in a spreadsheet and sorted or filtered by patient, date or outcome. A failed delivery shows the step that failed and an error message in plain words, so that a caregiver can see what to check.

5.11.2 User Experience Considerations

For patients, the experience is designed to be predictable: the robot arrives along a visible path at the scheduled time, stops beside the bed, places the container on the bedside table within reach and leaves. It never hands the container directly to the patient and never moves its arm towards the bed. For caregivers, the key experience is trust: every delivery either succeeds and is logged, or stops

and is logged with its reason. A future caregiver application (Section 9.4) would present the same information with notifications.

5.12 Hospital System Design

After the single-room system had been evaluated, the project supervisor asked for it to be extended into a complete, if small, hospital system: four patient rooms instead of one, a medicine storage unit holding different medicines for different patients, and a robot that decides by itself, from a medication schedule and the current time, which medicine to fetch and to which room to take it. This section describes the design of that extension. It reuses every tested part of the single-room system unchanged (the arm's pick and place, the line follower's image processing and steering, the smart bedside table and the confirmed attach and release) and adds the parts needed to choose between several medicines, rooms and routes.

5.12.1 Design Goals and Constraints

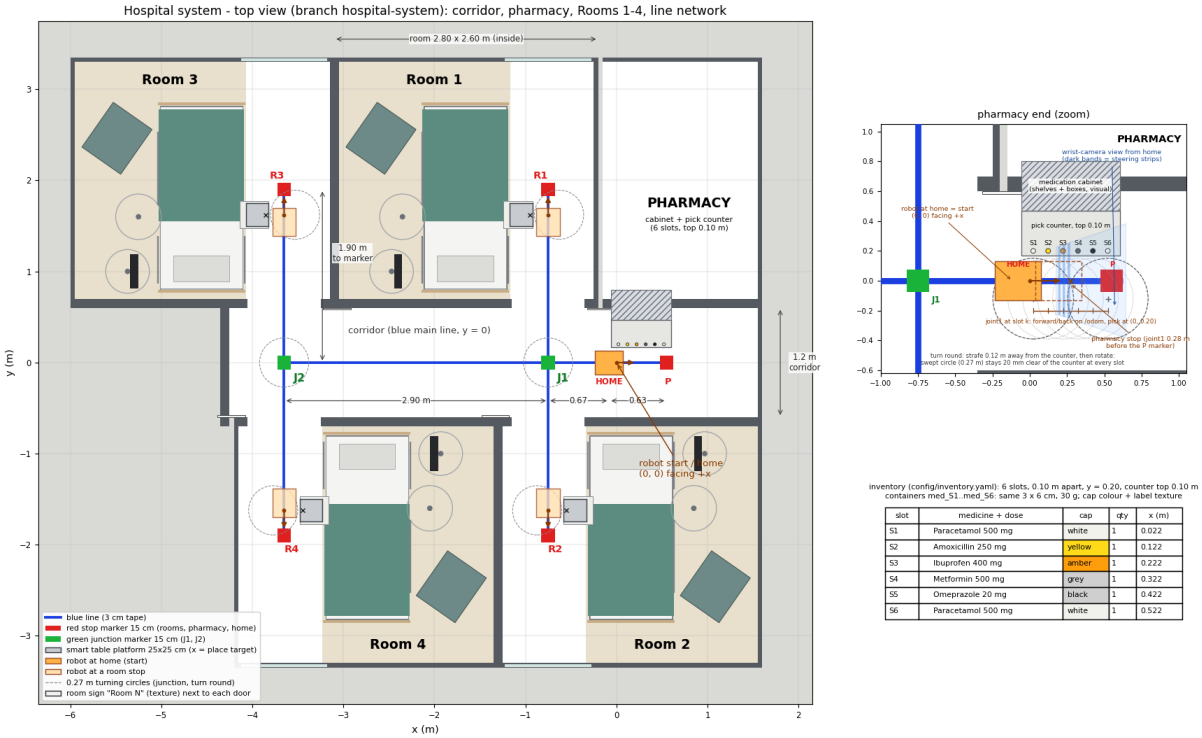
The extension was planned before anything was built, and the plan fixed five constraints:

- **Reuse what was tested.** Every room is an exact copy of the tested patient room, and the robot's pose relative to each bedside table is identical, so the place motion and the table's behaviour need no new tuning.
- **Navigation on a line network.** The robot has a camera but no LiDAR, and the simulation runs at only about 4 to 9 % of real time during a mission, so map-based navigation would be far too slow. A network of lines with marked junctions keeps the tested line follower and is predictable and easy to check, so the single path was extended into such a network.
- **Compact world.** The longest route had to stay below about 6 m one way, to keep a delivery within a reasonable wall-clock time.
- **Simulation speed.** The real-time factor (RTF) had to stay at or above 7 % with all four rooms (NFR-11).
- **The single-room version stays intact.** The hospital system was developed on its own branch, so the evaluated single-room system remained available unchanged.

5.12.2 Building Layout

The hospital (Figures 5.23 and 5.24) consists of a corridor 1.2 m wide and 5.8 m long, with the pharmacy at its east end and four patient rooms off it, two on each side. A blue main line runs along the middle of the corridor. Two junctions, marked by green 15 cm squares, connect it to the rooms: J1, a four-way junction, leads to Room 1 (north) and Room 2 (south), and J2, a T junction at the west end, leads to Room 3 (north) and Room 4 (south). A branch line runs from each junction into

its room and ends at the room's red stop marker, in front of the smart bedside table. Red stop markers also mark the pharmacy counter and the robot's home, which is right next to the pharmacy, so each delivery starts and ends a few centimetres from where the medicine is collected.



inventory (config/inventory.yaml): 6 slots, 0.10 m apart, y = 0.20, counter top 0.10 m
containers med_S1..med_S6: same 3 x 6 cm, 30 g; cap colour + label texture

slot	medicine + dose	cap	qty	x (m)
S1	Paracetamol 500 mg	white	1	0.022
S2	Amoxicillin 250 mg	yellow	1	0.122
S3	Ibuprofen 400 mg	amber	1	0.222
S4	Metformin 500 mg	grey	1	0.322
S5	Omeprazole 20 mg	black	1	0.422
S6	Paracetamol 500 mg	white	1	0.522

Routes (every delivery: home -> pharmacy slot -> room -> home; pick at arm-relative (0, 0.20) as tested) - times = measured step times of the 2026-11-05 missions

delivery	leg 1 home->P	leg 2 P->room (actions)	leg 3 room->home (actions)	sim s	wall @5.5%	wall @4.5%
Room 1	0.27 m	2.64 m: turn round, J1 right 90 deg, stop at red R1	2.01 m: turn round, J1 left 90 deg, stop at red HOME	186	56 min	69 min
Room 2	0.27 m	2.64 m: turn round, J1 left 90 deg, stop at red R2	2.01 m: turn round, J1 right 90 deg, stop at red HOME	186	56 min	69 min
Room 3	0.27 m	5.54 m: turn round, J1 straight, J2 right 90 deg, stop at red R3	4.91 m: turn round, J2 left 90 deg, J1 straight, stop at red HOME	262	80 min	97 min
Room 4	0.27 m	5.54 m: turn round, J1 straight, J2 left 90 deg, stop at red R4	4.91 m: turn round, J2 right 90 deg, J1 straight, stop at red HOME	262	80 min	97 min
demo R1-R4				897	4.5 h	5.5 h

Figure 5.23: Top view of the hospital system: corridor, pharmacy, four patient rooms and the line network, with the clearance checks

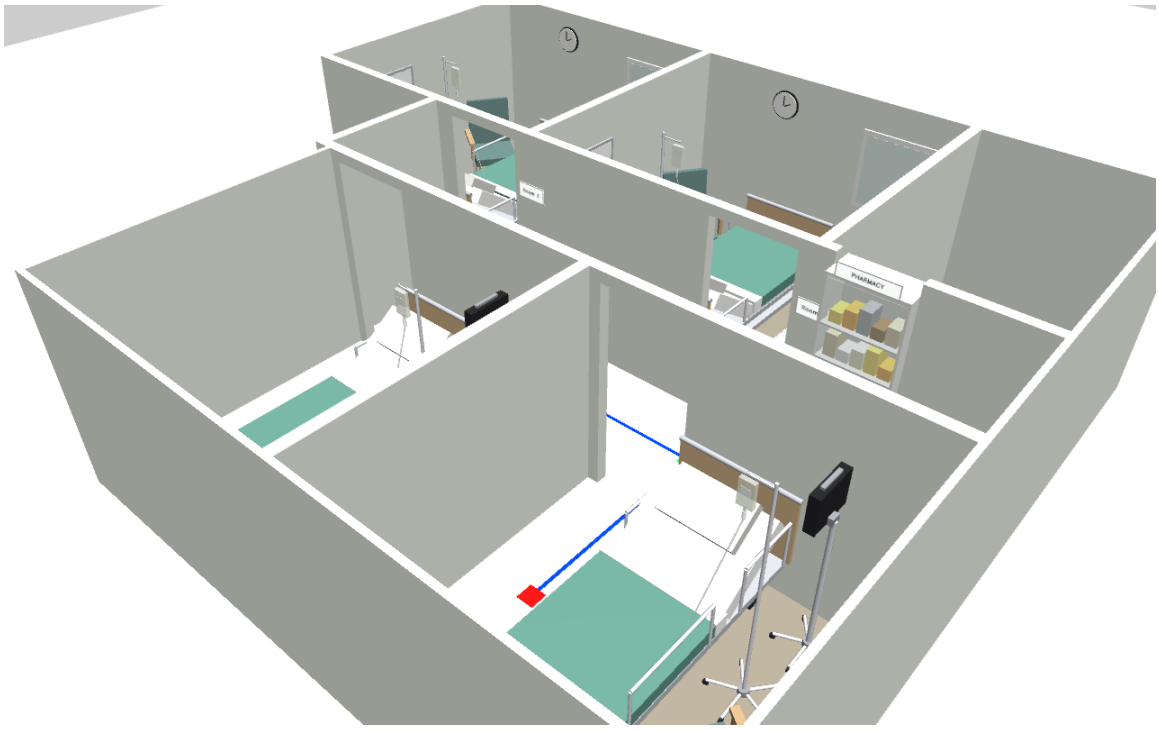


Figure 5.24: The simulated hospital in Gazebo, with the corridor, the pharmacy and the four furnished patient rooms

All four rooms are generated by one function from a room origin and a rotation; the two south rooms are the north rooms rotated by 180° about their junction. Each room contains the full furniture of the patient room of Section 5.7.2 (bed with split rails, IV pole, patient monitor, visitor chair, wall details) and one smart bedside table with exactly the design of Section 5.7.3. Because every room is the same room moved and rotated, the distance from the junction to the room's stop position (1.622 m), the place target 0.20 m to the robot's left, the platform's edge and the turning circle are identical in all four rooms. Each room's inside measures 2.80 × 2.60 m, and a sign on the wall gives its number. The world generator also extends the colour and clearance checks of the single room: no furniture may be blue, red or, near the floor, the junction green; every object stays at least 0.165 m from every line and outside every 0.27 m turning circle; and the generator draws the checks into the layout picture (Figure 5.23).

5.12.3 Medicine Storage Unit

The pharmacy (Figure 5.25) is a hospital medication cabinet with shelves of boxes (visual only) behind a pick counter whose top is at 0.10 m, the height of the arm's base plate, as at the original medicine station. The counter has six slots, 0.10 m apart along the corridor and 0.20 m to the left of the line, so that the arm's tested pick at (0, 0.20) applies at every slot. All six containers have the size and mass of the original container (3 × 6 cm, 30 g), so the gripper needs no change; they differ

only in the colour of their cap and in a printed label with the medicine's name, dose and slot. The inventory (Table 5.4) is a configuration file; Paracetamol is stocked in two slots, so that the system can also be shown to use a second container of the same medicine.

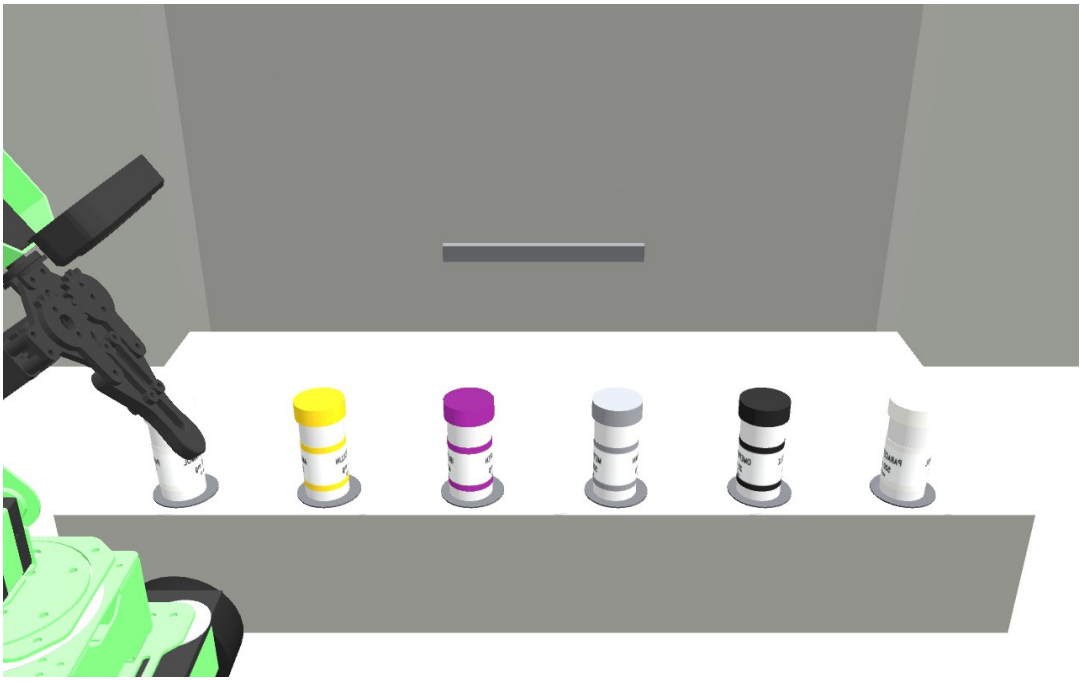


Figure 5.25: The medicine storage unit: six medicine containers with different caps on the pick counter, with the robot lined up at a slot

Slot	Medicine	Dose	Cap colour
S1	Paracetamol	500 mg	White
S2	Amoxicillin	250 mg	Yellow
S3	Ibuprofen	400 mg	Purple
S4	Metformin	500 mg	Grey
S5	Omeprazole	20 mg	Black
S6	Paracetamol	500 mg	White

Table 5.4: Inventory of the medicine storage unit

To reach a slot, the robot stops at the pharmacy's red marker and then moves along the line by odometry until its arm is level with the chosen slot. Because the counter runs alongside the line, this needs no sideways move away from the line. To turn round afterwards, the robot first moves 0.12 m sideways away from the counter, so that its 0.27 m turning circle stays 2 cm clear of the counter at every slot, then rotates by 180° and moves back onto the line.

5.12.4 Line Network and Route Planning

The line network is described by a map file as a graph (Figure 5.26). Its nodes are the pharmacy, home, the two junctions and the four rooms; its edges carry their length and heading; and a turn table gives, for every way of passing through a junction, whether the robot goes straight, turns left or turns right. A route planner finds the shortest path between two nodes with Dijkstra's algorithm and turns it into a list of events along the way: each marker the robot will meet, its colour, its expected position and the action to take there (pass, straight, left, right or stop). Every delivery consists of three such routes: home to the pharmacy, the pharmacy to the room, and the room back home. Table 5.5 lists the routes of the four rooms; the longest, to Rooms 3 and 4, is 5.54 m from stop to stop.

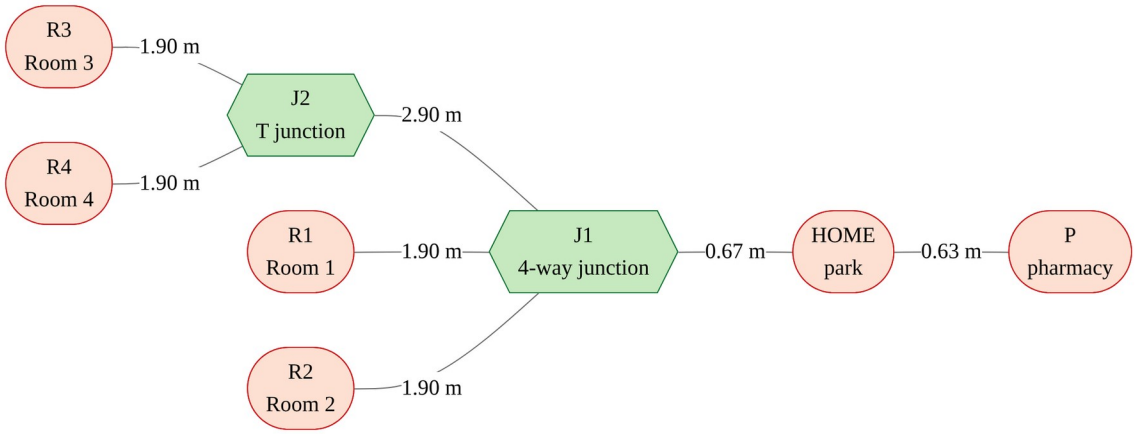


Figure 5.26: The line network as a graph, with the length of each edge

Room	Pharmacy → room	Room → home
1	Turn round, pass home, right at J1, stop	Turn round, left at J1, stop at home
2	Turn round, pass home, left at J1, stop	Turn round, right at J1, stop at home
3	Turn round, pass home, straight at J1, right at J2, stop	Turn round, left at J2, straight at J1, stop at home
4	Turn round, pass home, straight at J1, left at J2, stop	Turn round, right at J2, straight at J1, stop at home

Table 5.5: Routes from the pharmacy to each room and back home, as planned by the route planner

5.12.5 Junction Handling

The route follower extends the tested line follower without changing its image processing or its steering. It adds a look-ahead band in the camera image, 0.19 to 0.35 m ahead of the robot, in which it looks for markers, and it handles each marker as shown in Figure 5.27. A marker is accepted only if three things agree: its colour, its place in the planned order of events, and its position, measured

by odometry, within a window around the planned position. If a marker appears that does not match, or if a planned marker is not seen by the time it should be less than 0.12 m ahead, the robot stops with an error rather than guessing. In the tested fault cases (Section 7.7.1), a missing or wrong marker was detected and stopped the robot before it could take a wrong turn.

At an accepted marker the robot carries out the planned action. To go straight through a junction or pass the home marker, it drives "blind" on odometry, holding the line's heading, until it is 5 cm past the marker's centre, so that the branch line cannot pull it off course. To turn, it stops, moves onto the junction's centre and rotates by ±90° on odometry, then resumes following the line. At a red stop marker it uses the tested stop detection. On long straight stretches, more than 0.8 m before the next marker and more than 0.3 m after the last, the robot drives at 0.12 m/s instead of 0.08 m/s; this was adopted only after it had passed the same line test in Gazebo as the original speed (Section 7.7.2).

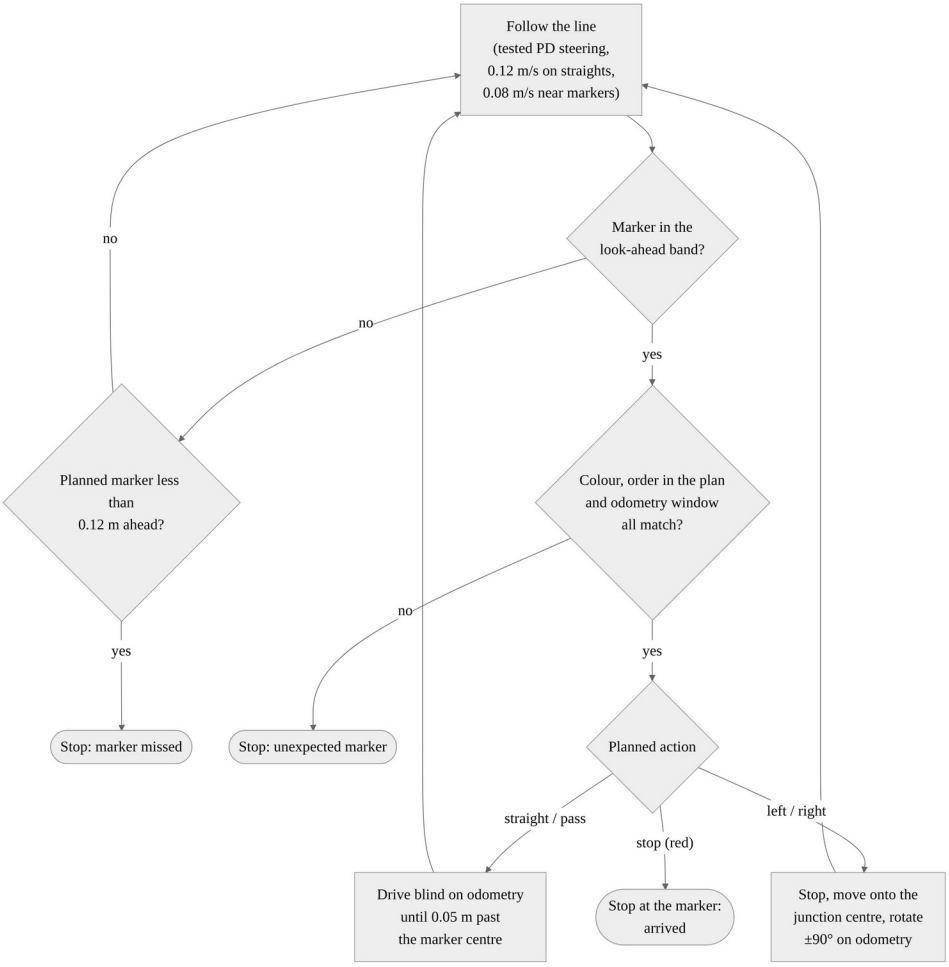


Figure 5.27: Handling of markers by the route follower

5.12.6 Cap Check

Picking the right container from the right slot depends on the inventory file being correct. To catch a container in the wrong slot, the robot checks its cap before picking: the arm turns to a look pose above the slot, the wrist camera takes an image of the cap, and the median colour of a small region of the image is classified (Table 5.6). If the colour does not match the inventory, the delivery stops with the error "wrong medicine?" before anything is picked. The cap colours were chosen so that none of them can be mistaken for the line, the stop markers or the junction markers, and so that the camera can separate them under the simulation's lighting; Section 6.12.4 describes how the first choice of colours failed this test and was changed. The check identifies a container only by its cap colour; it does not verify the medicine inside. Its thresholds were fitted to the colours rendered under this scene's lighting: grey renders at a value of 174 against a white/grey boundary of 175, and black at 55 against a boundary of 70, so the margins are small and the thresholds would have to be recalibrated, with wider margins, for any other lighting or for a real camera.

Class	Rule	Rendered colour (H, S, V)
White	$S < 45$ and $V \geq 175$	(0, 0, 255)
Grey	$S < 45$ and $70 \leq V < 175$	(111, 19, 174)
Black	$V < 70$	(0, 0, 55)
Yellow	$22 \leq H \leq 36$, $S \geq 110$, $V \geq 90$	(30, 200, 255)
Purple	$135 \leq H \leq 165$, $S \geq 90$, $V \geq 60$	(150, 186, 169)

Table 5.6: Classification of the cap colours (OpenCV HSV, hue 0-180, saturation and value 0-255)

5.12.7 Delivery Sequence

A delivery in the hospital system is a sequence of 15 steps, which extends the 11 steps of the single room with the pharmacy and the route:

1. Start check: the robot is at home and the container is in its slot.
2. Arm to the ready pose.
3. Follow the line to the pharmacy.
4. Line up with the slot.
5. Check the cap colour.
6. Pick the container, with the container's own confirmed attach.
7. Arm to the ready pose, carrying the container.
8. Turn round at the pharmacy.

9. Follow the planned route to the room.
10. Place the container on the room's smart table.
11. Arm to the ready pose.
12. Wait while the table lifts the medicine to the patient.
13. Turn round.
14. Follow the planned route home.
15. Park at the home position.

A delivery is scored as successful only if all of the following hold: every step completed; the right container (identified by its model name) stands on the raised platform of the right room's table, centred over it, at the platform height plus 3 cm within 1 cm and tilted less than 10°; it was placed within 30 mm of the target; and every other container is still where it was before the delivery.

5.12.8 Scheduling

The scheduler (Figure 5.28) reads the medication schedule (patient, room, medicine, dose and daily due times), the inventory and the map. In normal use it follows the real local time (Asia/Dubai). When one or more doses are due, it orders them by due time, and doses due at the same time by the length of their route, shortest first, so that the patient who can be served soonest is not kept waiting behind a longer trip. The robot carries one container at a time, so the doses are delivered one after another. Before each delivery the scheduler checks that a slot with the medicine is still full; if not, it logs the dose as out of stock and goes on with the next one. It records the lateness of each dose, the start time minus the due time, and writes one row per delivery to the delivery log and a detailed result file. If a delivery fails after the robot has moved, the scheduler starts no further deliveries, because the robot is no longer known to be at home.

On the virtual machine one delivery takes more than an hour of wall-clock time, although it takes only three to four minutes of simulated time, the time a robot moving at the same speeds would need in reality. With the real clock, a demonstration of several doses would therefore show every dose after the first as hours late. For demonstrations the scheduler therefore uses a hospital clock: simulated time mapped to a chosen start time, 08:00. Lateness and the captions of the demonstration video use this clock, so they show the delays the real robot would cause. A stop file allows a demonstration to be ended after the current delivery.

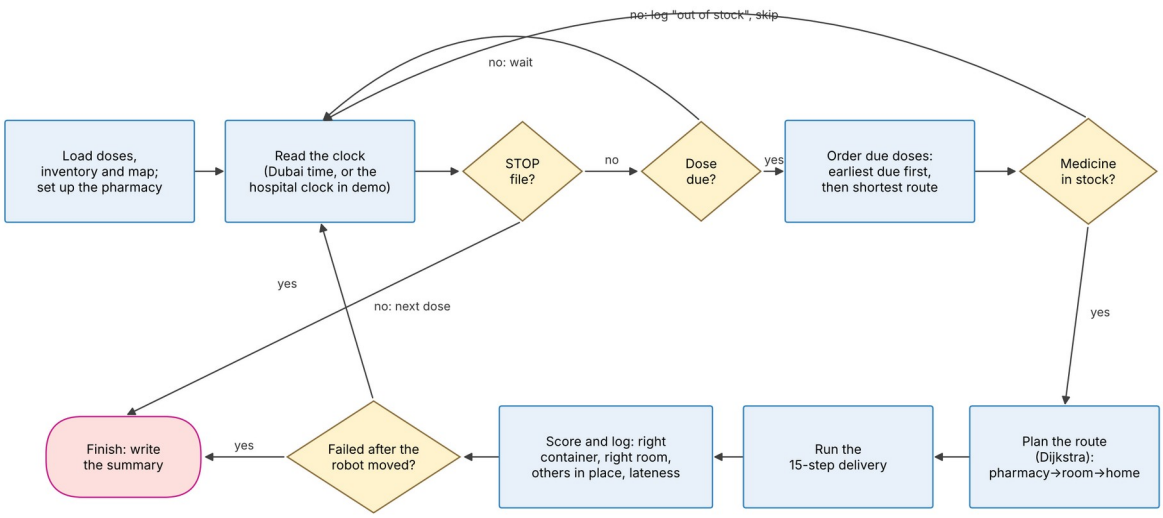


Figure 5.28: Processing loop of the hospital scheduler

6 IMPLEMENTATION

This chapter describes how the design of Chapter 5 was implemented: the development environment and the problems that had to be solved to make it usable, the robot and world models, the arm control and grasp, the mobile base and line follower, the mission controller, the scheduler, the tools used for evaluation and documentation, the patient room with the smart bedside table (Section 6.11) and the hospital system (Section 6.12). Selected source code is reproduced in Appendix C.

6.1 Development Environment

6.1.1 Setup

Development used the virtual machine image supplied by the manufacturer for its ROS 2 simulation course: Ubuntu 22.04 with ROS 2 Humble, Gazebo Fortress 6.16 and the JetArm Pro packages, running in VMware Workstation Pro on a Windows laptop [25]. The packages include the arm's URDF description (`jetarm_6dof_description`), a Gazebo package (`robot_gazebo`) with controllers and launch files, and a MoveIt 2 configuration. TrackMed's own files are kept in a separate repository and copied into the Gazebo package by an install script, so the manufacturer's files are never edited directly.

6.1.2 Problems Solved

Several problems had to be solved before the environment could be used for development. They are listed in Table 6.1, because each one cost time and each has a lesson for anyone repeating this work.

Problem	Cause	Solution
Gazebo showed a blank 3-D view	The default OGRE 2 renderer fails with the VMware graphics adapter, although it reports OpenGL 4.3	Launch Gazebo with the OGRE 1 renderer (`--render-engine ogre`) in every launch file
Administrator password unknown	Pre-configured image	Set a new password in recovery mode and add a passwordless sudo rule for the development user
Shutdown hung for 30 minutes on automatic updates	Unattended upgrades on a nearly full disk	Disable automatic updates in the VM
Disk 97 % full (21 GB)	Image created with a small disk	Expand the virtual disk to 40 GB, then grow the partition and file system (`growpart`, `resize2fs`); 18 GB free afterwards
Simulation very slow (4-6 % of real time with the graphical interface)	Two CPU cores shared by physics, rendering, MoveIt and the desktop	Run without MoveIt; later run headless and reduce the camera rate

Problem	Cause	Solution
VM clock four hours ahead	Image set to a foreign time zone	Set the time zone to Asia/Dubai; measure durations with a monotonic clock

Table 6.1: Environment problems and their solutions

Figure 6.1 shows the rendering problem solved: Gazebo's sample world with the OGRE 1 renderer. Figure 6.2 shows an early physics test, in which a sphere dropped into the world falls under gravity and comes to rest on the ground.

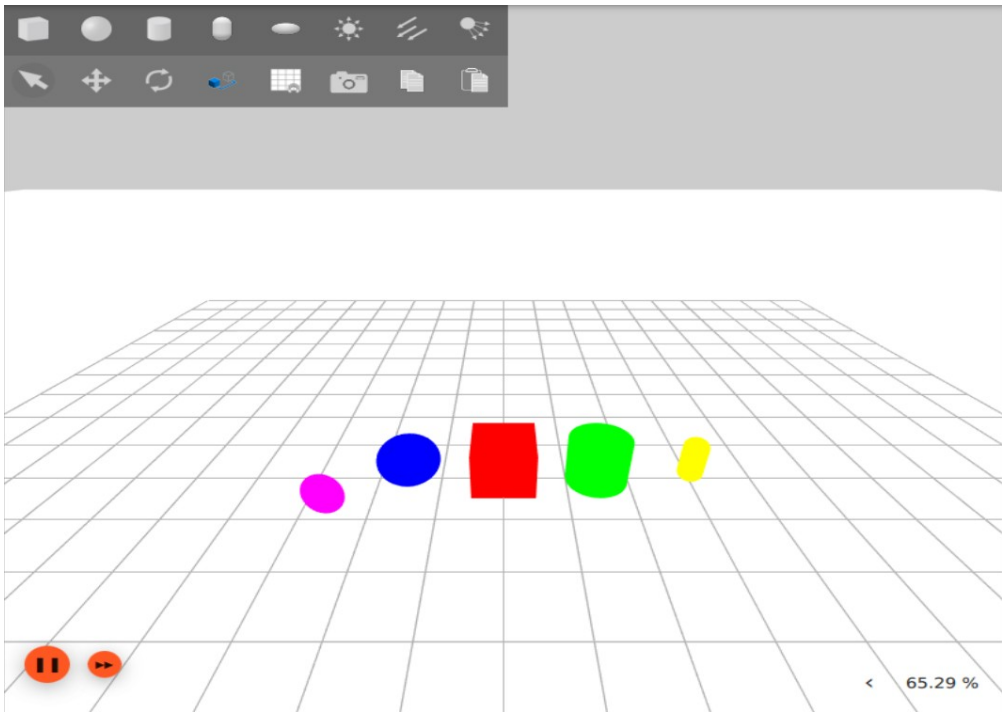


Figure 6.1: Gazebo's sample world rendered with the OGRE 1 renderer after the blank-view fix

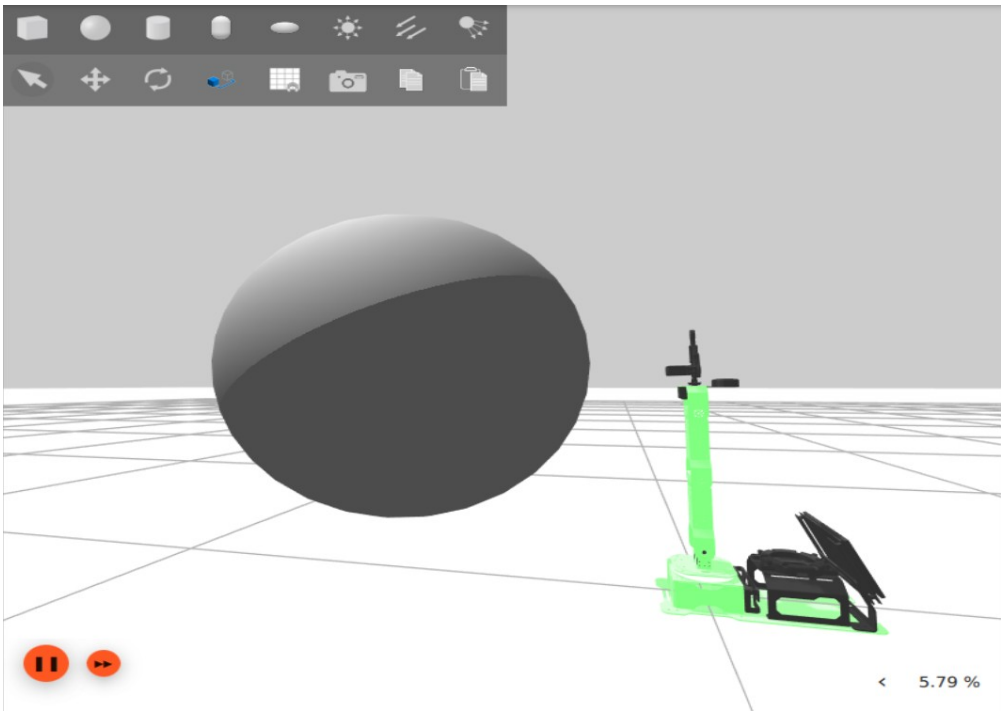


Figure 6.2: Physics test: a sphere falls under gravity and comes to rest beside the arm

6.2 Arm on a Fixed Base

The first stage used the manufacturer's model of the arm fixed to the ground (Figure 6.3), launched with its arm and gripper controllers. Three facts discovered at this stage shaped the rest of the implementation:

- The controllers' state topics did not publish in this configuration, so all joint positions are read from `/joint_states`.
- With the simulation running at about 5 % of real time, any wait measured in wall-clock seconds ends long before the motion does. Every wait therefore checks the measured joint positions and simulated time stamps instead.
- The kinematics in the inverse-kinematics solver matched the simulator exactly: for a commanded tool point at (0.180, 0, 0.045) m, the transform reported by ROS (`tf2_echo base_link end_effector_link`) gave (0.180, 0.000, 0.045) m.

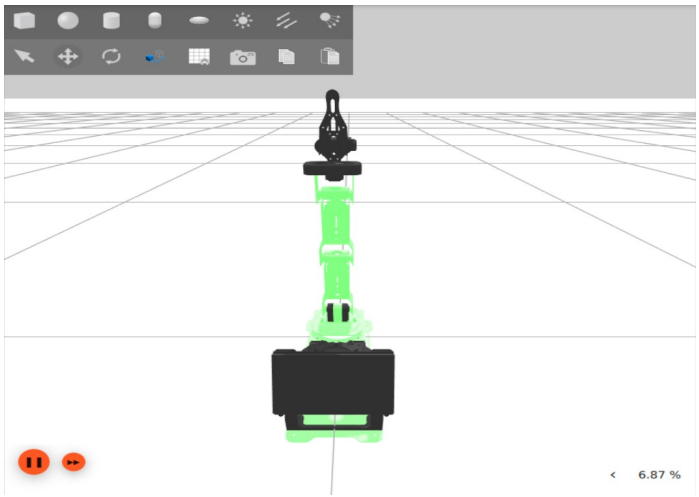


Figure 6.3: The JetArm Pro model on a fixed base in Gazebo, all joints at zero

6.3 Medication Container Model

The container was written as an SDF model with a cylinder collision of 30 mm diameter and 60 mm height, a 30 g mass with the matching inertia, a friction coefficient of 1.5, no restitution and a stiff contact (contact stiffness 10^6 N/m) with a 1 mm minimum depth. A white body and a blue cap are drawn as visuals. It is spawned at the medicine station with a ROS 2 command or by the mission controller. Figure 6.4 shows the container standing beside the arm after spawning.

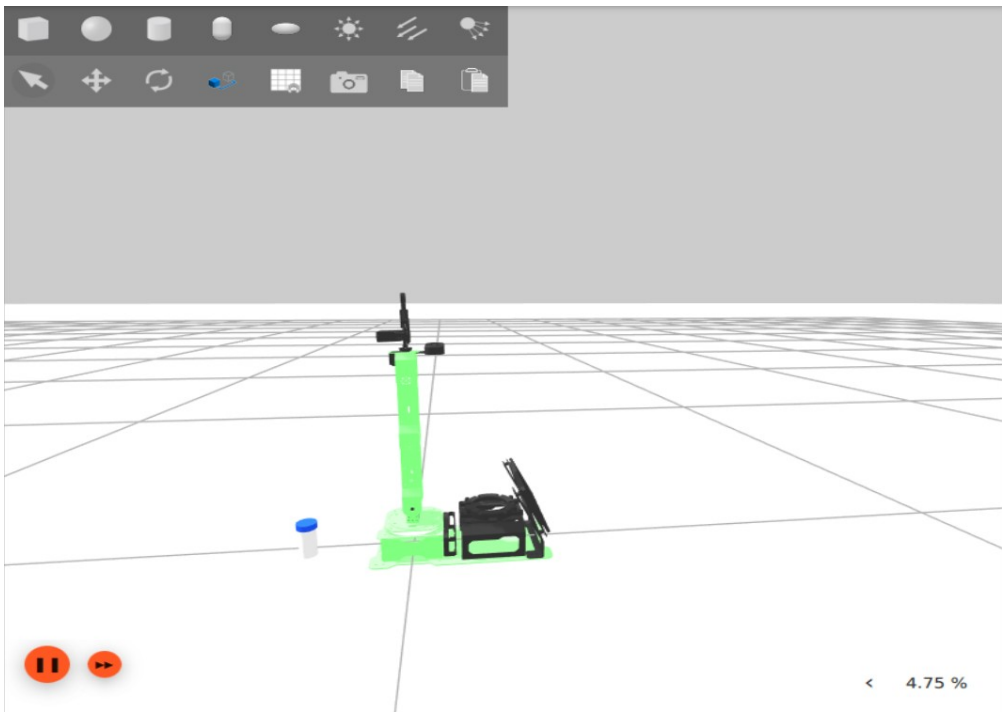


Figure 6.4: The medication container spawned beside the arm

6.4 Grasp Development

The grasp was developed in four iterations, each tested with the arm on the fixed base:

1. **First attempt (squeeze grasp at 0.045 m).** The gripper closed on the container, but the fingers held it near the cap and the container hung tilted below them (Figure 6.5). The simulation slowed to 0.28 % of real time while the container was squeezed.
2. **Measurement.** With no container present, the arm was moved to the grasp pose and the tool-point position was read from ROS, and the gripper was photographed from the side (Figure 6.6). This showed that the claw tips curl well below the tool point, so the flat pads are higher than the tips.
3. **Second attempt (grasp at 0.055 m, close to 0.58 rad).** The fingers now gripped only the top rim of the container (Figure 6.7), confirming that the pads were too high.
4. **Final grasp height (0.038 m).** Lowering the tool point to 38 mm above the surface placed the pads around the container's middle, with the claw tips about 6 mm above the surface (Figure 6.8). The gripper settled at 0.59 rad, the value predicted from the meshes for a 30 mm object.

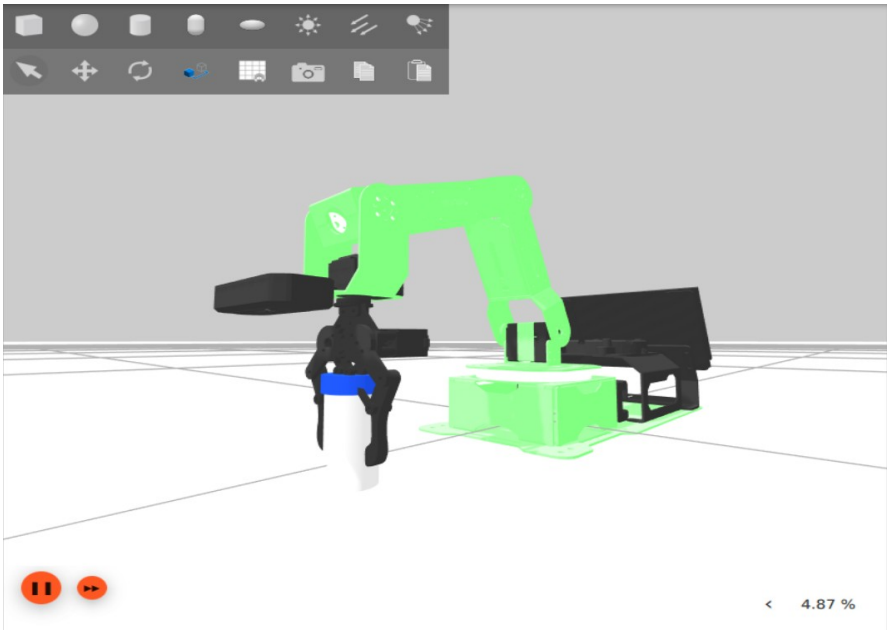


Figure 6.5: First grasp attempt: the container held near the cap and tilted

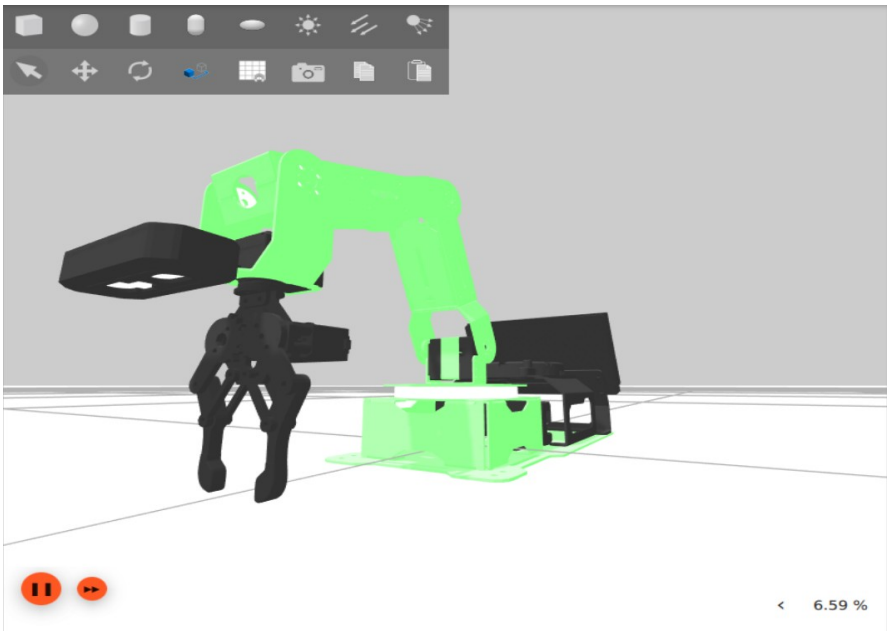


Figure 6.6: Side view of the gripper at the grasp pose without a container

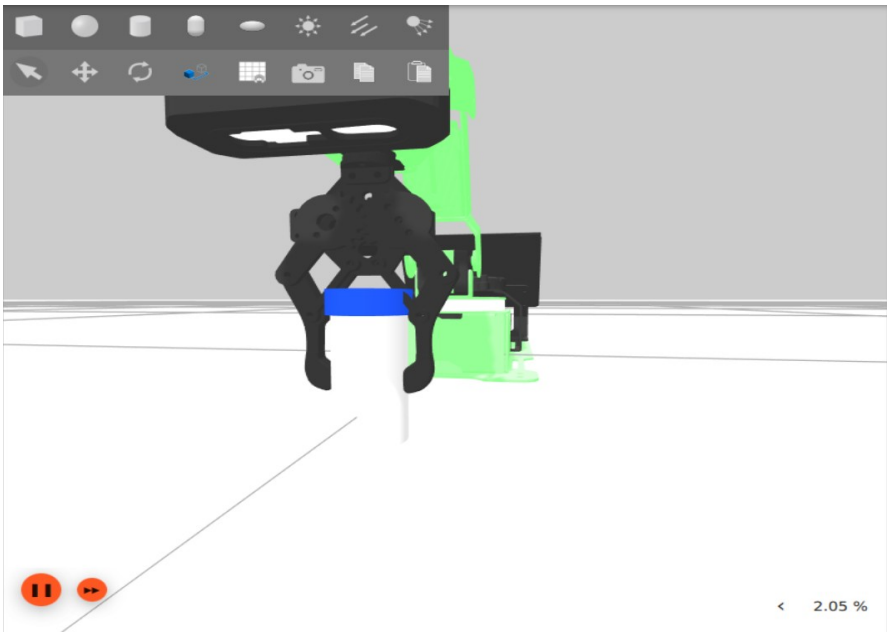


Figure 6.7: Second attempt: the fingers grip only the top rim of the container

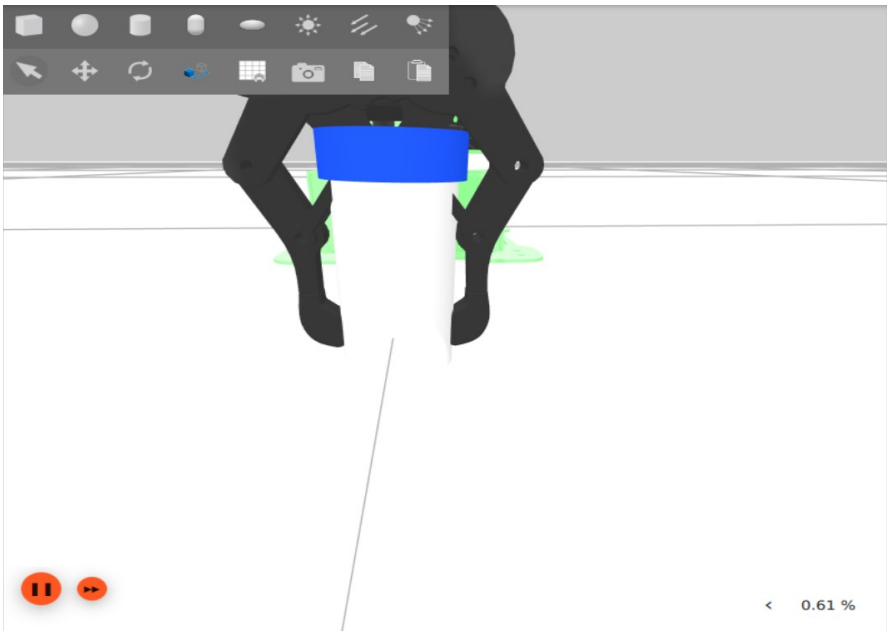


Figure 6.8: Final grasp height: the pads hold the container around its middle

With the correct height, the friction grasp held the container, but the simulation fell to between 0.15 % and 0.6 % of real time, so lifting it by 3 cm took longer than 15 minutes. The assisted grasp of Section 5.5.4 was therefore implemented: the fingers close to 0.75 rad, just short of contact, and a DetachableJoint fixes the container to the last arm link. The DetachableJoint plugin was added to the robot description with the arm's fifth link as parent and the container's body as child, and with dedicated attach, detach and state topics [15].

One property of the plugin caused an early failure: it creates the joint as soon as the child model appears, so a container spawned near the robot was immediately welded to the gripper and floated with it. The mission therefore releases the weld right after spawning the container, and only re-attaches it once the gripper is around it. With these changes the complete fixed-base pick-and-place worked: the arm picked the container, carried it to a second position and released it, and the container dropped onto the ground and stood upright (Figures 6.9 and 6.10).

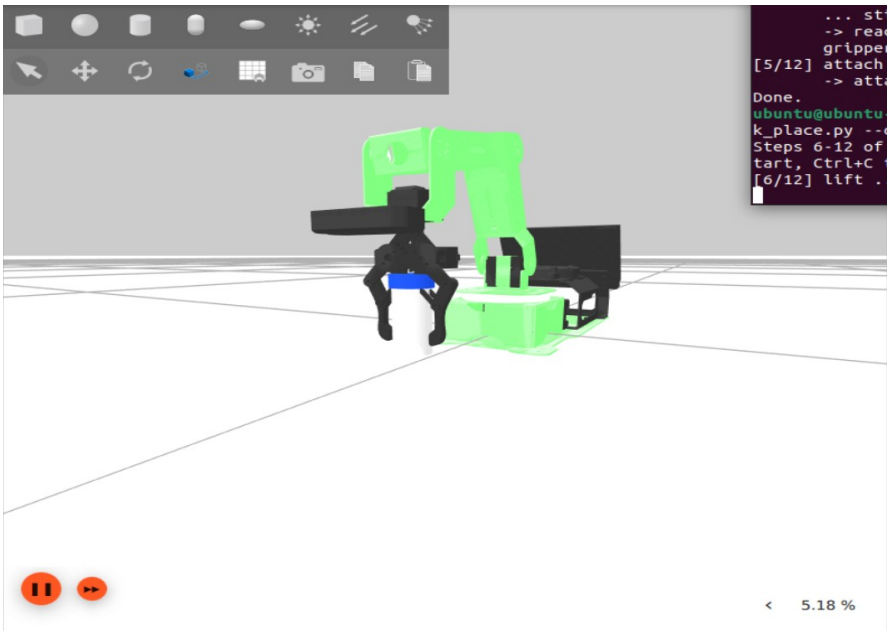


Figure 6.9: Assisted grasp: the container carried in the gripper

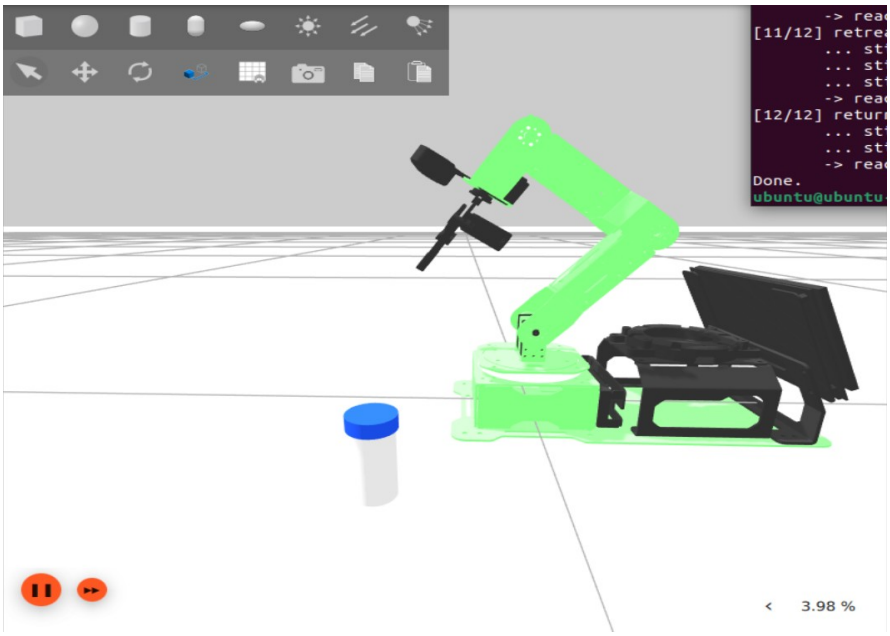


Figure 6.10: Assisted grasp: the container released at the drop-off position, standing upright

6.5 Robot and World Models

6.5.1 Mobile Robot Description

The mobile robot is described in a xacro file that combines the manufacturer's arm description with a model of the mecanum chassis. The arm macro is attached to the chassis at the plate height (0.10 m) instead of to the world. The chassis has a box body and a top plate, four wheels drawn as

cylinders with hubs, and a collision box. Gazebo systems are added for the arm controllers, the joint-state publisher, the DetachableJoint and, in the final version, the three planar base joints. The wrist camera is a separate link fixed to the fourth arm link, 50 mm above the gripper and 30 mm along it, pointing along the gripper, with a 60° field of view, 320 × 200 pixels and an update rate of 4 Hz.

Two errors in the first version of the robot description were found during testing. First, the arm macro silently ignores an origin written as a plain element, so the arm was placed inside the chassis instead of on its plate; it was fixed by passing the origin in the form the macro expects. Second, the frictionless setting intended for the four support spheres under the chassis was not applied, because the spheres had been given names; removing the names fixed it. Both errors were caught by comparing the camera height and the base speed with their expected values.

6.5.2 Base Motion Model: from Velocity Control to Planar Joints

The base was first moved by Gazebo's VelocityControl system, which sets the model's velocity directly. Two problems appeared. With normal floor friction, the base reached only 7 % of the commanded speed (0.006 m/s for 0.08 m/s), because friction removed up to 0.2 m/s in each 0.02 s physics step. Making the floor frictionless fixed the speed, but the model then rose steadily by 1 mm per simulated second, reaching 5 cm by the end of a trip, which would have placed the container 5 cm too high on the bedside table.

The final model replaces VelocityControl with a planar base: a static world link connected to the chassis through three joints: an X joint, a Y joint and a rotation joint. Gazebo's JointController systems drive the three joints at commanded velocities. A small node, `planar_base.py`, converts velocity commands in the robot's frame into world-frame joint velocities using the current heading, and publishes odometry computed from the joint positions. The base now moves exactly as commanded, cannot drift vertically, and needs no contact with the floor, which also makes the simulation lighter. Its odometry agreed with Gazebo's true pose. The change was developed on a separate version-control branch and merged after testing.

6.5.3 Hospital World

The world file is generated by a Python script (`make_hospital_world.py`) from a few parameters: the table height, the path width, the turn radius, the distances along the path and the arm's reach. The script writes the ground, the path as a series of thin flat boxes (straight segments and nine short segments for the arc), the red stop markers, the medicine station, the bedside table and the bed, and it checks that the robot has at least 3.5 cm of clearance from both tables. It also places a fixed overview camera above the ward, which is rendered by the Gazebo server and used

for screenshots and the demonstration video without the graphical interface. Figure 6.11 shows the first version of the generated world in Gazebo before the robot is added, and Figure 6.12 the robot at its start position. The patient area of this world was later replaced by a furnished patient room with the smart bedside table (Section 6.11).

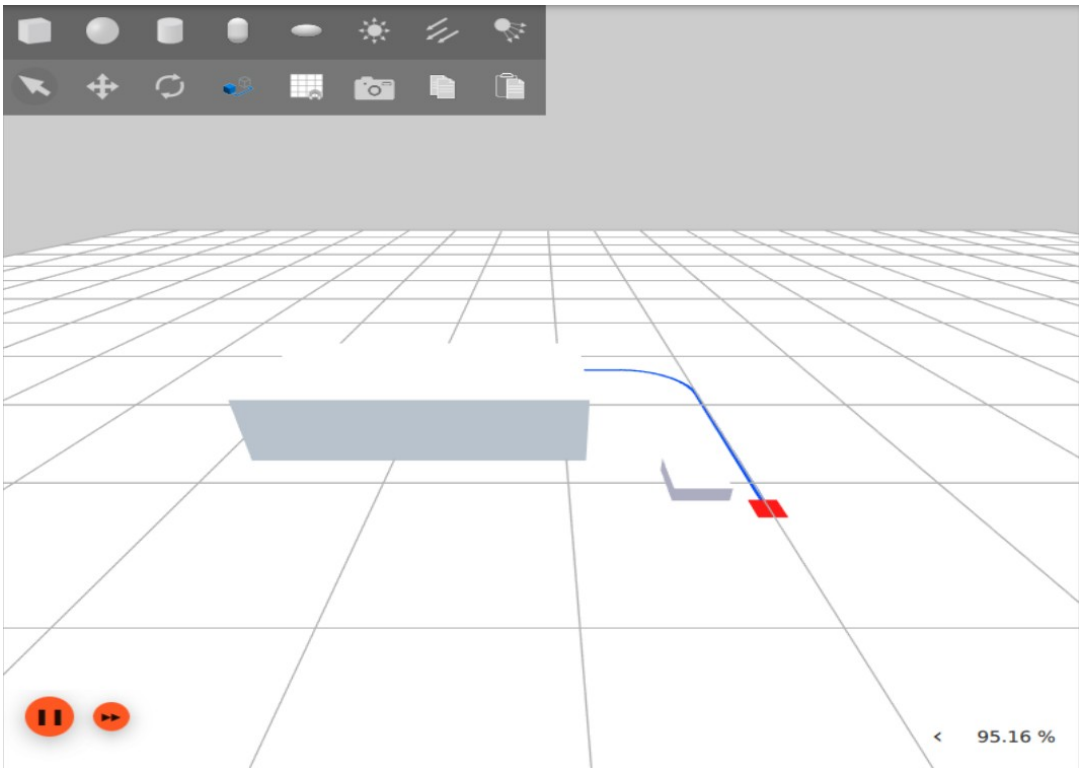


Figure 6.11: The generated hospital world in Gazebo: path, stop markers, medicine station, bedside table and bed

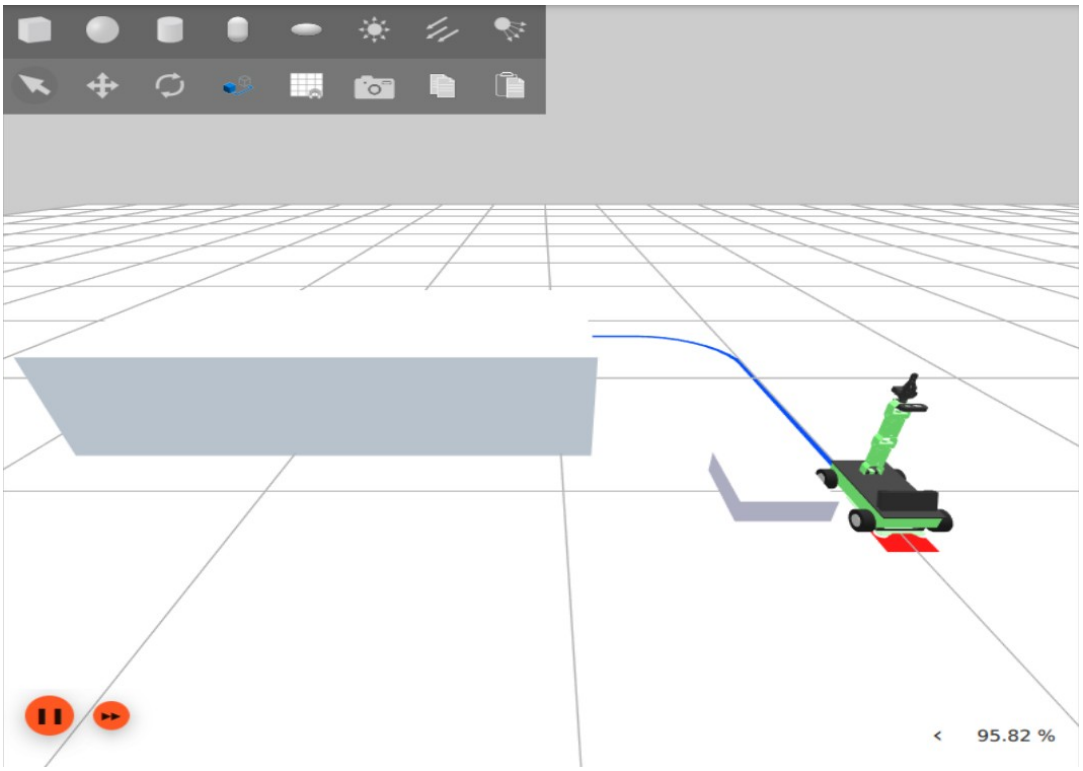


Figure 6.12: The robot at its start position on the home marker, beside the medicine station

6.5.4 Launch File

A single launch file starts the whole simulation: Gazebo with the hospital world and the OGRE 1 renderer, the ROS-Gazebo bridge for the camera, joint states, base and weld topics, the robot description, the planar-base node and the arm and gripper controllers, which are started in sequence once the robot has been spawned. A `gui:=false` option starts Gazebo without its window for test runs, which saves processing time, and a `video:=true` option selects a variant of the world in which the overview camera renders at 2 Hz for recording.

6.6 Arm Control Implementation

The arm controller (`jetarm_pick_place.py`) contains the forward and inverse kinematics of Section 5.5, the waypoint planner for picks and places, and a ROS 2 node that sends trajectories and waits for them to complete. Waypoints are sent as joint trajectories with a time for each point; the node then waits until every joint is within 0.03 rad of its target. Gripper motions are waited for in a different way: the gripper may stop early on the container, so the node also accepts a gripper that has stopped moving (less than 0.003 rad change over 0.5 simulated seconds). An early version measured this settling time on the wall clock, which at a low real-time factor made the gripper appear settled while it was still open; the check was corrected to use simulated time (Section 7.5.3).

The node also offers command-line actions for testing: show the state, move home, open or close the gripper, move to given joint angles, and print the planned waypoints.

6.7 Line Follower Implementation

The line follower (`line_follower.py`) subscribes to the camera images, runs the algorithm of Section 5.6 on each frame and publishes velocity commands. It writes a debug image about once a second showing the three strips, the detected centres and the estimated line position, which was essential for checking the colour thresholds in the simulator. In Gazebo, the rendered path colour was measured as hue 111, saturation 250 and value 255, well inside the detection range, while the floor (saturation 0) was rejected.

Before the line follower was run in Gazebo, it was tested in closed loop against a synthetic camera: an image generator that projects the floor of the hospital world into the camera using the same camera geometry, combined with a simple kinematic model of the robot. This test found and fixed a missing section of the path behind the start, needed for the return trip, before any simulator time was spent (Section 7.2).

6.8 Mission Controller Implementation

The mission controller (`mission.py`) runs the eleven steps of Section 5.8 and can start from any step for testing. It reuses the arm controller for picks and places, runs the line follower as a separate process and waits for its result, and uses `base_moves.py` for the closed-loop sideways moves and rotations. Container poses are read from Gazebo's pose topic, which gives the simulator's ground truth. At the end of each mission it writes a JSON result file with the outcome, the final poses, the placement error and tilt, the attempt counts and the durations.

Two implementation errors were found and fixed during the first full missions. The first: the base pose was read from the last odometry message received, but no messages were processed while the line follower ran as a separate process, so after the trip the mission still believed the robot was at the start. A turn-round computed from that stale pose would have driven the robot across the room. The pose function now waits for two fresh odometry messages, and every base move aborts if the robot moves more than 5 cm away from its goal. The second: attach and release were first sent with a one-shot command-line tool, and some of these commands were lost. They are now sent through `medicine_weld.py`, which keeps one publisher open, waits for the weld state and resends up to five times (Section 7.5.3).

6.9 Scheduler Implementation

The scheduler (`scheduler.py`) reads the schedule file, computes the next due time of every entry, and sleeps until the earliest one falls due. It then runs the mission with a label for the delivery, reads the mission's result file, scores it with the same scoring module as the evaluation, and appends a row to the delivery log. If the log was written with an older set of columns, it is rewritten once with the current columns and the old rows are kept. Deliveries that fall due at the same time run one after the other. Options allow listing the next due times, running one entry a given number of minutes from now, and running only once, which is how it was tested.

6.10 Evaluation and Documentation Tools

6.10.1 Automated Evaluation

The evaluation tool (`evaluate.py`) runs a given number of complete missions, each from a fresh simulation, following the pipeline in Figure 6.13. Before each run it checks that no simulation is running; it then starts the simulation without the graphical interface, waits until all three controllers report that they are active, runs the mission, scores the result from the simulator's poses, shuts the simulation down and checks that no simulator process remains. After each run it rewrites the results table and the summary, so that results survive an interruption. Options allow adding runs to an existing results folder, stopping cleanly after the current run, and marking runs whose wall-clock time is invalid, for example because the computer slept (detected when the wall clock and the monotonic clock disagree by more than 60 s). A separate tool, `rescore.py`, recomputes all scores from the stored raw poses after a change to the scoring rules.

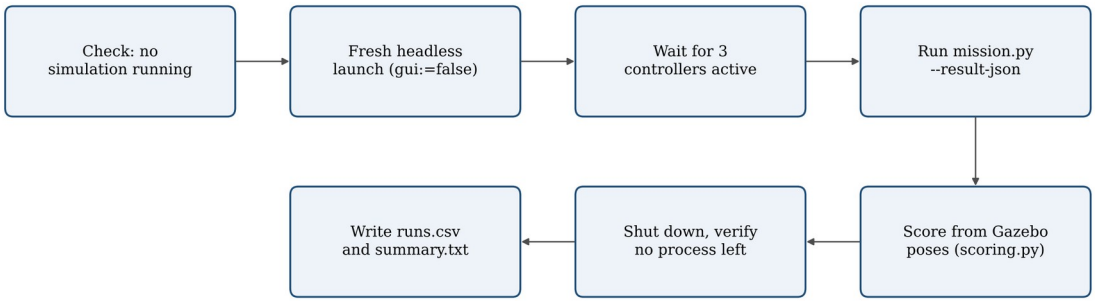


Figure 6.13: Automated evaluation pipeline for one run; the pipeline repeats for each run

6.10.2 Screenshots and Video

Because the graphical interface slows the simulation and can be closed accidentally during a long run, screenshots and video are taken from the overview camera rendered by the Gazebo server. A screenshot tool saves the first frame rendered after it was called, using simulated time to ensure the

picture is current, and a recorder writes the overview frames to an H.264 video with the simulated time and the current mission step printed on each frame.

6.10.3 Version Control and Backup

All TrackMed files are kept in a Git repository, with a commit for every working milestone, and backed up to GitHub. For the final evaluation the repository was made public, with a README, documentation of the architecture, tests, use and troubleshooting, an automated run of the offline tests on every change, and a tagged release (Appendix A). Risky changes, such as the confirmed attach and release and the patient room with the smart table, were developed on separate branches in a second working copy while an evaluation continued on the stable version, and merged only after testing. A project notes file in the repository records the design decisions, known issues and next steps, so that work can be resumed after an interruption.

6.11 Patient Room and Smart Bedside Table

The patient room and the smart bedside table were added as the seventh development stage, on a separate branch, after the 5-run evaluation of Section 7.5 had shown that the delivery itself was reliable. The work kept one constraint throughout: the robot's line following, pick and place had to stay exactly as evaluated, so the room and the table were designed around the robot and not the other way round.

6.11.1 Generating the Room

The furniture is generated by the same world script as the rest of the ward (`make_hospital_world.py`), from named constants for every dimension, position and colour, so that the room can be changed and regenerated consistently. Each item is a separate static model built from boxes, cylinders and spheres: the bed alone has a deck frame, a wheeled base with four casters and two lift columns, the mattress, the raised back section with its pillow, the blanket and turned-down sheet, the headboard and footboard with a patient chart, and the split side rails, each rail made of a top tube and two posts. The patient monitor's screen is a texture generated by the script as a 640 × 480 image with the ECG, pleth and respiration traces and the vital-sign numbers. With the room the world has 111 visual elements, but only one collision shape was added besides the table: a single box covering the bed's deck and mattress. The walls and all other furniture are visual only, because nothing in the simulation can touch them.

Before writing the world file the script checks every material colour against the line follower's blue range and the stop detector's red range, and every footprint against the path and the turning circles (Section 5.7.2). The check caught one existing problem: the medicine station had a pale lavender

colour with a hue of 120, inside the blue range. It had never been detected, because its saturation of 0.11 is far below the line follower's minimum of 0.47, but it was changed to neutral grey so that the rule holds without exceptions.

The first rendering of the room looked flat and washed out: the scene's ambient light of 0.8, which the line follower's colour thresholds had been tuned under, made white surfaces lose all shading and turned the blanket pale cyan. Instead of changing the scene light, which would have required re-checking the line follower, the furniture materials were given an ambient colour of 0.6 times their diffuse colour, and the blanket and the room floor were darkened. Figures 6.14 to 6.16 show the result.

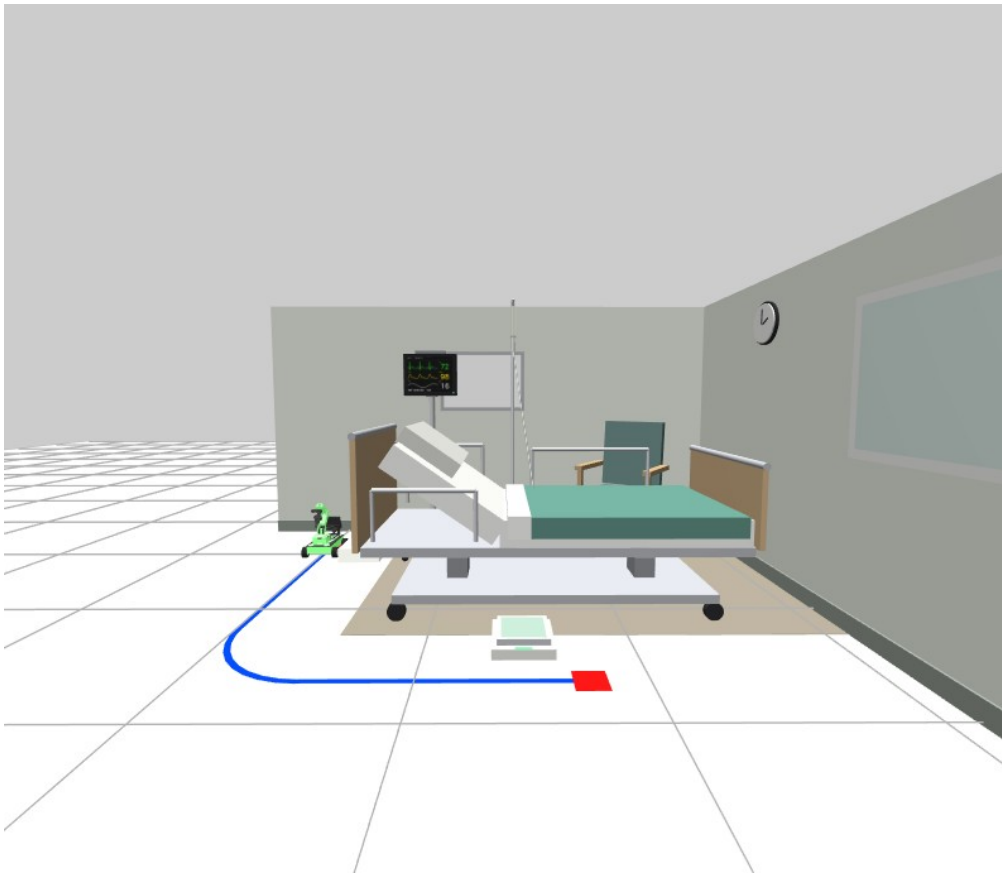


Figure 6.14: The furnished patient room loaded in the Gazebo graphical interface

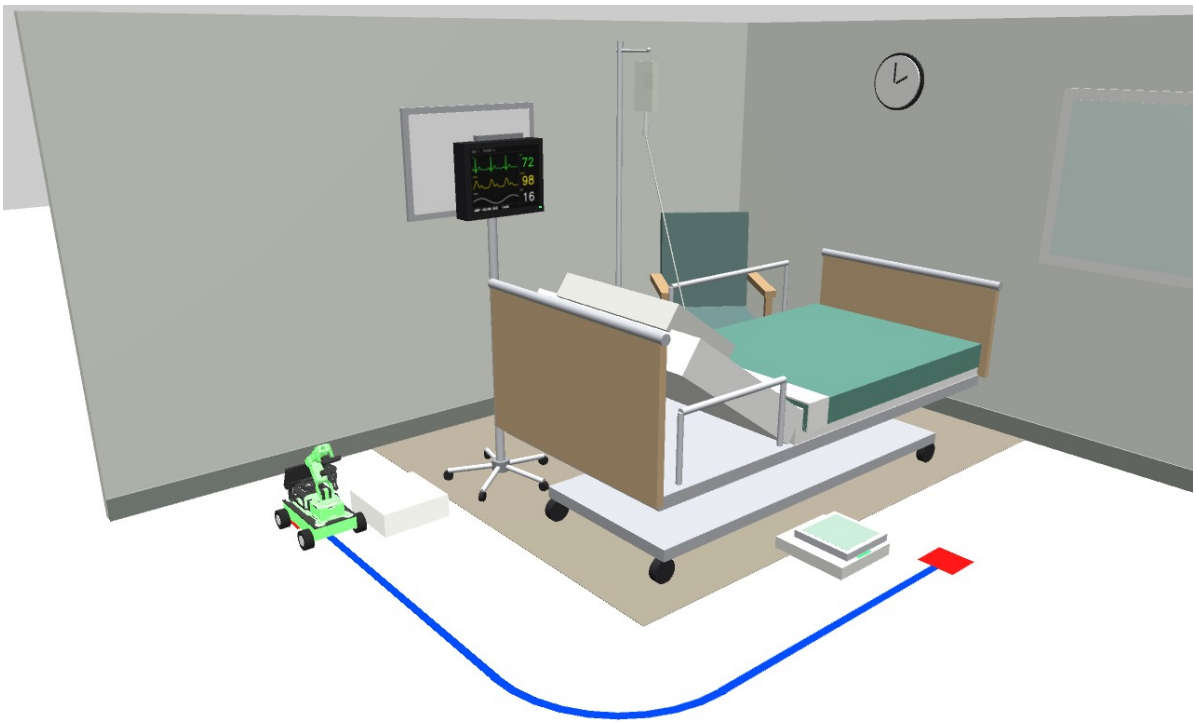


Figure 6.15: The patient room from the overview camera, with the robot at its start position and the smart table lowered beside the bed

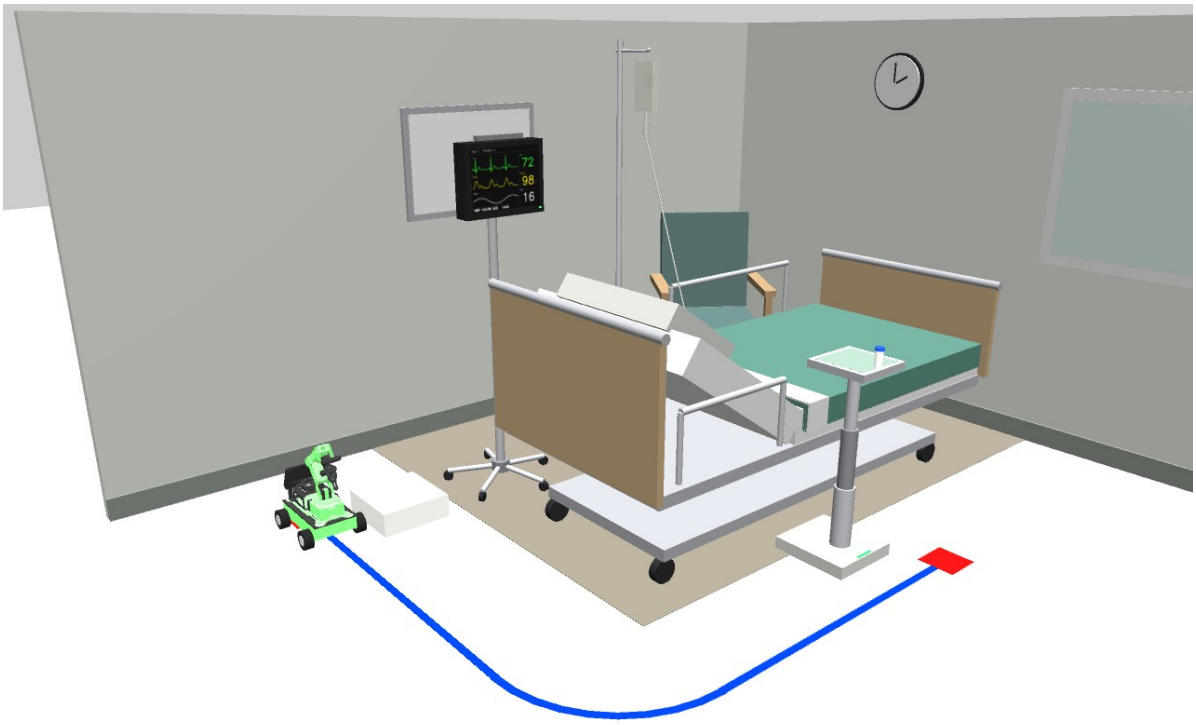


Figure 6.16: The same view with the smart table raised, presenting the container beside the bed

6.11.2 Smart Table Model

The table is a model with a fixed base, three tube links and the platform, connected by three prismatic joints in series. Two problems with the joints were found and solved by building a separate world containing only the table.

The first problem was that a stage which had sagged onto its lower joint limit before the controllers started stayed there, leaving the platform at 0.077 m instead of 0.10 m. The physics engine (DART) does not apply a velocity command to a joint that rests on a position limit. The joint limits were therefore set 3 cm beyond the commanded travel at both ends, so a stage never touches a limit in normal use.

The second problem appeared with the four stages first planned: the fourth joint in the chain never moved, and its stage simply fell onto its limit. The same failure occurred whatever the joint names, their order in the file, the inertias, the collision shapes or the sensor, while three joints in series, or four joints each attached to the base, worked. The design was therefore changed to three stages of 0.233 m, which the specification allowed (three or four stages) and which still hides the column when lowered.

A contact sensor is attached to the platform's top collision shape, and the world's contact system publishes the contacts, which a bridge passes to ROS 2. The container appears in these messages as its own collision shape against the platform, so the controller can tell the container from any other object.

6.11.3 Table Controller Node

The table controller (`smart_table.py`) is started by the launch file together with the rest of the simulation. It subscribes to the contacts, the table's joint states, the weld state, the arm's joint states and the odometry, and it runs the state machine of Section 5.7.3 on simulated time taken from the message time stamps, like every other part of the system. While the platform moves, the controller computes the profile position at each new joint-state message and sends a third of the height above 0.10 m to each stage; while the platform holds still, it repeats the set-point every 0.5 simulated seconds. At start-up the state is taken from the measured height, so a restarted controller never moves the table unexpectedly. The node also offers raise, lower and status commands for testing, and it can be imported by the mission controller, which uses its monitor class to wait for a given table state.

6.11.4 Mission, Scoring and Video Changes

Three changes were made to the mission controller, all outside the robot's own motions. Before placing, it waits until the table reports that it is lowered, for up to 120 simulated seconds. During the place, a separate thread records the container's pose at the moment the contact state turns steady, which is the pose used for the placement error. After the arm has returned home, a new step 8 waits for the table to report raised, for up to 90 simulated seconds, and then checks that the container stands on the raised platform. The turn-round step that follows uses the platform's actual height in its safety check.

The scoring module scores the placement from the pose at the contact confirmation when it is present, and from the final pose otherwise, so both earlier evaluations score exactly as before. With a table, a run is successful only if the table also rose and the container was found centred on the raised platform, at the platform height plus 3 cm within 1 cm, and tilted less than 10°. The evaluation tool, the delivery log and the result files record the new fields (Table 5.3).

The overview camera was moved higher and further back so that it frames the whole route, the room and the table at both heights, and the video recorder prints a second caption line during step 8 with the table state and the platform height. For report pictures a separate world variant adds three high-resolution room cameras, one of which looks across the bed from the patient's side (Figure 6.17).

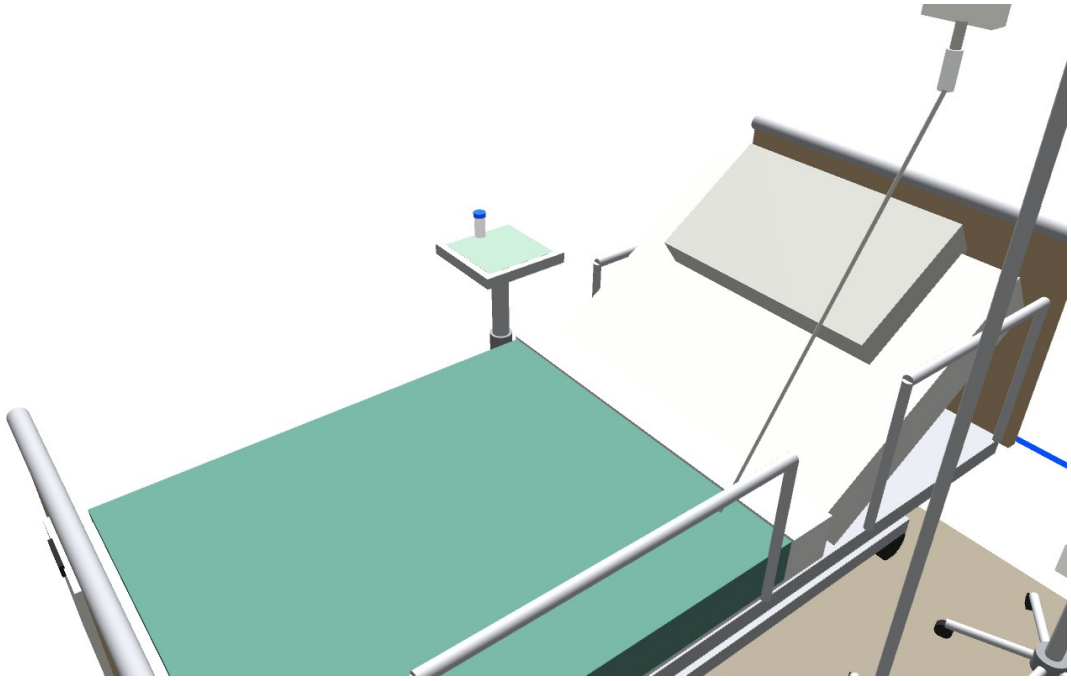


Figure 6.17: The raised smart table seen across the bed from the patient's side, with the container at bed height

6.11.5 Problems Solved

Table 6.2 lists the problems met while building the room and the table and how each was solved.

Problem	Solution
A platform centred on the place target would overlap the robot by 1.4 cm (body) and 5.4 cm (wheels)	Robot-side edge kept at the old table's edge; platform centred on the target in the other direction only
Stages resting on a joint limit ignored their velocity commands (platform stuck at 0.077 m)	Joint limits 3 cm beyond the travel at both ends
The fourth joint of four stages in series never moved	Three stages of 0.233 m
Washed-out colours under the scene light the line follower was tuned for	Furniture ambient colour 0.6 × diffuse; darker blanket and room floor; scene light unchanged
The medicine station had a faint blue hue	Changed to neutral grey; colour check added to the generator
The world file was rejected: a visual and a collision shape had the same name	Collision shape renamed
A short test world was still running when the next simulation started, so the two shared topics and services	Check for any simulator process before every launch, including after time-limited tests
The table test expected no contact while the container was welded 1 mm above the platform, but the container drops onto it before the weld forms	Test changed to "the table stays lowered while the container is welded"; the table logic itself was correct
The patient-side camera saw only the backrest, because the lowered table is hidden behind the mattress	Camera moved higher and further back; picture taken with the table raised
A photo session spawned nothing: a relative file path is resolved by the simulator in its own working directory, yet the spawn tool still reported success	Absolute file paths everywhere
A batch script stopped silently while loading the ROS environment	A shell option that treats unset variables as errors removed

Table 6.2: Problems met while building the patient room and the smart table

6.12 Hospital System Implementation

The hospital system was implemented on its own branch in about two days, following the plan of Section 5.12: first the parts that can be tested without the simulator (map, planner, scheduler, route follower with a synthetic camera), then the world, then each new robot capability in Gazebo, and finally a complete demonstration.

6.12.1 World Generation

The hospital world is generated by a new script (`make_system_world.py`) from the layout numbers in a second module (`system_layout.py`), which also draws the layout picture and checks the clearances. The room function of the single-room generator was reused, with an origin and a rotation added, so that the single-room world is generated exactly as before. The finished world has 459 visual elements, against 111 for the single room. A single smart-table controller node serves all four tables, each under its own name (`/smart_table_1` to `/smart_table_4`); the table's status lights in this world are amber, so that nothing green except the junction markers is near the floor. The map file used by the route planner is written by the same generator, so the planner's graph and the world cannot drift apart. Figures 6.18 and 6.19 show the corridor and a room from the inside.

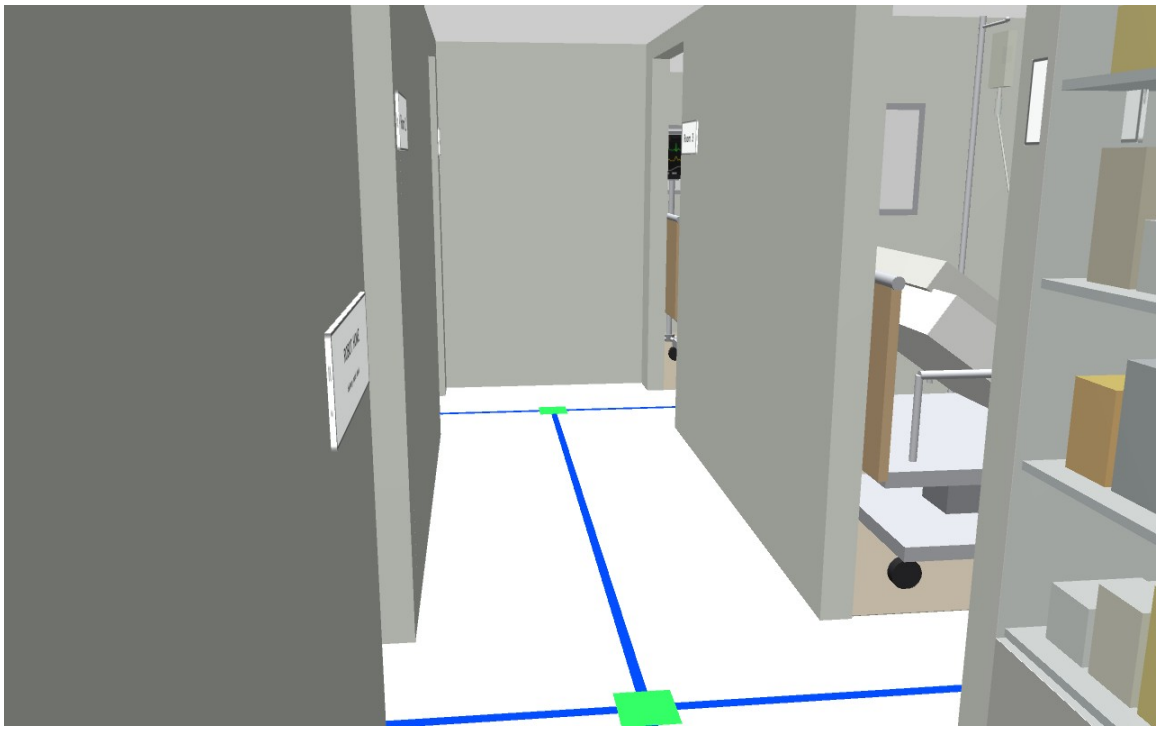


Figure 6.18: The corridor seen from a room door, with the blue line, a green junction marker and a room's branch line

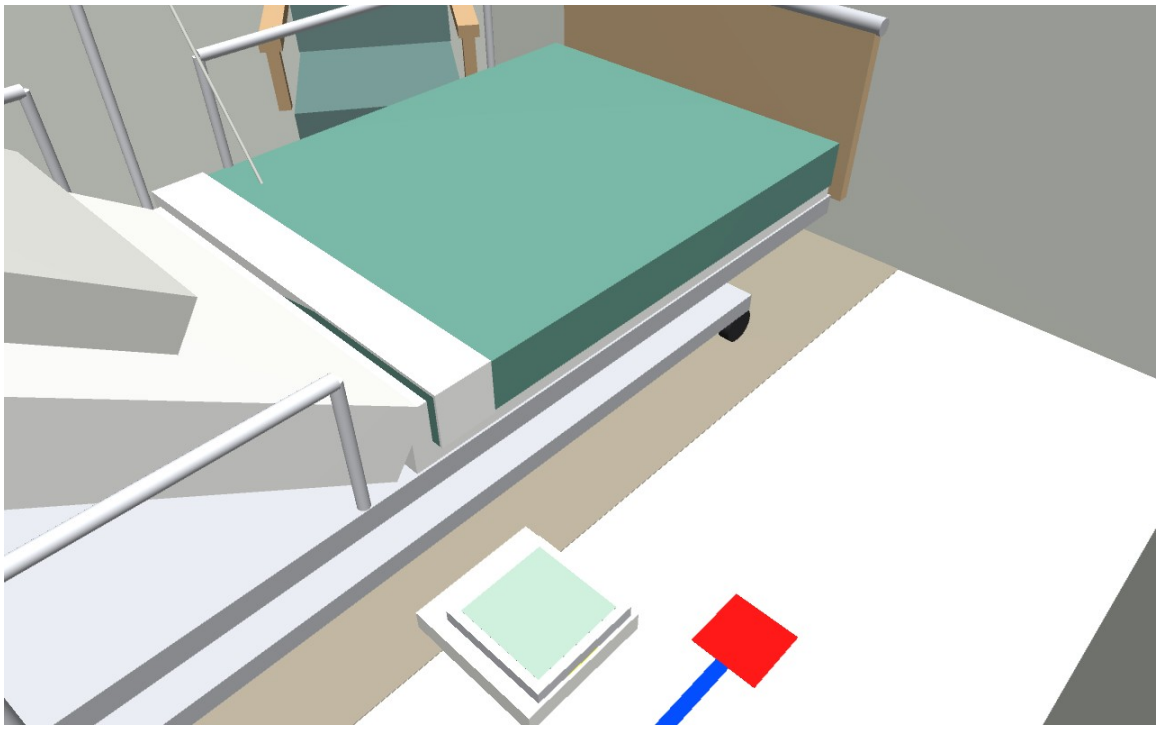


Figure 6.19: Room 1, with the stop marker and the smart bedside table lowered beside the bed

6.12.2 Pharmacy Set-Up

Each container needs its own detachable joint, with its own attach, release and state topics, so that the robot can pick one container without disturbing the others. A detachable joint forms its weld as soon as its container appears in the world, so the containers are spawned only after the robot, one at a time, and each weld is released, and the release confirmed, immediately after its container appears. The containers are never spawned again; a used slot simply stays empty. After set-up every container stands 0.7 to 1.3 mm from the centre of its slot, all displaced in the same direction along the counter; the four containers delivered in the demonstration were 1.1 to 1.3 mm from their slots when they were picked.

6.12.3 Route Follower and Planner

The route follower (`route_follower.py`) imports the tested line follower and adds the look-ahead band, the marker matching and the junction actions of Section 5.12.5. The only change to the tested code was to move its steering block into a function of its own, and its output was checked offline to be identical. The route planner (`route_planner.py`) loads the map, runs Dijkstra's algorithm and produces the list of events; it can also print the routes of all rooms (Table 5.5). Because the original synthetic-camera test of Section 7.2 was not in the repository, a new offline simulator was written from the measured camera geometry and a kinematic model of the base. It

was first checked against Gazebo on the original single-room route, where it reproduced Gazebo's stop to within 4.4 mm, before it was used to test the new routes.

6.12.4 Cap Colours

The cap colours were first chosen as white, yellow, amber, grey and black. When the wrist camera's images of the caps were analysed during the first pick test, amber and yellow both came out with a hue of 30: under the scene's ambient light both materials saturate, so the camera saw them as the same colour. The amber cap was replaced by purple, which renders at a hue of 150, 20 steps from both the line's blue and the markers' red and far from yellow and the junction green. A test that reads all six caps from the robot's real pick positions then classified every cap correctly (Figure 6.20), and only then was the cap check switched on for the demonstration.

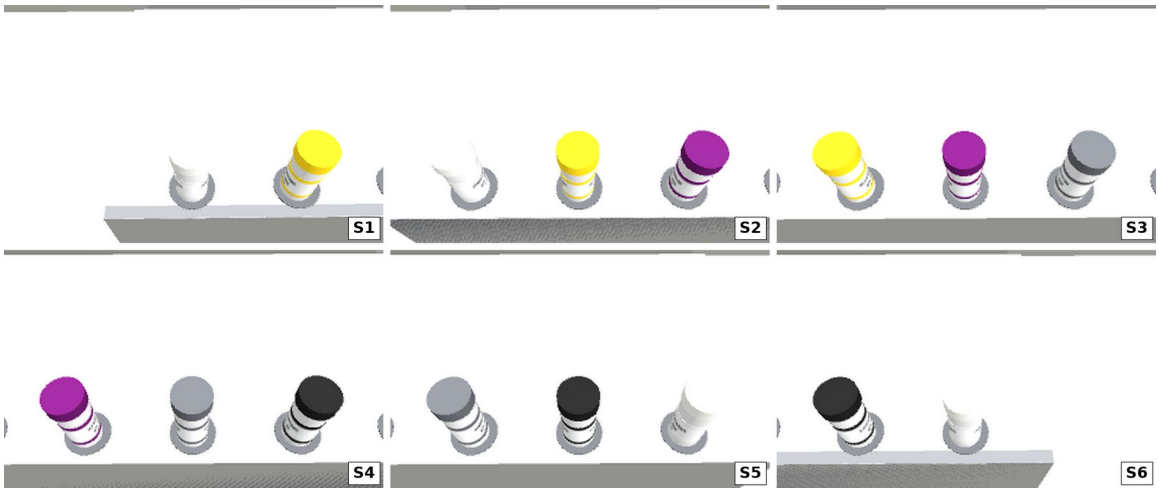


Figure 6.20: The six caps as seen by the wrist camera during the cap test, all classified correctly

6.12.5 Fine Line-Up at the Slot

The first complete delivery placed the container 2.8 mm from its target, against 0.2 mm in the single room. The cause was found from the saved result file without running another simulation: the base's coarse move command ends once it is within about 2 mm of its goal, and here the robot came to rest 1.9 mm short of the slot (at x = 0.2242 m against a slot at 0.2223 m). It therefore gripped the container 3.2 mm off the centre of the gripper, the weld kept that offset, and the container was released 2.7 mm short of the target. In the single room the robot had started exactly at the pick position, so the offset never arose. A fine line-up was added: after the normal move, a slow move at 0.01 m/s to within 0.5 mm, which keeps the normal result if it does not finish within 20 simulated seconds. In the cap test the robot then stopped within 0.5 mm of every slot.

6.12.6 Robust Arm Wait

The first demonstration ran for four hours and stopped in its third delivery, at the step "arm home (carrying)". The arm was in fact exactly at home: both publishers of joint states showed zero error. But inside the delivery process the joint-state messages had stopped arriving at the subscription, so its stored value stayed 0.040 rad short of the target for more than 20 minutes while the process kept running. The wait then timed out after one hour of wall-clock time and failed the delivery, and the scheduler stopped as designed. The cause in the ROS 2 client library could not be found without attaching a debugger to the running process, so the wait was made robust instead, in three ways:

- **Simulated time.** Every arm and gripper wait now times out on simulated time (the trajectory's duration plus 30 s), with wall-clock time only as a three-hour backstop for a simulation that has stopped altogether.
- **Frozen-cache recovery.** If no joint-state message has arrived for 15 s of wall-clock time, the wait reads one fresh message through a separate node. If that message is newer than the stored one, the stored state was frozen: the wait loads the fresh message, creates the subscription again and logs the event.
- **Settled arm home.** A move to the home pose is accepted within 0.03 rad, as before, or within 0.06 rad once the joints have stopped moving. Moves that need precision, such as the descent to a pick, still require 0.03 rad.

An offline test with a simulated joint-state publisher covers seven cases, including a frozen subscription, and a short Gazebo check froze the subscription on purpose while the robot carried a container (Section 7.7.2).

6.12.7 Demonstration Runner

The complete demonstration takes between five and six hours of wall-clock time, longer than the network connection or the development session could be relied on. It was therefore written as one self-contained script (`run_demo.sh`) that runs detached from the session: it launches the video world without a window, starts the recorder, runs the scheduler in demonstration mode with the cap check, watches that the simulator is still alive, stops after at most 14 hours, and on any ending shuts everything down and writes a summary. The recorder adds captions with the hospital clock, the room, the patient, the medicine and the current step, and an inset of the wrist camera (Figure 6.21).

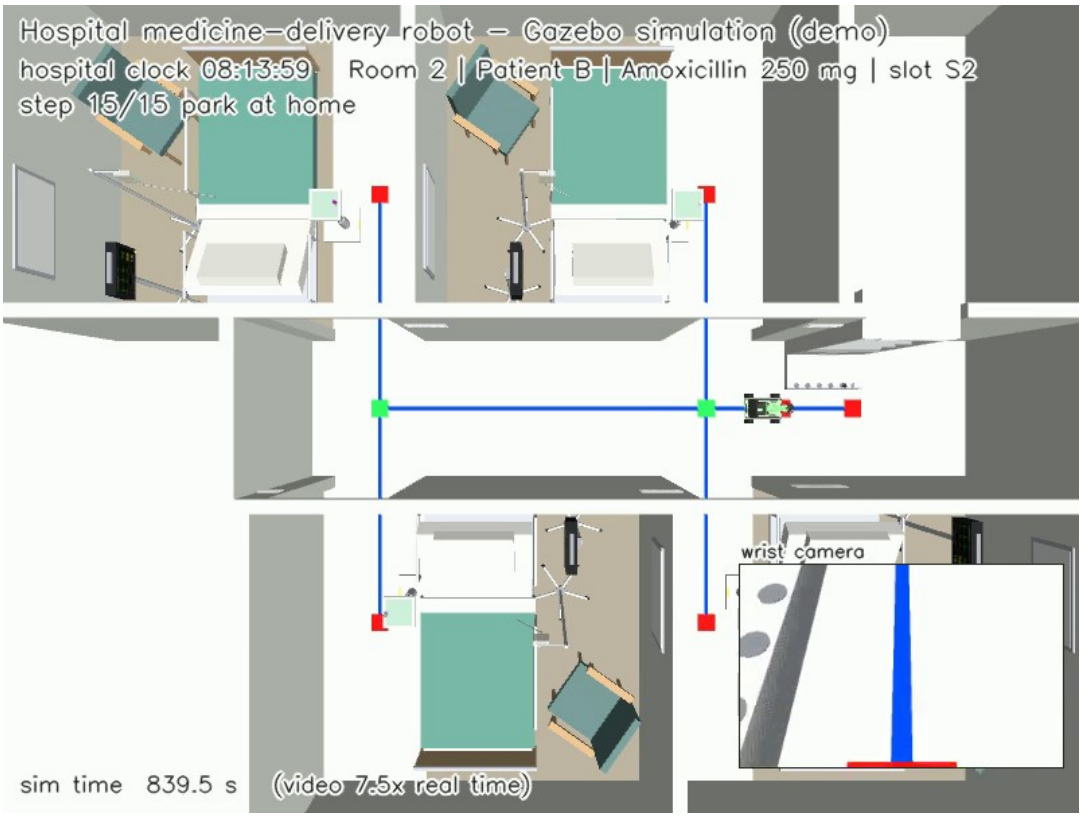


Figure 6.21: Frame of the demonstration video at its end: the robot parked at home and all four tables raised, with the caption and the wrist-camera inset

6.12.8 Problems Solved

Table 6.3 lists the problems met while building the hospital system and how each was solved.

Problem	Solution
Amber and yellow caps rendered with the same hue (30)	Amber cap replaced by purple (hue 150); cap test 9 of 9
The base's move stops within 2 mm, so the container was gripped off-centre (2.8 mm placement)	Fine line-up to 0.5 mm; robot within 0.5 mm of every slot afterwards
Joint-state messages stopped reaching the delivery process; the arm wait timed out on wall-clock time (demonstration attempt 1)	Waits timed on simulated time; frozen-subscription detection and recovery; settled arm home within 0.06 rad
The stop file ended the demonstration at once, not after the current delivery	Stop file checked by the scheduler between deliveries
The check for a running simulator matched the command line of the checking shell itself	Check matches the simulator programs only and excludes its own process tree
The world file was rejected: a visual and a collision shape of the counter had the same name	Collision shape renamed

Problem	Solution
A long test could run past the end of the day's network connection	Deadline option for the test runner, which stops the test cleanly and writes its result
Attempt 2 overwrote attempt 1's per-delivery result files, which are named by hospital-clock time	Attempt 1's figures kept in its summary, log and delivery log; result files are now named by the start of the run

Table 6.3: Problems met while building the hospital system

7 TESTING AND RESULTS

This chapter presents the testing of TrackMed and its results. Testing followed the incremental approach of Section 5.2: offline tests of the algorithms, then tests of each subsystem in Gazebo, a scheduled end-to-end delivery, two automated evaluations of five complete missions each, tests of the patient room and the smart bedside table, and finally tests of the hospital system with four rooms, a medicine storage unit and a smart scheduler. The chapter ends by verifying every requirement of Chapter 4 and discussing the limitations of the results.

7.1 Test Strategy

Table 7.1 summarises the test levels, what each one checks and the environment in which it runs. Each level was passed before the next was attempted, so that a failure at a higher level could be traced to the integration rather than to a component.

Level	What is tested	Environment
Offline	Kinematics against the URDF, reach, gripper opening, camera view, line follower in closed loop	Python models built from the same geometry
Subsystem	Base speed and heading, arm accuracy, camera images and colours, line following each way, turn-round, pick, place	Gazebo, one subsystem at a time
End-to-end	A complete delivery started by the scheduler at its due time	Gazebo, full system
Evaluation	Five complete missions, each from a fresh simulation, scored automatically	Gazebo, headless, automated
Patient room	Smart table on its own, then complete scheduled missions; effect on simulation speed	Gazebo, headless
Hospital system	Planner, scheduler and routes offline; route, pick, cap and arm-wait tests; complete deliveries and a four-dose demonstration	Python and Gazebo, headless

Table 7.1: Test levels

All Gazebo tests used the simulator's ground truth for scoring: the positions of the robot and the container were read directly from Gazebo rather than estimated by the robot, so that measurement errors in the robot cannot hide errors in its behaviour.

7.2 Offline Tests

The results of Sections 7.2 to 7.4 were obtained before the project's result files were kept in the repository. They are recorded in the development notes (`CLAUDE.md` in the repository), written at the time of each test; the raw output of these early tests was not kept. Every later result has its result file in the repository's `results` folder.

7.2.1 Kinematics, Reach and Gripper

The forward kinematics of the solver reproduced the URDF at the ready pose, where both give a tool point at (0.090, 0, 0.179) m, and at the zero pose, where both give 0.507 m above the base. For every point in the reachable band, the inverse-kinematics solution passed back through the forward kinematics within 1 mm, and out-of-reach targets were rejected with an error instead of a wrong solution. The gripper opening computed from the meshes predicted contact with a 30 mm container at about 0.59 rad, which matched the value later measured in Gazebo (0.59-0.60 rad).

7.2.2 Line Follower in Closed Loop

The line follower was tested against a synthetic camera that renders the floor of the hospital world as the wrist camera would see it, combined with a kinematic model of the robot, using the same code as in Gazebo. Three cases were run (Figure 7.1 and Table 7.2): a nominal start, a start offset by 4 cm sideways and 9° in heading, and the return trip from the patient's stop. The first run of the return case failed: the robot lost the line just before home, because the path ended at the start position while the robot needs to see path ahead of it to reach the home marker. The path was extended back to the home marker and all three cases then succeeded.

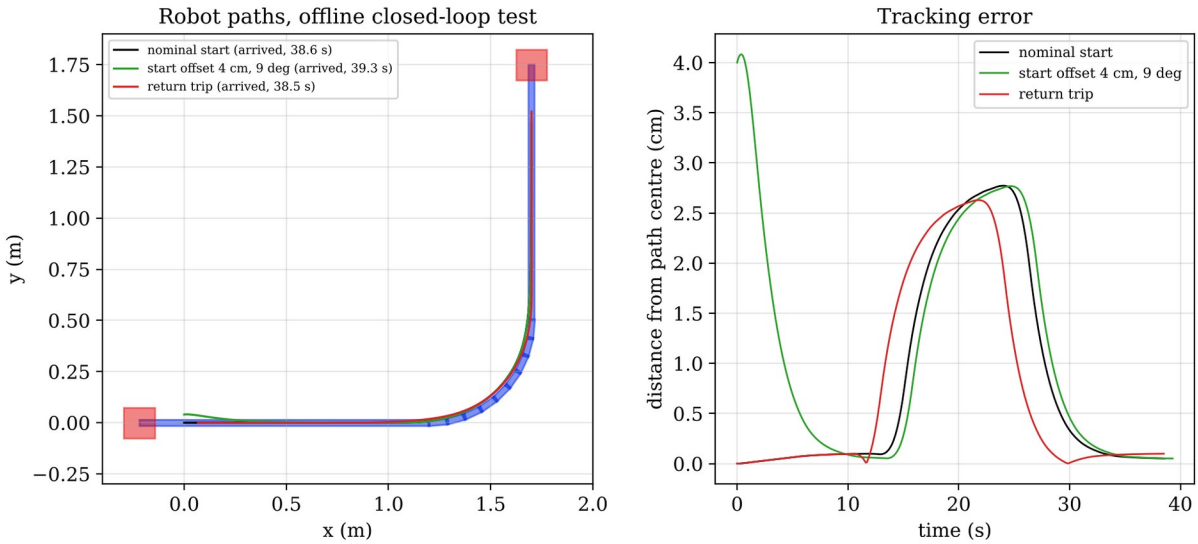


Figure 7.1: Offline closed-loop test: robot paths and distance from the path centre

Case	Result	Time to stop	Maximum distance from path
Nominal start	Arrived at patient marker	38.6 s	2.8 cm
Start offset 4 cm, 9°	Arrived at patient marker	39.3 s	4.1 cm

Case	Result	Time to stop	Maximum distance from path
Return trip	Arrived at home marker	38.5 s	2.6 cm

Table 7.2: Results of the offline line-following test

The largest deviation occurs in the 90° curve, where the camera, which looks about 0.2 m ahead, sees the curve before the robot reaches it. The offset start is corrected within about 10 seconds.

7.2.3 Effect of the Camera Frame Rate

Rendering the camera is the largest cost in the simulation, so the test was repeated at lower frame rates (Figure 7.2). The path deviation was unchanged at 2.8 cm from 10 Hz down to 2.5 Hz, while the robot stopped slightly later at lower rates because the marker is seen less often. A rate of 4 Hz was chosen: in the offline test it adds 2.5 cm of stopping distance, and it reduces the rendering load by 60 %. In Gazebo the stop at 4 Hz was only 0.9 cm further than at 10 Hz (Table 7.3). The extra distance depends on where the frames fall relative to the edge of the marker, which can differ by up to one frame interval of travel (2 cm at 4 Hz), so a single run at each rate cannot reproduce the offline average exactly.

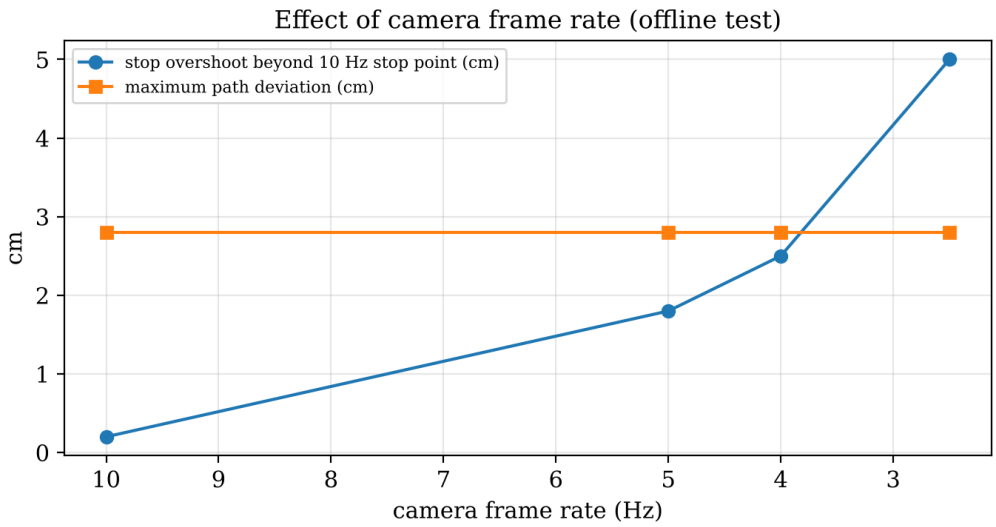


Figure 7.2: Effect of the camera frame rate on the offline line-following test

7.3 Subsystem Tests in Gazebo

7.3.1 Base Motion

The base was commanded to drive at 0.08 m/s and its speed was measured from Gazebo's true pose and from odometry (Figure 7.3). With the first base model (VelocityControl with floor friction) it

reached only 0.006 m/s; the planar base reached 0.078 m/s by Gazebo's true pose and 0.080 m/s by odometry, with the height constant at 0.000 m. Sideways moves and rotations matched their commands in the same way. A 90° rotation timed open-loop stopped at 88.8°, one physics step short, which is why the turn-round is closed on odometry rather than timed.

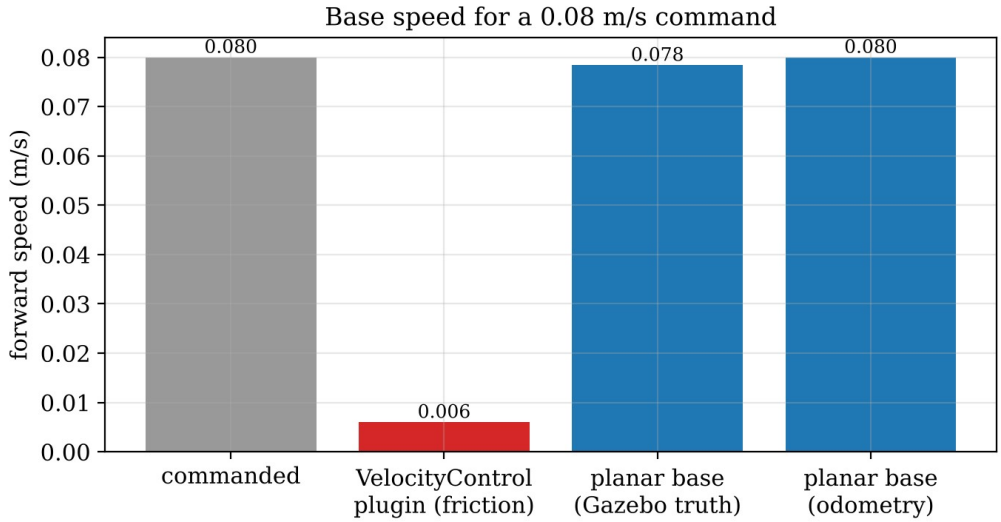


Figure 7.3: Measured base speed for a 0.08 m/s command

7.3.2 Simulation Speed

The real-time factor was measured in each configuration used (Figure 7.4). The simulation runs close to real time without the camera, but rendering the wrist camera and the graphical interface on two processor cores reduces it to 2-6 %. Running headless with the camera at 4 Hz raised it to about 9 % with the robot standing still; later idle measurements of this kind gave 7 to 11 % (Sections 7.6.3 and 7.7.5). During a mission, with the arm and the base moving, the factor is lower, and it varied between runs: 8.6 % in the first scheduled delivery (31.4 minutes of wall-clock time), 5.6 % on average in the second evaluation (47.6 minutes), 5.1 % and 6.0 % in the patient room (58.5 and 49.8 minutes), and 3.9 % to 4.6 % in the hospital demonstration with the video recorder running (67 to 103 minutes). The friction grasp reduced the real-time factor to about 0.3 %, which is the main reason for the assisted grasp.

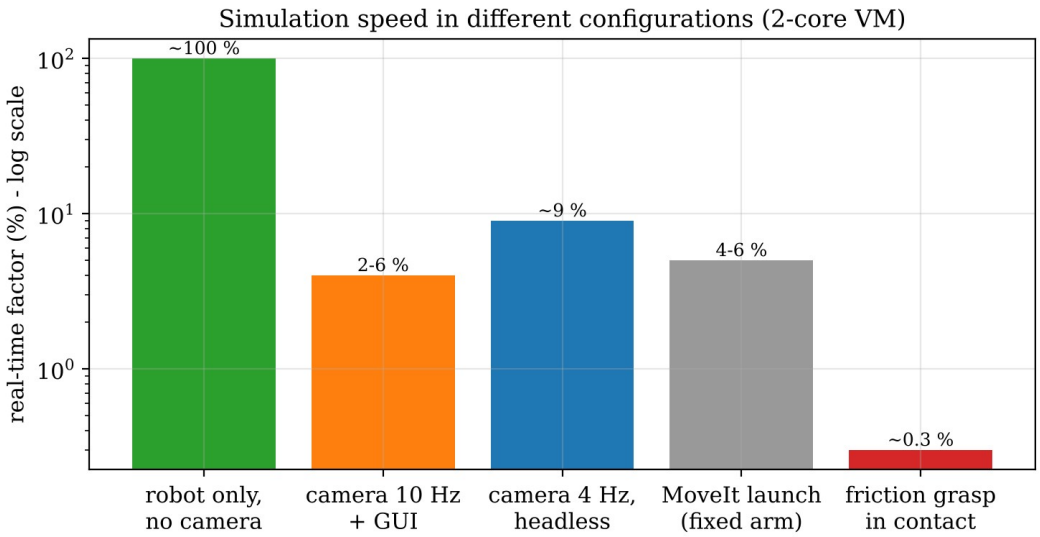


Figure 7.4: Real-time factor of the simulation in different configurations

7.3.3 Arm and Camera

The arm reached its commanded tool point exactly: for a target at (0.180, 0, 0.045) m, ROS reported (0.180, 0.000, 0.045) m. The camera delivered images at 4.03 Hz of simulated time, in RGB format, with the path rendered at hue 111, saturation 250 and value 255 and the floor at saturation 0. After the near clipping distance was raised to 0.15 m, the gripper no longer appeared in the image.

7.3.4 Line Following

Table 7.3 lists the line-following runs in Gazebo. The planned stop point of each run is the stop predicted by the offline test of Section 7.2.2; FR-3 requires every stop within 5 cm of it. Figure 7.5 shows the robot following the path, and Figure 7.6 the robot stopped at the patient's marker.

Run	Time	Result
To the patient, camera 10 Hz	38.6 s	Stopped at (1.699, 1.463) m, heading 90.0°, 3 mm from the planned stop point
To the patient, camera 4 Hz	38.7 s	Stopped at (1.699, 1.472) m, 12 mm from the planned stop point (9 mm further on than at 10 Hz, as expected from the lower rate)
Turn-round at the patient	17 s	Rotated 179.7°; back on the line within 0.1 mm sideways; no contact with the table
Home, camera 4 Hz	38.2 s	Stopped at the home marker, 2.6 cm beyond the planned stop point

Table 7.3: Line-following runs in Gazebo

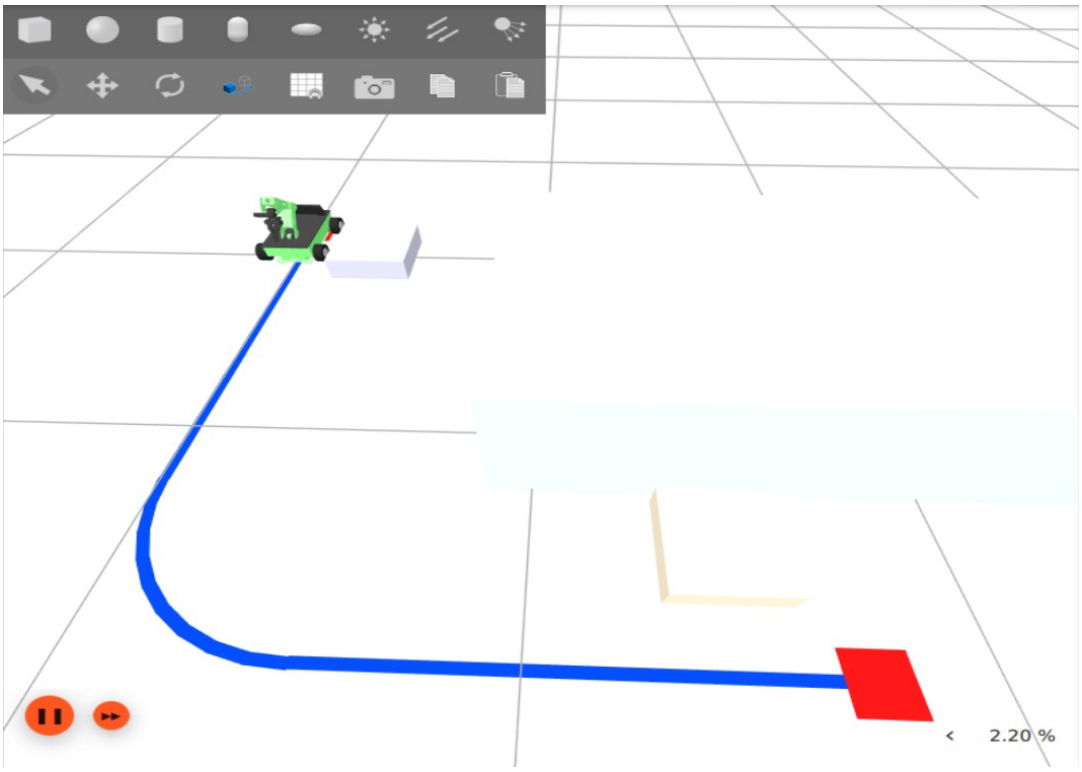


Figure 7.5: The robot following the blue path towards the curve

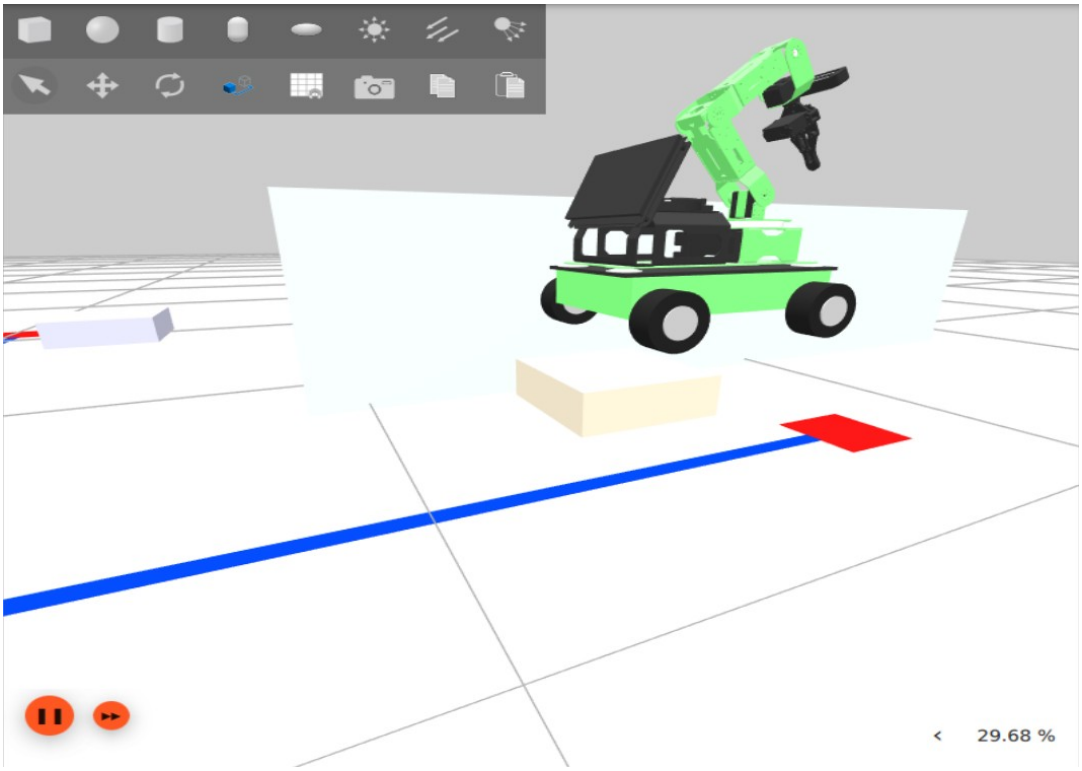


Figure 7.6: The robot stopped at the patient's marker beside the bedside table

7.3.5 Pick and Place on the Mobile Robot

On the mobile robot, the pick at the medicine station closed the gripper to 0.78 rad around the container, and the container rose with the gripper to a height of 0.227 m. In the first complete mission, the container was placed on the bedside table at (1.5001, 1.4741) m, about 1 mm from the target, and stood upright.

7.4 Scheduled End-to-End Delivery

The scheduler was given one delivery, Paracetamol 500 mg for Patient A in bed 1, due one minute after it started, in a freshly launched simulation. It started the mission exactly at the due time, and the mission completed all ten steps (Table 7.4); the mission had ten steps at that time, and step 8, the lift by the smart table, was added later (Section 5.8.1). The robot parked at (0.002, 0.000) m with a heading of −0.3°, ready for the next delivery, and the delivery was written to the log. A 23-second overview video of this run, at 7.5 times real speed with the current step printed on each frame, is included in the project repository (Appendix A).

Measure	**Result**
Start time	At the due second
Steps completed	10 of 10 (the mission before the smart table)
Placement error	0.08 mm
Container tilt	0.002°
Final robot pose	(0.002, 0.000) m, −0.3°
Mission duration	161.5 s simulated, 31.4 min wall-clock
Logged	Yes, in deliveries.csv

Table 7.4: Result of the scheduled delivery

7.5 Full-Mission Evaluation

7.5.1 Method

Two evaluations of five independent runs were carried out with the automated tool of Section 6.10.1. Each run started a fresh simulation, ran the complete mission, which had ten steps before the smart table was added, and scored the result from Gazebo's poses. A run counts as a **success** only if the mission completes, the container stands on the bedside table within 30 mm of the target, and its tilt is less than 10°. The first evaluation used single, unconfirmed attach and release commands. The second used the confirmed commands, the physical checks after each critical step and the corrected gripper wait.

7.5.2 Results

Table 7.5 lists every run and Figure 7.7 summarises the outcomes and the placement errors.

Evaluation	Run	Result	Placement error	Tilt	Sim. time	Observation
1 (unconfirmed)	1	Fail	199.9 mm	180°	162.7 s	Late release: container dropped off the table, upside down on the floor
1	2	Success	0.09 mm	0.0°	160.5 s	
1	3	Fail	2,034.8 mm	34.3°	162.3 s	Release lost: container stayed in the gripper and was carried home
1	4	Fail	1,966.7 mm	0.0°	161.8 s	Attach lost: gripper lifted empty; container never left the station
1	5	Success	0.07 mm	0.0°	161.6 s	
2 (confirmed)	1	Success	0.19 mm	0.0°	160.2 s	All commands confirmed on the first attempt
2	2	Success	0.20 mm	0.0°	160.1 s	All commands confirmed on the first attempt
2	3	Success	0.20 mm	0.0°	160.3 s	All commands confirmed on the first attempt
2	4	Success	0.19 mm	0.0°	160.2 s	All commands confirmed on the first attempt; the computer slept for about an hour during the run, so its wall time is excluded
2	5	Success	0.19 mm	0.0°	160.1 s	All commands confirmed on the first attempt

Table 7.5: Results of the two 5-run evaluations

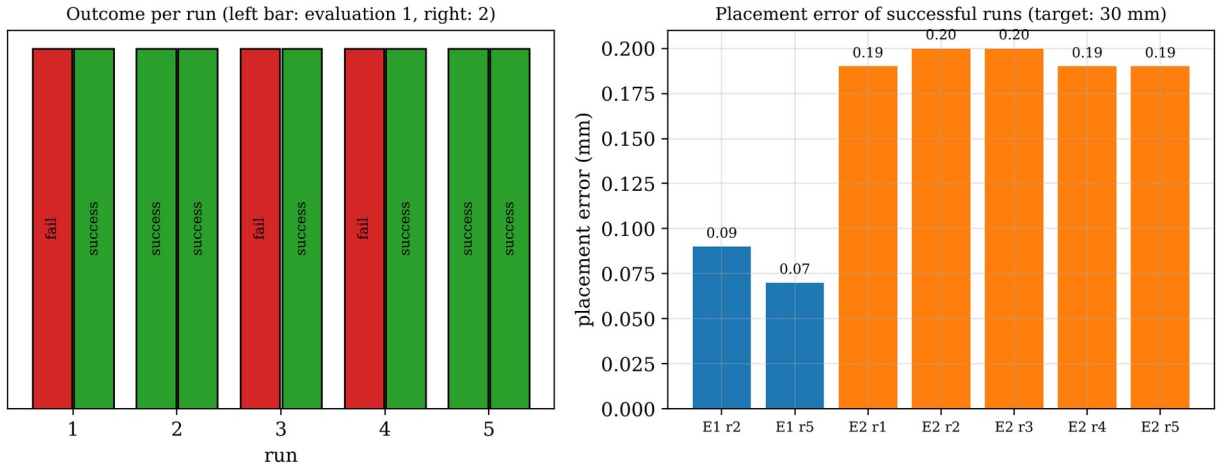


Figure 7.7: Outcome of each evaluation run and placement error of the successful runs

The first evaluation succeeded in 2 of 5 runs (40 %). The second, run after the attach and release commands were confirmed, succeeded in all 5 runs (100 %), with a mean placement error of 0.19 mm (standard deviation 0.01 mm), a mean mission time of 160.18 simulated seconds (range 160.08-160.30 s) and every attach and release confirmed on the first attempt. Its missions took a mean of 47.6 minutes of wall-clock time (n = 4; run 4 was left out because the computer slept during it). Every successful run in either evaluation placed the container within 0.2 mm of the target with no measurable tilt, and every completed mission took 160-163 simulated seconds.

7.5.3 Failure Analysis

All three failures of the first evaluation had the same root cause, and none was caused by driving, line following or arm motion (Figure 7.8). Attach and release were sent with a command-line tool that opens a new connection for each message and sends it once. If the connection to the simulator is not ready when the message is sent, the message is lost or arrives late. Run 4 lost an attach, so the gripper lifted nothing; run 3 lost a release, so the container was carried home in the gripper; and run 1 most likely received a late release after the robot had begun to move, so the container fell off the table edge.

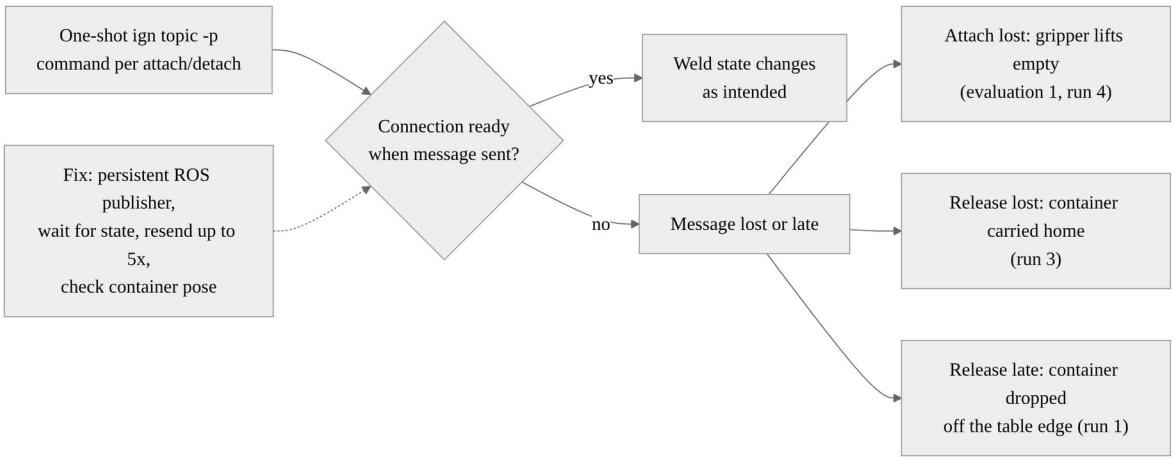


Figure 7.8: Cause of the failures in evaluation 1 and the fix

The mechanism was confirmed from the container positions recorded after every step from run 3 onward: in run 3 the container was at a height of 0.227 m after the placement step, still in the gripper, and it remained in the gripper through the rest of the mission. The fix sends every command through one persistent publisher, waits for the DetachableJoint system to report the new state, resends up to five times, and then checks the container's real position before the robot moves on.

Testing the fix exposed a second, hidden problem. In a short test, the attach command was sent while the gripper was still wide open (1.397 rad). The gripper wait judged the gripper "settled" after half a second of wall-clock time without movement, but at a low real-time factor half a wall-clock second is only a few hundredths of a simulated second, so the gripper had barely started to close. The wait was changed to simulated time, and a check that the gripper ends between 0.35 and 1.2 rad was added before the attach.

The evaluation itself also contained an error. Its first scoring rule counted run 4 of the first evaluation as a success, because the container stood upright on a table, although it was the wrong table. Success now also requires the container to be within 30 mm of the target, and all runs were re-scored from their stored raw poses.

7.6 Patient Room and Smart Table Tests

The patient room and the smart bedside table were tested in three steps after the 5-run evaluation: the table on its own, then complete scheduled missions in the new room, and finally the effect of the room on the simulation speed. Because each complete mission takes close to an hour of wall-clock time, the 5-run evaluation was not repeated with the table; this is discussed in Section 7.9.3.

7.6.1 Stand-Alone Table Test

A test script (`test_smart_table.py`) drove the table through its whole behaviour in a fresh simulation, with the robot at its start position and the container spawned directly on the lowered platform, and checked 29 conditions (Table 7.6). The table stayed lowered while the container was welded, while the weld was released but the arm was not yet home, and while the weld was attached again; it started to lift only once all three conditions of Section 5.7.3 held. It reached 0.7997 m after 18.6 simulated seconds at a cruise speed of 39.9 mm/s. The container stood on the raised platform at a height of 0.830 m, exactly the platform height plus half the container's height, with a tilt of 0.002°, and it had moved by only 0.08 mm during the lift. When the container was removed, the table started lowering 3.04 s later and returned to 0.1004 m. Manual raise and lower also worked, and a table raised by hand without a container stayed raised. Figure 7.9 shows the table with the container before and after the lift.

One of the 29 checks failed, and the failure was in the test, not in the table. The test expected no contact while the container was welded 1 mm above the platform, but the container drops onto the platform in the first physics step, before the weld forms, so it is welded while touching. The table correctly stayed lowered, because the weld was not released. The check was changed to "the table stays lowered while the container is welded", but the test was not run again, so its recorded result remains 28 of 29. The test's speed check accepted 30 to 50 mm/s, a wider band than the 40 mm/s

limit of NFR-9; the measured cruise speed of 39.9 mm/s is within the limit, but the acceleration was not measured separately and is set only by the controller's profile.

Check	Result	Status
Lowered at start-up	Platform at 0.100 m	Pass
Stays lowered while the container is welded	0.100 m for 3 s	Pass
No contact while welded 1 mm above the platform	Contact steady (container dropped before the weld formed)	Test error
Stays lowered with the weld released but the arm away from home	0.100 m for 3 s	Pass
Stays lowered with the weld attached again	0.100 m for 3 s	Pass
Starts lifting once the arm is home	State "lifting"	Pass
Reaches the raised height	0.7997 m after 18.6 s	Pass
Lift speed 30-50 mm/s	39.9 mm/s cruise	Pass
Container on the raised platform	z 0.830 m, tilt 0.002°	Pass
Container did not slide	0.08 mm	Pass
Lowers when the container is removed	After 3.04 s, to 0.1004 m	Pass
Manual raise without a container, stays raised	0.7998 m, raised for 6 s	Pass
Manual lower	0.1004 m	Pass
All checks	28 of 29 passed	1 test error

Table 7.6: Results of the stand-alone smart table test

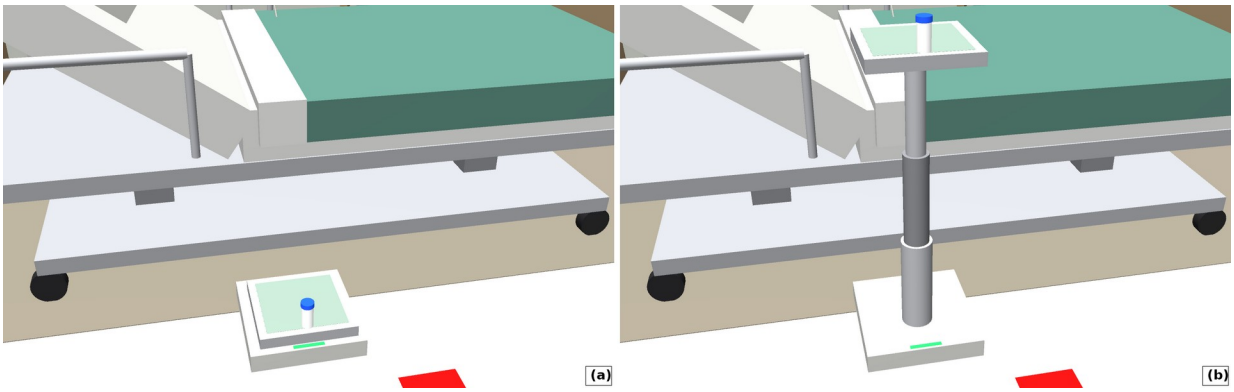


Figure 7.9: Close-up of the smart table with the container: (a) lowered after the place, (b) raised beside the mattress

7.6.2 Scheduled Missions in the Patient Room

Two complete missions were started by the scheduler in the new room, each from a fresh simulation: the first in the normal world and the second in the video world, in which the overview camera renders at 2 Hz for recording. Before them, the line follower was run once on its own in the new world: it arrived at the patient marker after 38.7 simulated seconds at (1.6992, 1.4723) m with

a heading of 89.99°, identical to its position before the room was added, and the wrist camera at the stop saw only the path and the marker.

Both missions succeeded (Table 7.7). In each, every gripper command was confirmed on the first attempt, the table's contact sensor confirmed the container while the table was still lowered, the table lifted it to 0.80 m once the arm was home, and the container was found upright on the raised platform before the robot turned round. The placement errors, 0.27 mm and 0.25 mm, are as small as in the 5-run evaluation, which shows that the table changed nothing in the robot's place motion.

Measure	**Mission 1 (normal world)**	**Mission 2 (video world)**
Result	Success, 11 of 11 steps	Success, 11 of 11 steps
Attach / detach attempts (spawn, pick, place)	1 / 1 / 1	1 / 1 / 1
Placement error at contact confirmation	0.27 mm	0.25 mm
Container tilt at placement	0.13°	0.13°
Contact confirmed (table lowered, 0.100 m)	Sim t 82.08 s	Sim t 84.36 s
Lift: start, raised, duration	91.48 s, 110.04 s, 18.56 s	93.70 s, 112.38 s, 18.68 s
Platform height after the lift	0.7998 m	0.7997 m
Container after the lift	z 0.830 m, tilt 0.00°, on the platform	z 0.830 m, tilt 0.00°, on the platform
Patient stop	(1.6993, 1.4730) m, 90.00°	(1.6992, 1.4719) m, 89.99°
Parked at	(0.002, 0.000) m, −0.26°	(0.002, 0.000) m, −0.26°
Mission time, simulated	178.34 s	178.38 s
Mission time, wall-clock	58.5 min	49.8 min

Table 7.7: Results of the two scheduled missions in the patient room

The missions are 18.1 s longer than in the second evaluation (178.3 s against 160.2 s): the lift itself takes 18.6 s, and the rest of the mission ran about 0.5 s faster. The complete cycle of 178.3 s is still within the 180 s target of NFR-1, although with little margin. Figures 7.10 and 7.11 show the second mission from the overview camera, at the main steps of the delivery.

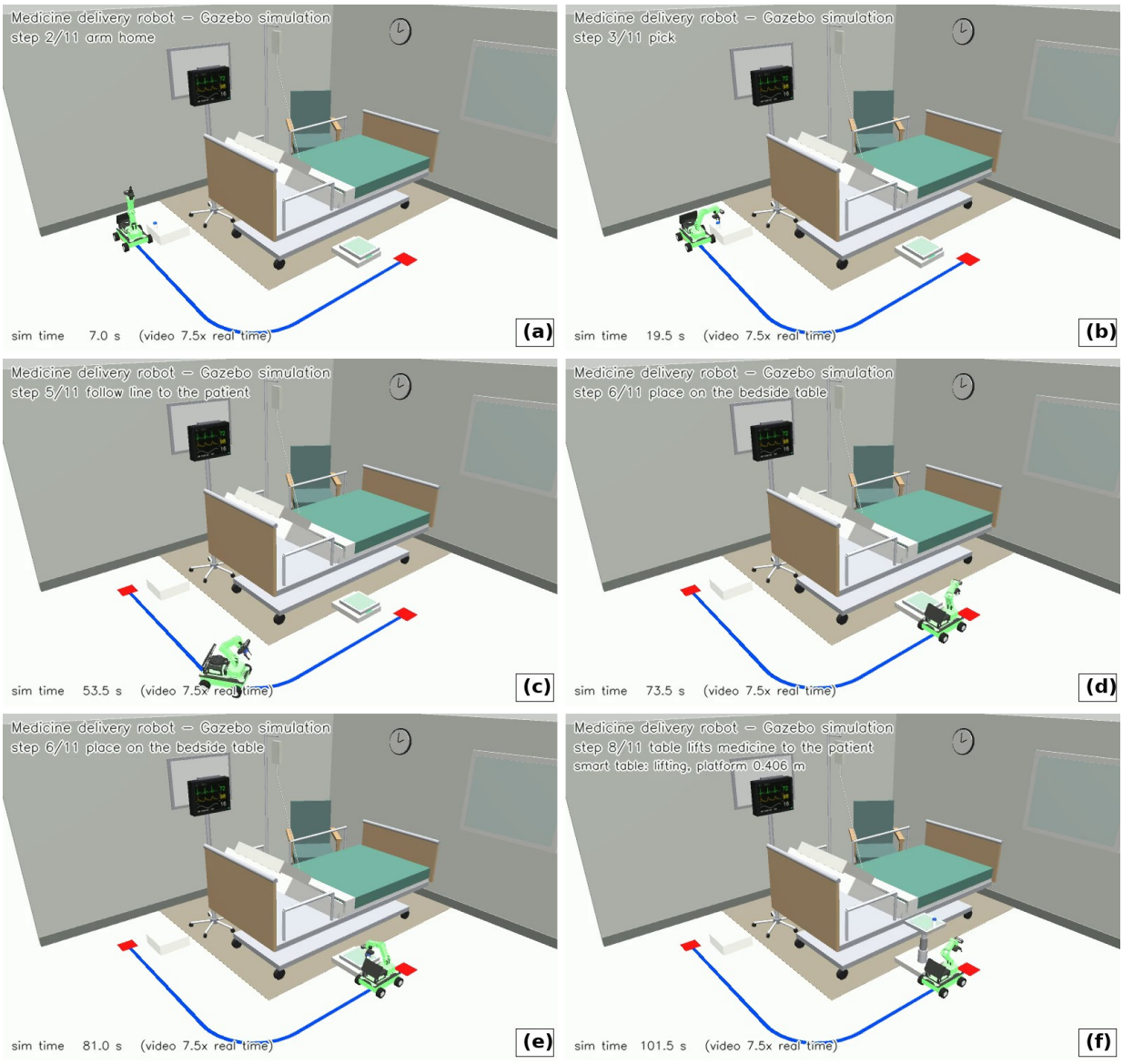


Figure 7.10: Scheduled mission in the patient room, part 1: (a) arm to the ready pose, (b) pick at the medicine station, (c) following the line, (d) arrival at the table, (e) place on the lowered table, (f) the table lifting the medicine (platform at 0.41 m)

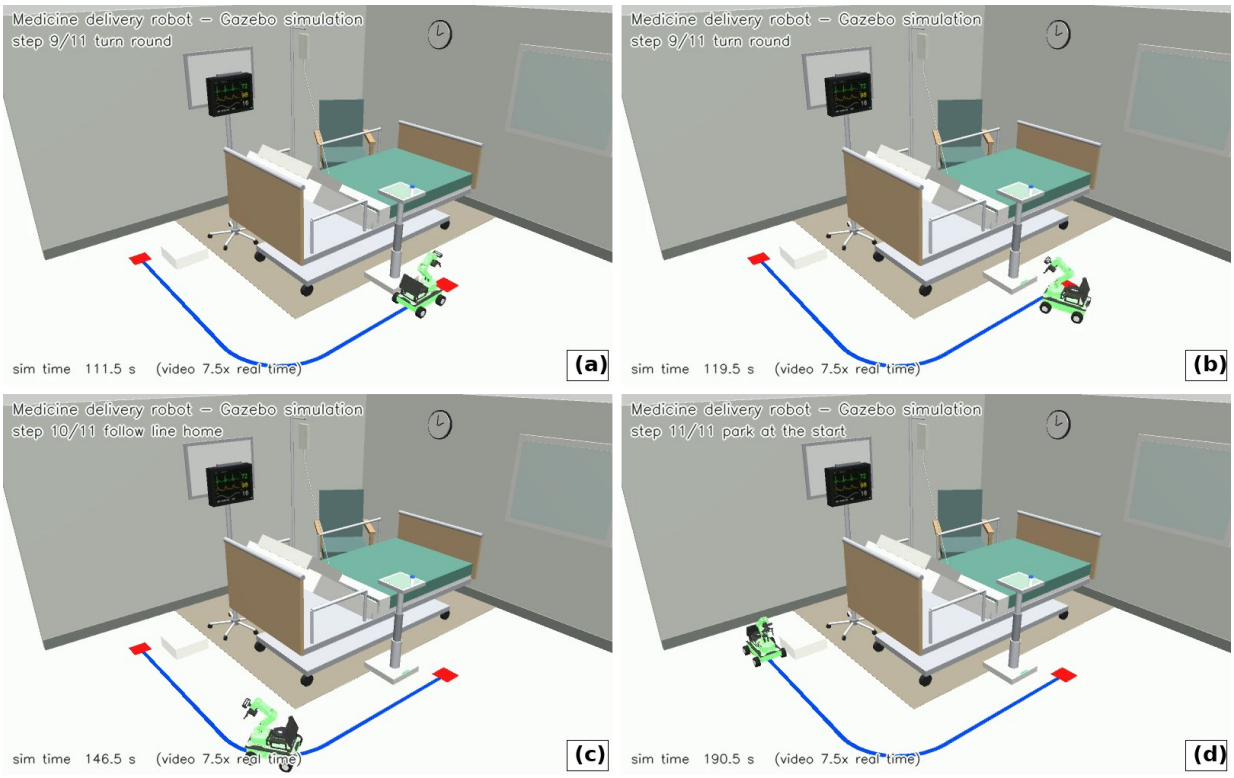


Figure 7.11: Scheduled mission in the patient room, part 2: (a) table raised beside the bed, (b) turning round, (c) following the line home, (d) parked at the start

7.6.3 Effect on the Simulation Speed

The real-time factor was measured with the same headless procedure before and after the room was added: a fresh launch, the arm moved home, 20 s to settle, then the ratio of simulated to wall-clock time over 180 s. The first three measurements of the original world, taken in the afternoon, already varied from 6.75 % to 8.24 % on the same world, so the final comparison was interleaved, with the computer otherwise idle: after, before, after, before (Figure 7.12). The two runs with the room gave a mean of 8.74 %, and the two runs of the original world 9.37 %, a difference of 6.7 %. This is below the 10 % limit of NFR-10 and well within the run-to-run spread of the original world, whose five measurements ranged from 6.75 % to 10.33 %, so with only two samples per world the room had no measurable effect on the simulation speed, and no simplification was needed. Like all the real-time factors compared in this chapter, these were measured headless with the robot standing still; the rate during missions is lower (Section 7.3.2).

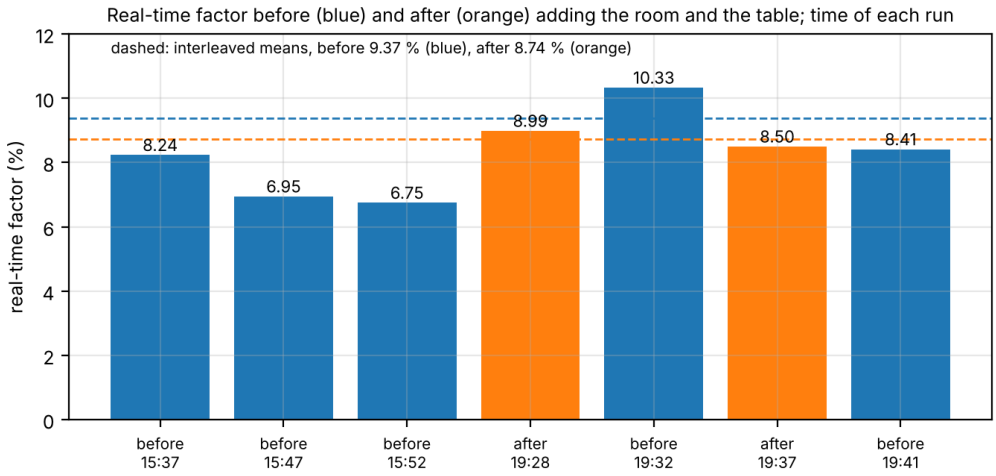


Figure 7.12: Real-time factor of the original world and of the world with the patient room and the smart table

7.7 Hospital System Tests

The hospital system was tested in the order planned in Section 5.12: offline tests without the simulator, then each new capability in Gazebo, then one complete delivery to the farthest room, and finally a demonstration of four scheduled deliveries to the four rooms. Every Gazebo test started from a fresh launch of the simulation, and no simulator process was left after any of them.

7.7.1 Offline Tests

Table 7.8 summarises the offline tests, which were all repeated after the last code change. The new offline route simulator, built from the measured camera geometry, first reproduced the tested single-room route: it stopped the robot at (1.6993, 1.4752-1.4767) m after 38.8-39.0 simulated seconds, against (1.6992, 1.4723) m after 38.6-39.0 s in Gazebo. It then drove 17 route legs, which cover all four rooms and the routes from all six slots, each from three start conditions (on the line, and offset by ±12 mm and ±1°), giving 51 runs at 0.08 m/s and 51 with 0.12 m/s on the straights; every run stopped within 15.6 mm of its expected point. In three fault cases (the J1 marker missing, the J2 marker red instead of green, and Room 1's stop marker missing) the robot stopped with an error, as designed.

Test	What it covers	Result
Route simulator, old route	Reproduces the Gazebo-tested single-room route	4 of 4
Route simulator, 0.08 m/s	17 route legs × 3 start conditions	51 of 51
Route simulator, 0.12 m/s on straights	17 route legs × 3 start conditions	51 of 51

Test	What it covers	Result
Route simulator, fault cases	Missing or wrong markers stop the robot	3 of 3
Unit tests	Planner, map, scheduler queue, hospital clock, stock, scoring	19 of 19
Arm-wait test	Reached, settled, failures, frozen subscription, gripper, no messages	7 of 7

Table 7.8: Offline tests of the hospital system

7.7.2 Subsystem Tests in Gazebo

Table 7.9 lists the Gazebo tests. The world loaded with all four tables lowered and all six containers in their slots. In the route test the robot drove from home to the pharmacy, from there to Room 3 and back home, which includes every kind of junction event: passing the home marker, going straight through J1, a right turn at J2 and a left turn on the way back. All nine checks passed at 0.08 m/s, with stops 7.1, 15.6 and 7.1 mm from the expected points and the robot never more than 1.9 mm off the line. Repeated with 0.12 m/s on the straights, the test passed again, with stops 5 to 7 mm off and the two long legs 11 s shorter, so the higher speed was adopted.

In the pick test the robot picked the containers in slots S1, S3 and S6, carried each to the ready pose and put it back. Every attach and release was confirmed at the first attempt, each container went back 0.4, 1.3 and 0.7 mm from its slot, and the neighbouring containers, 7 cm away from each, moved by 0.00 mm, so the open gripper never touched them. The cap test, after the change to a purple cap, read all six caps correctly, with the robot lined up within 0.5 mm of every slot. Finally, a short check of the new arm wait, with the robot carrying a container, reached the home pose in 3.6 simulated seconds both normally and with the joint-state subscription frozen on purpose, which the wait detected after 15 s and recovered from.

Test	Result	Status
World load and photographs	4 tables lowered, 6 containers in their slots	Pass
Route test, Room 3, 0.08 m/s	9 of 9; stops 7.1 / 15.6 / 7.1 mm; line offset ≤ 1.9 mm; 39 min	Pass
Route test, Room 3, 0.12 m/s on straights	9 of 9; stops 5-7 mm; long legs 11 s shorter; 33 min	Pass
Pick and replace at S1, S3, S6	12 of 12; replaced within 0.4-1.3 mm; neighbours moved 0.00 mm; 107 min	Pass
Cap test, all 6 slots	9 of 9; all caps read correctly; robot within 0.5 mm of each slot	Pass
Complete delivery to Room 3	Success; placement 2.8 mm (before the fine line-up); 72 min	Pass
Arm home while carrying, normal and frozen	Reached in 3.6 sim s both times; freeze detected and recovered	Pass

Table 7.9: Gazebo tests of the hospital system

7.7.3 Complete Delivery to the Farthest Room

The first complete delivery went to Room 3, the farthest room, with Ibuprofen from slot S3. It succeeded in 232 simulated seconds (72 minutes of wall-clock time): the robot took the planned turns, stopped at Room 3 at (−3.6508, 1.6273) m with a heading of 89.97°, placed the container on the table, waited while the table lifted it to 0.80 m in 18.6 s, and parked 2 mm from home. The other five containers were untouched. The placement error of 2.8 mm, much larger than in the single room although well within the 30 mm limit, led to the fine line-up described in Section 6.12.5.

7.7.4 Demonstration of Four Scheduled Deliveries

The demonstration created four doses on the hospital clock: Ibuprofen for Patient C in Room 3 and Paracetamol for Patient A in Room 1, both due at 08:00, Metformin for Patient D in Room 4 at 08:02, and Amoxicillin for Patient B in Room 2 at 08:04. The cap check was switched on. These doses are built into the scheduler's demonstration mode, which does not read the schedule file (`doses.yaml`); reading that file, and choosing the medicine due from it, is covered by the scheduler's unit tests (Table 7.8).

Attempt 1 succeeded in its first two deliveries (Rooms 1 and 3, placed 1.6 and 1.5 mm from the target), but its third delivery stopped at "arm home (carrying)" because of the frozen joint-state subscription described in Section 6.12.6, and the fourth dose was not run. The robot itself was safe at the pharmacy; the system had stopped exactly as designed for a failure after the robot had moved.

Attempt 2, after the fix, ran from 13:13 to 18:54 (5.7 hours of wall-clock time, 845 simulated seconds) and **all four deliveries succeeded** (Table 7.10). In every delivery the cap was read correctly, every attach and release was confirmed at the first attempt, the table rose to 0.7996-0.7999 m with the container upright on the platform at 0.830 m, the other containers moved by at most 0.14 mm, and the robot parked within 2 mm of home. The frozen-subscription recovery was never needed.

Room	Patient	Medicine	Slot	Due	Started	Late	Placement	Lift	Sim. time	Wall time
1	A	Paracetamol 500 mg	S1	08:00	08:00:00	0.0 min	2.0 mm	18.4 s	185 s	67 min
3	C	Ibuprofen 400 mg	S3	08:00	08:03:05	3.1 min	1.4 mm	18.5 s	238 s	103 min
4	D	Metformin 500 mg	S4	08:02	08:07:03	5.1 min	0.6 mm	18.7 s	239 s	93 min

Room	Patient	Medicine	Slot	Due	Started	Late	Placement	Lift	Sim. time	Wall time
2	B	Amoxicillin 250 mg	S2	08:04	08:11:02	7.0 min	1.7 mm	18.5 s	181 s	74 min

Table 7.10: Results of the demonstration (attempt 2)

The order of the deliveries shows the scheduler working as designed (Figure 7.13). Rooms 1 and 3 were both due at 08:00; Room 1 went first because its route is shorter, and Room 3 followed as soon as the robot was back. Room 4 and then Room 2 followed in order of their due times. Because the robot carries one container at a time, each dose waited for the previous delivery, and the last started 7 hospital-minutes after it was due. The medicine reached the patient, on the raised table, 2.2, 5.7, 7.7 and 9.2 hospital-minutes after the due times of Rooms 1, 3, 4 and 2. The robot stopped in every room within 7 mm of the design stop position, which does not affect the placement because the place target is computed from the robot's actual stop, and every junction event happened as planned (Table 7.11).

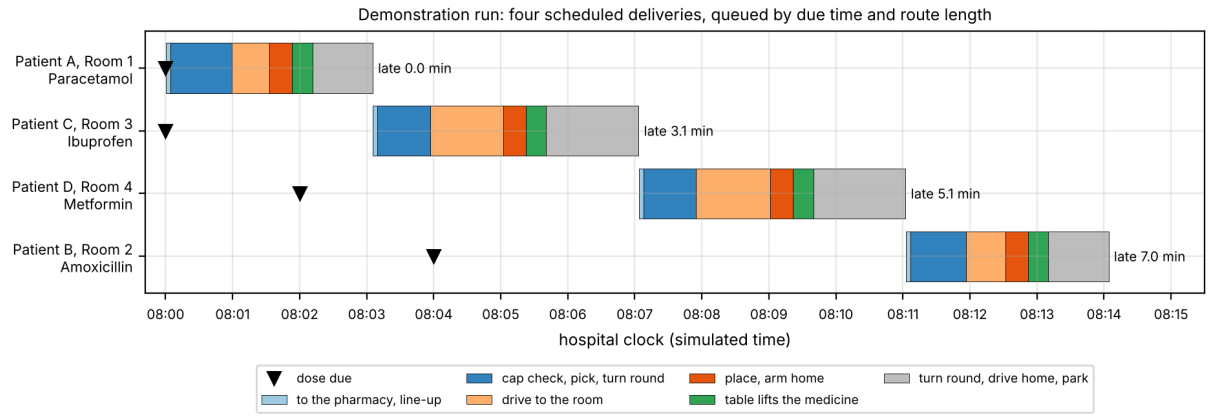


Figure 7.13: Timeline of the demonstration on the hospital clock: the phases of each delivery, the due times and the lateness

Room	Pharmacy → room	Room → home
1	J1 right (70.0)	J1 left (165.8)
3	J1 straight (250.5), J2 right (279.3)	J2 left (375.0), J1 straight (410.5)
4	J1 straight (489.5), J2 left (518.3)	J2 right (614.0), J1 straight (649.8)
2	J1 left (728.8)	J1 right (824.5)

Table 7.11: Junction events of the demonstration, with the simulated time at which each marker was seen (s)

Figures 7.14 to 7.16 are frames of the demonstration video, which is 114 s long at 7.5 times real time. They show the first delivery from the pharmacy to Room 1, the four kinds of junction turn, and the four tables lifting the medicine to their patients.

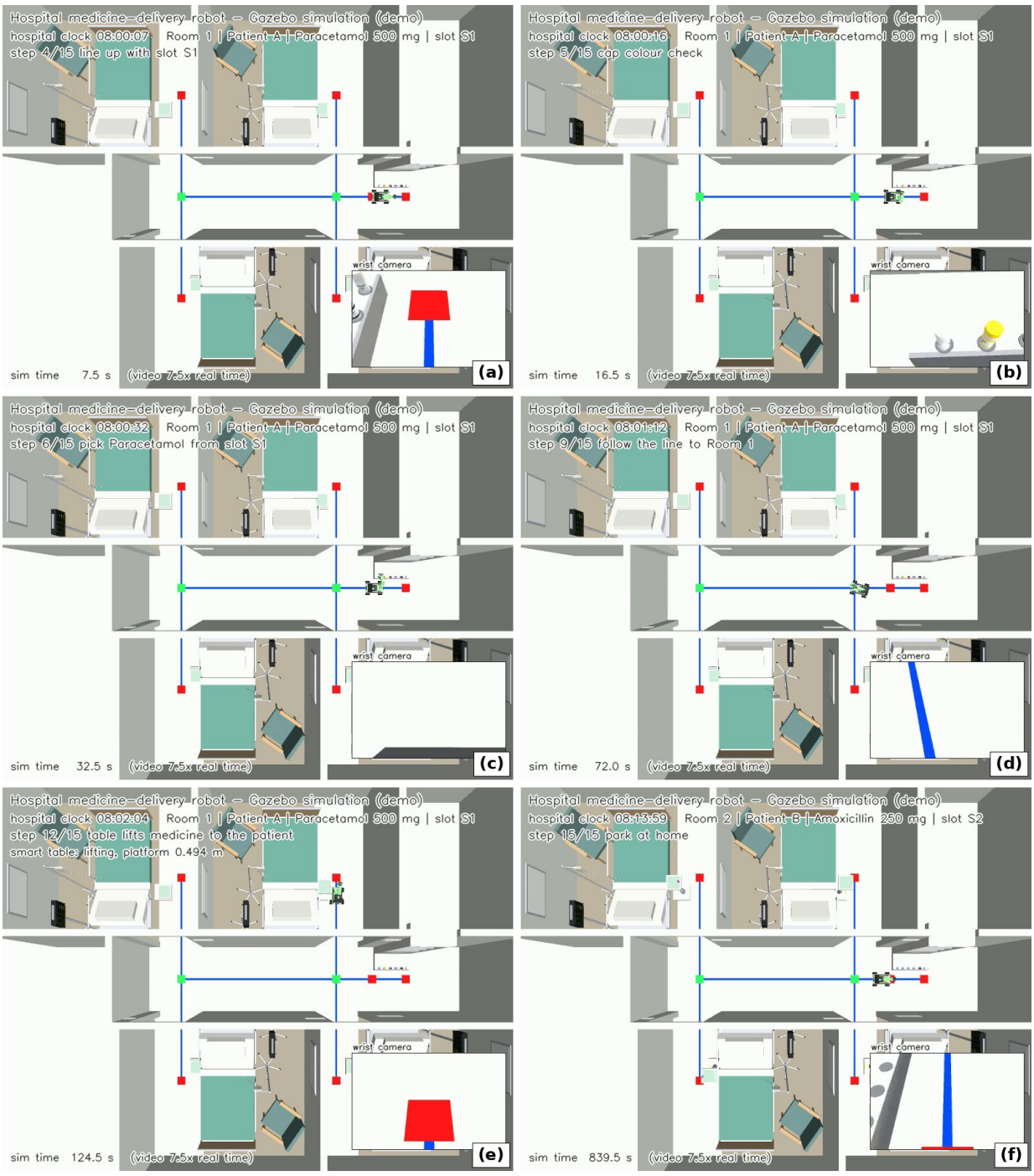


Figure 7.14: First delivery of the demonstration: (a) line-up at slot S1, (b) cap check (white), (c) pick, (d) right turn at J1, (e) the table in Room 1 lifting the medicine, (f) the robot parked at home after all four deliveries, with all tables raised

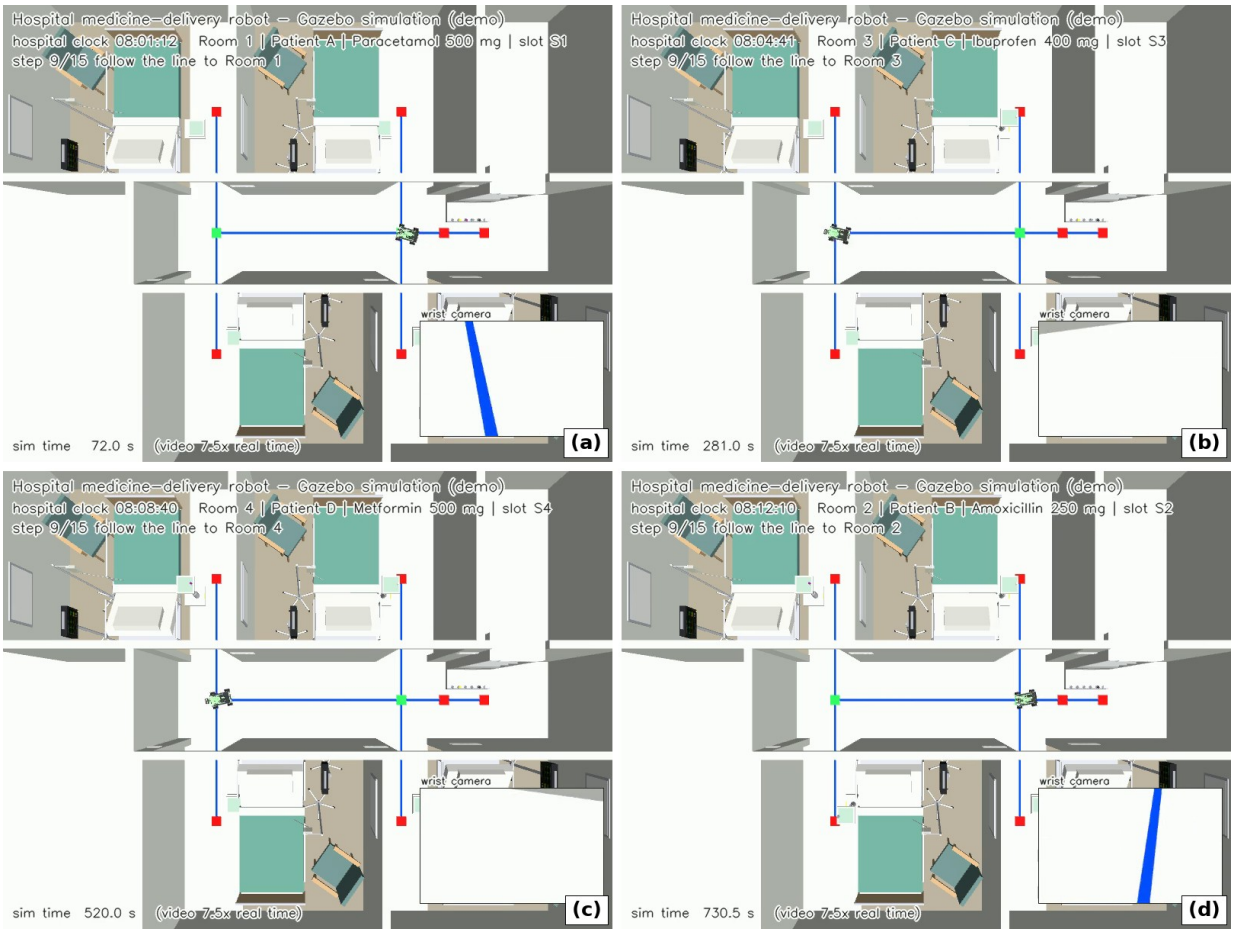


Figure 7.15: Junction turns in the demonstration: (a) right at J1 to Room 1, (b) right at J2 to Room 3, (c) left at J2 to Room 4, (d) left at J1 to Room 2

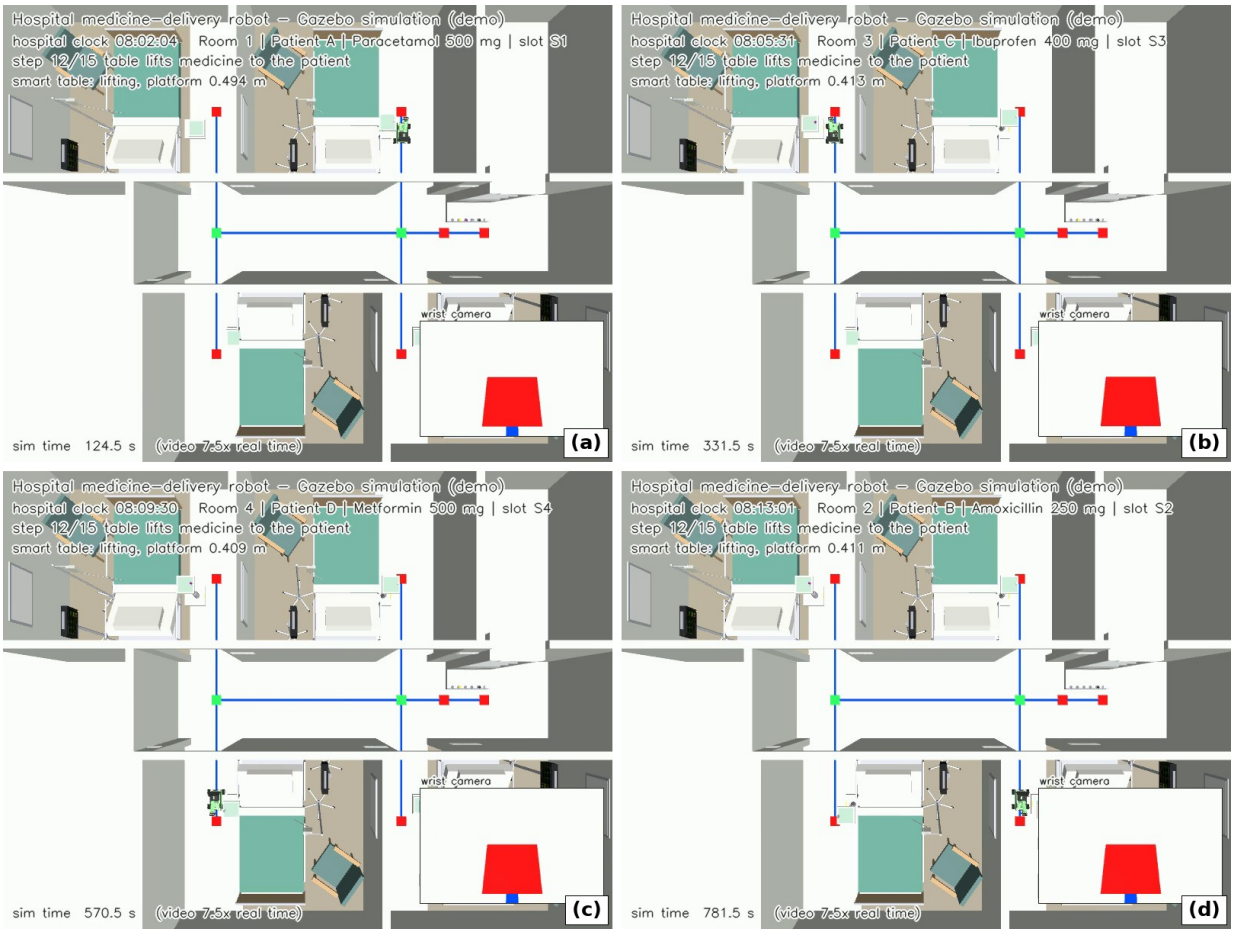


Figure 7.16: The smart tables of (a) Room 1, (b) Room 3, (c) Room 4 and (d) Room 2 lifting the delivered medicine

7.7.5 Placement Accuracy and Simulation Speed

Figure 7.17 compares the placement error of every successful scored delivery in the project: the successful runs of both evaluations, the first scheduled delivery, the patient-room missions, and the hospital deliveries of the first Room 3 test and of both demonstration attempts. The demonstration's errors of 0.6 to 2.0 mm are larger than the 0.2 mm of the single room, but far inside the 30 mm limit. Two small offsets in the same direction explain them: the robot lines up about 0.5 mm from the slot centre despite the fine line-up, and the delivered containers stood 1.1 to 1.3 mm from their slot centres when they were picked, so the gripper takes each container slightly off-centre. As in the single room, the placement error is measured from the place target computed from where the robot actually stopped in the room, so it describes the arm's place motion. Measured from the map's place target instead, the containers were 3.8 to 6.2 mm away, because the robot stopped 5.1 to 6.6 mm from the map's stop position; both are far inside the 30 mm limit.

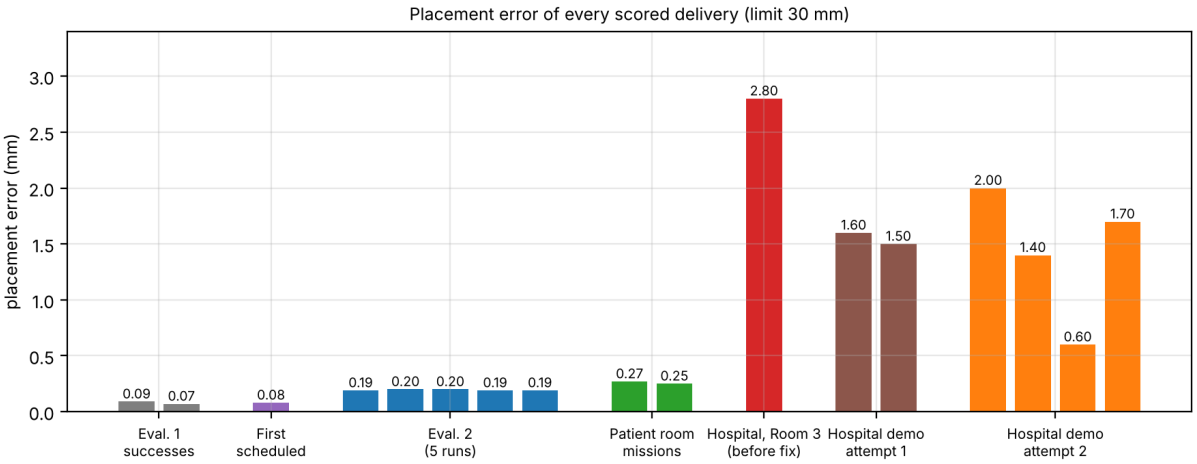


Figure 7.17: Placement error of every successful scored delivery: both evaluations, the first scheduled delivery, the patient-room missions, the first hospital delivery before the fine line-up and both demonstration attempts

The real-time factor was measured as for the single room, interleaving the two worlds (Figure 7.18). With four rooms and six containers it was 7.46 % and 9.23 % (mean 8.35 %), against 10.67 % and 10.70 % (mean 10.68 %) for the single room on the same afternoon: a reduction of 22 %, but still above the 7 % budget of NFR-11, so no simplification was needed. Like the patient-room comparison, these are idle, headless measurements, taken with only two samples per world. During the demonstrations, with the robot moving and the recorder running, the simulation ran at about 4 to 5 % of real time, which is why a three-to-four-minute delivery took more than an hour.

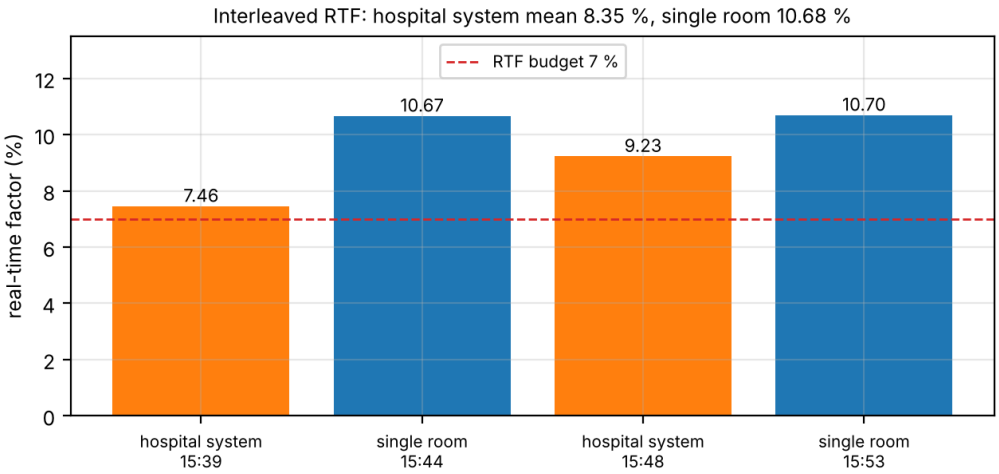


Figure 7.18: Real-time factor of the hospital system and of the single-room world, measured alternately

7.8 Requirement Verification

Table 7.12 compares each requirement with the measured result.

ID	Target	Result	Status
FR-1	Start within 1 min of the due time	Started at the due second	Met
FR-2	Container rises with the gripper	Lifted in every run of evaluation 2 (height 0.227 m), in both patient-room missions and in every hospital delivery	Met
FR-3	Stop within 5 cm of the planned point	Ward: 3 mm (10 Hz camera), 12 mm (4 Hz), 2.6 cm at home; hospital: 5-16 mm	Met
FR-4	Within 30 mm of target, tilt < 10°	Evaluation 2: 0.19-0.20 mm, 0.0°; patient room: 0.25-0.27 mm, 0.13°; hospital: 0.6-2.8 mm, at most 1.1° at placement	Met
FR-5	Park within 5 cm and 5° of the start	Ward and patient room: 2 mm, 0.3°; hospital: within 2 mm	Met
FR-6	Detect a failed grasp or release	Five container checks implemented after evaluation 1 revealed the failure mode; in every later run each attach and release was confirmed at the first attempt, so the stop on a failed check was never triggered	Implemented, not exercised by a failure
FR-7	Record every delivery	Logged with outcome, placement error and attempts	Met
FR-8	Stop within 2 s of losing the line	Implemented and tested offline; not triggered in Gazebo	Partly verified
FR-9	Daily and one-off times	Both forms implemented; scheduled runs used a due time one minute ahead and the daily entries; a dated one-off entry was not run as a delivery	Partly verified
FR-10	No overlapping missions	Deliveries run one after another	Met by design
FR-11	Lift to 0.80 m within 30 s, container on the platform	18.4-18.7 s; 0.7996-0.7999 m; container upright on the platform in the table test, both patient-room missions and all hospital deliveries	Met
FR-12	No movement while the gripper is above the table or holding the container	Table stayed lowered while welded and until the arm was home (3 checks)	Met
FR-13	Lower after the container is taken	Lowering started 3.04 s after removal	Met
FR-14	Choose the due medicine and its slot from the schedule and the clock	Right container in 4 of 4 demonstration deliveries (built-in demonstration doses); schedule file covered by unit tests	Met

ID	Target	Result	Status
FR-15	Reach the right one of four rooms by a planned route	Route tests 9 of 9 (twice); all junction events of the demonstration as planned	Met
FR-16	Check the cap before picking; a wrong cap stops the delivery	Cap test: 6 of 6 caps classified correctly; 4 of 4 in the demonstration; a wrong cap was never presented	Partly verified: classification only
FR-17	Queue doses due together; log lateness	Rooms 1 and 3 due together, shorter route first; lateness logged	Met
FR-18	Skip and log a medicine that is out of stock	Scheduler unit tests	Met (offline)
FR-19	Leave the other medicines undisturbed	Neighbours moved 0.00 mm (pick test), ≤ 0.14 mm (demonstration)	Met
FR-20	Stop on a missing or unexpected marker	3 of 3 fault cases in the route simulator	Met (offline)
NFR-1	Single-room cycle ≤ 180 s simulated	160-163 s; 178.3 s with the smart table. Hospital system, for which no target was set: 181-239 s	Met in the single room; not met by the hospital routes
NFR-2	≥ 4 of 5 runs succeed	2 of 5 before the fix; 5 of 5 after	Met
NFR-3	0.08 m/s near markers and furniture; ≤ 0.12 m/s on corridor straights	0.078-0.080 m/s at 0.08 m/s; route test at 0.12 m/s passed (stops 5-7 mm)	Met
NFR-4	No rotation within 0.27 m of a table	Robot moves 0.12 m sideways before rotating	Met
NFR-5	Hardware code isolated	Same interfaces as the hardware; but the container checks, the weld grasp, the exact odometry and the table's contact sensor exist only in simulation	By design, not verified on hardware
NFR-6	Fresh launch per run; version control	Automated; all milestones committed and backed up	Met
NFR-7	Plain-text schedule, spreadsheet log	YAML schedule, CSV log	Met
NFR-8	Joints ≥ 0.1 rad inside limits	Enforced by the IK solver	Met
NFR-9	Table ≤ 40 mm/s and 40 mm/s², container does not slide	39.9 mm/s cruise; container moved 0.08 mm; acceleration not measured	Met for speed
NFR-10	RTF drop ≤ 10 % with the room	−6.7 % (idle means 8.74 % vs 9.37 %, two samples each), within the run-to-run spread	Met: no measurable effect
NFR-11	RTF ≥ 7 % with the hospital system, idle	8.35 % idle (7.46 % and 9.23 %); about 4-5 % during the demonstration	Met for the idle measurement
NFR-12	Longest route ≤ 6 m one way	5.54 m (Rooms 3 and 4)	Met

Table 7.12: Requirements verification

7.9 Discussion

7.9.1 Objectives

The results show that objectives O1 to O3, O5, O7 and O8 were met in simulation. The robot follows the path to the patient and back (O1), places the container accurately and upright (O2), starts deliveries on schedule and logs them (O3), and was evaluated over repeated independent runs (O5). Objective O6 is met by design but not verified: the software uses the same interfaces as the hardware, but it never ran on the robot, and four parts exist only in simulation and must be replaced on hardware: the container checks, which read the container's pose from the simulator; the weld that stands in for a friction grasp; the exact odometry of the planar base; and the ideal contact sensor of the smart table. The smart bedside table presented the medication at 0.80 m, above the mattress and beside the patient's hand, in the table test and in both missions in the patient room (O7). In the hospital system the robot chose the medicine due from the schedule, checked its cap against the inventory, picked it from the right slot, found the right one of four rooms and delivered all four doses of the demonstration (O8). Objective O4, detecting and stopping on a failed grasp or release, is implemented but not exercised: the checks were added after the first evaluation exposed the need for them, but in every later run each attach and release was confirmed at the first attempt, so no check ever had to stop the robot. A test that blocks an attach or a release on purpose is needed to show the stop working, as for the stop on a lost line (FR-8).

The most important result is not the accuracy, which mainly reflects the simulator's ideal joints, but the reliability lesson of the two evaluations. Every component worked in isolation, yet the system failed in three of five complete missions because a gripper command was assumed to have worked. Confirming each command and checking the real state of the container turned the same components into a reliable system.

7.9.2 Comparison with the Proposal

Compared with the proposal, the design trades some autonomy for simplicity and verifiability. Line following on a network of marked lines replaces LiDAR mapping, which the platform does not support; the routes are visible and easy to check, and the route planner still lets the robot choose between rooms. A medicine storage unit replaces the three cups on the chassis, which keeps the chassis free and lets one design serve any number of doses and medicines, at the cost of a pick at the start of each delivery. The final system goes beyond the proposal in two ways it did not foresee: the smart bedside table that lifts the medication to the patient, and the check of each container's cap against the inventory before it is picked.

7.9.3 Limitations

The results must be read with the following limitations:

- **Simulation, not hardware.** All results come from simulation. Real servos, camera noise, lighting and floor texture will reduce accuracy; the 0.2 mm placement error reflects the simulator's ideal joints more than the real arm's.
- **Assisted grasp.** The container is carried by a simulated fixed joint rather than by friction, so grip strength and slip were not tested. A friction grasp worked but was too slow to simulate on the available computer.
- **Kinematic base.** The base moves exactly as commanded; wheel slip, motor limits and the mecanum rollers are not modelled.
- **Estimated dimensions.** The chassis dimensions are estimates (±15 %); this mainly affects the camera height and the reach to the tables.
- **Structured environment.** The path is marked, the tables are fixed, and there are no people or obstacles.
- **Smart table tested in two missions.** The 5-run evaluation used the earlier bedside table. The smart table was tested on its own and in two complete missions, both successful, but not in a repeated evaluation; its contact sensor is ideal, and the patient taking the container was simulated by removing it.
- **Small sample, no noise.** Five runs per evaluation show that the failure mode was removed, but are too few to estimate a long-run success rate with confidence. Because the simulation has no sensor or actuator noise, the runs of the second evaluation are almost identical (0.19-0.20 mm, 160.08-160.30 s): they test for timing faults such as lost commands, not for robustness to disturbances. Only the offline route simulator varied the start position (±12 mm and ±1°).
- **Failure paths not exercised.** The stops on a failed container check (FR-6), on a lost line (FR-8) and on a wrong cap (FR-16) are implemented, but none was triggered in Gazebo.
- **Real-time hospital schedule not run.** The real-time mode was tested only offline. Within one run of the simulation the storage unit is never restocked, and a delivered container stays on its raised table because no patient takes it, so a second dose to the same room would stop at the place step; the schedule file therefore holds one dose per room.
- **No motion watchdog.** The base keeps its last velocity command, and the waits for camera images, odometry and joint states have only wall-clock backstops; a crashed node could leave the robot moving until the demonstration's time limit.

- **Hospital system tested in one demonstration.** The hospital system was tested in one complete delivery and one four-delivery demonstration (all successful after the fixes), not in a repeated evaluation, because one demonstration takes almost six hours. The cap colours were calibrated under the simulation's lighting only, and the cause of the frozen joint-state subscription is still unknown; the system detects and recovers from it, but that recovery was exercised only on purpose. The scheduler process also ends with an abort (exit code −6) in the ROS 2 client library when it shuts down after its last delivery; all results are written before this, so it affects no result, but its cause has not been found.

These limitations define the work needed before the system can be used near patients, which is set out in Section 9.4.

8 IMPACT OF THE ENGINEERING SOLUTION

This chapter discusses the impact of TrackMed beyond its technical performance: its economic, societal and global significance, its environmental and ethical implications, and other issues that a deployment would have to address.

8.1 Economic Impact

The main economic value of TrackMed is nursing time. Medical-surgical nurses spend about 72 minutes of each shift on medication administration [3], and part of this time is spent walking doses from the medicine room to the bedside. A robot that carries routine scheduled doses can return some of that time to direct care, although assessing the patient, answering questions and recording the dose remain nursing tasks. With a global shortage of 5.9 million nurses [4], time saved on routine transport is time available for care that only a nurse can give.

The second economic factor is cost. Hospital delivery robots such as TUG and Moxi have shown that delivery work can be automated at scale [9], [8], but they are purpose-built platforms that require integration with hospital infrastructure. TrackMed uses an educational-grade arm and chassis and open-source software throughout, which lowers the cost of trying bedside delivery in small clinics, care homes and teaching hospitals. A marked floor path is cheap to install and to change, and needs no mapping or localisation infrastructure.

Medication errors cost an estimated US$ 42 billion a year worldwide [2]. A delivery robot does not prevent errors in prescribing or preparing a dose, but it can reduce errors of timing and omission, and its log provides a record of every delivery that can help find where errors occur.

8.2 Societal Impact

For patients, a dose that arrives at the bedside on time removes one barrier to adherence, which averages only 50 % for long-term therapies [1]. For patients with limited mobility it removes the need to get up and reach a device or a nurses' station, which supports independence and dignity. The smart bedside table carries this further: the container is lifted to the side of the bed, above the mattress, which is intended to let a patient who cannot sit up or lean over the bed rail take the dose without help; whether a patient can actually reach the raised platform, 0.22 m above the mattress, was not analysed. In the hospital system the robot also checks each container's cap colour against the inventory before picking it, a simple, automatic check against a container placed in the wrong slot. As populations age, with one in six people worldwide expected to be 60 or older by 2030 [5], the number of patients who need this kind of help will grow.

For staff, the impact depends on how the robot is introduced. If it takes over routine transport and frees time for patient care, it reduces workload and the physical strain of repeated rounds. If it were used to reduce staffing, it could reduce human contact for patients who already see few people. TrackMed is designed and presented as a tool that supports nurses, not one that replaces them.

For education, the project shows how an affordable educational platform and open-source tools can be used to build and evaluate a complete healthcare robot, and how simulation can carry a project through the loss of its hardware.

8.3 Global Significance

The approach is not specific to one hospital or country. It needs only a network of marked lines, a medicine storage unit, a bedside table at a reachable height and a schedule, and the hospital system showed that it scales from one room to several by describing the line network as a graph, so it can be adapted to different layouts by regenerating the world model and marking new lines. The open-source software stack runs on widely available hardware, which matters for health systems with limited budgets. Wider use would require obstacle detection, a way for staff to load and restock the storage unit, a more general identification of medicines than cap colours, several robots sharing the corridors, and integration with hospital information systems, all of which are listed as future work.

8.4 Environmental Impact

The robot runs on a rechargeable battery and contains motors, servos, a camera and an embedded computer, all of which become electronic waste at the end of their life. A production version should use replaceable batteries and repairable, modular components, and should be designed for disassembly and recycling. The energy used per delivery is small, since the robot moves slowly over short distances, so the environmental case rests mainly on long equipment life and repairability rather than on energy savings.

Developing and testing in simulation had environmental benefits of its own. It avoided repeated physical prototypes, test consumables and wear on the hardware, and allowed many complete missions to be run with no material cost. Fewer walking trips for staff do not by themselves reduce energy use, but they reduce physical strain.

8.5 Ethical Considerations

Four ethical issues stand out for a robot that delivers medication.

8.5.1 Patient Safety and Accountability

A robot that delivers the wrong dose, delivers to the wrong bed, or leaves a dose where it is not taken can cause harm. Responsibility must remain with the clinician who prepared and approved the dose, and the robot must make its actions traceable: TrackMed logs every delivery with its outcome, and it stops and reports rather than continuing when a step fails. The design choice of placing the container on the table rather than handing it to the patient avoids contact between the arm and the patient.

8.5.2 Privacy

Schedules and delivery logs identify patients and their medicines. In the UAE, health data processed with information and communication technology must be kept confidential, stored inside the country unless an exception is approved, and retained for at least 25 years [22], and other personal data fall under the Personal Data Protection Law [23]. A deployed system must therefore limit the data it stores, control who can see and change it, and keep an audit trail.

8.5.3 Human Contact and Employment

Delivery robots should take over transport, not the nurse's contact with the patient. Medication rounds are also an opportunity to observe the patient, and a system that removed them entirely could miss early signs of deterioration. The robot is intended to free nurses' time for this kind of care.

8.5.4 Honest Reporting

The results of this project come from simulation. They are reported with their limitations (Section 7.9.3) so that no reader mistakes them for evidence of clinical safety or of performance on hardware.

8.6 Other Issues

- **Safety standards.** A hardware version would need assessment against ISO 13482 for mobile servant robots [19], including speed limits near people, safe stopping and protection against crushing by the arm.
- **Regulation.** If the robot is regarded as part of medication administration, it may be regulated as a medical device, requiring a quality management system such as ISO 13485 [21] and electrical safety assessment under IEC 60601-1 [20].
- **Cybersecurity.** A networked version must authenticate users and devices, encrypt communication, and restrict who can change schedules or command the robot, since a tampered schedule is a safety risk.

- **Hygiene.** Containers and gripper surfaces must be easy to clean between patients, and the robot must not carry contamination between rooms.
- **Acceptance.** Patients and staff must be informed about what the robot does and does not do, and must be able to stop it.

8.7 Challenges and Future Trends

The main challenge for bedside delivery robots is operating safely among people in rooms that change: beds and chairs move, visitors stand in the way and doors open and close. Current trends that address this include depth-camera obstacle detection, learning-based manipulation that copes with varied objects, and fleet management systems that coordinate several robots and integrate with hospital records. Simulation is also becoming a standard part of developing healthcare robots, because it allows many scenarios, including rare and dangerous ones, to be tested before a robot meets a patient [18].

9 CONCLUSIONS AND FUTURE WORK

9.1 Summary of the Design Solution

This project designed, implemented and evaluated TrackMed, a mobile robot that collects a medication container from a medicine station at a scheduled time, follows a marked path to the patient, places the container on a smart bedside table, which lifts it to the patient, and returns. The system combines a Hiwonder JetArm Pro arm with a mecanum chassis, camera line following ported from the manufacturer's own algorithm, a closed-form inverse-kinematics solver derived from the arm's URDF, a kinematic model of the mecanum base, a mission controller that confirms every step against feedback, a scheduler with a delivery log, an automated evaluation tool, and a smart bedside table with a contact sensor and a telescopic column, set in a furnished patient room. It was finally extended into a small hospital system: a medicine storage unit with six medicines, four patient rooms connected by a network of marked lines, a route planner, junction handling, a check of each container's cap colour against the inventory and a scheduler that decides which medicine to fetch and to which room to take it.

After the physical robot failed during development, the full system was implemented and evaluated in a Gazebo simulation built from the manufacturer's robot model. In simulation the robot completed the full delivery cycle in 160-163 simulated seconds, placed the container within 0.2 mm of its target with no measurable tilt in every successful run, and parked within 2 mm of its start pose. A first evaluation succeeded in 2 of 5 runs, with all failures caused by lost gripper commands; after the commands were confirmed and the container's position checked at each critical step, all 5 of 5 runs succeeded. In the furnished patient room, the smart table detected the container and lifted it from 0.10 m to 0.80 m in 18.6 s, and both complete scheduled missions succeeded, with placement errors of 0.27 and 0.25 mm and the container upright on the raised platform. In the hospital system all final offline and Gazebo tests passed after the fixes described in Section 6.12, and a demonstration of four scheduled doses, run after a first attempt had stopped on a fault and the fault had been fixed, delivered all four to the right rooms, in the order set by their due times and route lengths, with the right container on each raised table, the other medicines undisturbed and placement errors of 0.6 to 2.0 mm. Each hospital delivery took 181 to 239 simulated seconds, longer than the 180 s set for the single-room ward (NFR-1), because the routes are up to 5.54 m long.

Objectives O1-O3, O5, O7 and O8 were met in simulation. O4 (stopping on a failed grasp or release) and O6 (transfer to hardware) are met by design but were not exercised: no check ever had

to stop the robot, and the software never ran on hardware. Physical validation, the remaining part of the original plan, could not be carried out because the robot failed.

9.2 Key Challenges and Problem-Solving Approaches

The project met a series of challenges, each of which was solved by measuring the problem before changing the design:

- **Loss of the hardware.** Solved by moving to a simulation built from the manufacturer's model, with the same interfaces as the robot, so that no design work was lost.
- **An unusable simulator.** The blank 3-D view, a full disk and a very slow simulation were solved by changing the renderer, expanding the disk, and running headless with a lower camera rate.
- **Grasping.** The grasp height was found by measuring the tool point and the finger geometry, and the slow friction grasp was replaced by a confirmed assisted grasp.
- **Base motion.** A base that moved at 7 % of its commanded speed and slowly rose was traced to floor friction and the velocity-control plugin, and replaced by a planar joint model.
- **Unreliable commands.** Lost attach and release commands were found by an automated evaluation and fixed by confirming each command and checking the container's real position.
- **Reach of the patient.** The arm can place only at its own base height, far below a patient in bed. Instead of changing the robot, the bedside table was made to lift the medication, with its platform placed so that it never comes closer to the robot than the old table did; two simulator limitations met on the way (joints stuck on their limits, and a fourth stage in series that would not move) were isolated in a table-only world and designed around.
- **Scaling to a hospital.** Choosing between several medicines, rooms and routes was added without changing the tested pick, place, steering or table: every room is the same tested room moved and rotated, every slot is reached so that the tested pick applies, and every marker on a route is checked against the plan, so that a missing or wrong marker stops the robot, as the offline fault cases showed.
- **A fault that could not be reproduced.** During the first hospital demonstration the joint-state messages stopped reaching the delivery process. Its cause could not be found, so the system was made to detect and recover from it, which was then tested by freezing the messages on purpose.

- **Time.** Because the simulation runs far slower than real time, every wait, timeout and rate was moved to simulated time, which removed a hidden failure in the gripper wait.

9.3 Acquired Technical Skills and Lessons Learned

Through this project the team gained practical experience in ROS 2 and Gazebo, robot description and simulation, robot kinematics, computer vision with OpenCV, feedback control, mission design, automated testing and version control. The most important lessons were:

- **Plan for the hardware to fail.** A simulation of the robot, kept from the start, makes a project resilient to the loss of its hardware.
- **Verify, do not assume.** The largest reliability problem was a command assumed to have worked; checking the real state after every critical step turned 2 of 5 successes into 5 of 5.
- **Measure in the right time base.** In a slow simulation, any check based on the wall clock fires too early.
- **Test the test.** A scoring rule that only checked "upright on a table" counted a failure as a success; the evaluation itself must be checked.
- **Build in small, tested steps under version control,** so that each change can be compared with a known good state.
- **Design new features around what already works.** The smart table and the hospital system were added without changing a single tested robot motion, so the earlier evaluation still applies to the robot, and only the new parts needed new tests.
- **Plan before building, and make long runs independent of the session.** A written plan, checked before any code was written, caught a turning-circle conflict at the pharmacy, and a self-contained demonstration script let a six-hour run finish without supervision.

9.4 Future Work and Recommendations

1. **Hardware validation.** Repair or replace the robot, measure the real chassis, and run the same mission and evaluation tool on hardware. Besides the arm, gripper and velocity interfaces, four simulation-only parts must be replaced: the container checks, which read the container's pose from the simulator, the weld grasp, the exact odometry and the table's ideal contact sensor.
2. **Friction grasping.** Replace the assisted grasp with a real friction grasp, using faster computing for simulation, and confirm the grasp from the gripper's position or motor current.

3. **Obstacle detection and safety.** Use the depth camera to stop for people and objects on the path, add an emergency stop, and slow down near beds, following ISO 13482.
4. **A larger hospital.** The hospital system already serves four rooms and six medicines; extend it to several floors and several robots sharing the corridors, replace the cap colours by printed tags or barcodes that can identify any number of medicines, and restock the storage unit automatically.
5. **Caregiver application.** Build the mobile application planned in the proposal to edit schedules and receive delivery and failure notifications, with authentication and encrypted communication.
6. **Longer and harder evaluation.** Repeat the 5-run evaluation with the smart table and the hospital demonstration, then run at least 20-30 independent missions, including disturbances such as a shifted container, an offset start or a partly worn path, to estimate reliability with confidence, and add fault-injection runs that block an attach or a release, present a wrong cap and hide the line, to exercise the safety stops of FR-6, FR-16 and FR-8.
7. **Reducing the sim-to-real gap.** Add camera noise, lighting variation and wheel slip to the simulation, and measure the real chassis dimensions.
8. **Building the smart bedside table.** Build the table with a linear actuator or a motorised telescopic column, a load cell or capacitive pad as the sensor, and limit switches at both ends; add a pinch guard and a stop on contact while moving, and adjust the raised height to the bed's actual height.

REFERENCES

[1] World Health Organization, *Adherence to Long-Term Therapies: Evidence for Action*, Geneva: WHO, 2003. Summary: "Failure to take prescribed medicine for chronic diseases is a massive, world-wide problem," WHO news release, 1 July 2003. Available at https://www.who.int/news/item/01-07-2003-failure-to-take-prescribed-medicine-for-chronic-diseases-is-a-massive-world-wide-problem

[2] World Health Organization, "WHO launches global effort to halve medication-related errors in 5 years," news release, 29 March 2017. Available at https://www.who.int/news-room/detail/29-03-2017-who-launches-global-effort-to-halve-medication-related-errors-in-5-years

[3] Hendrich A., Chow M. P., Skierczynski B. A. and Lu Z., "A 36-Hospital Time and Motion Study: How Do Medical-Surgical Nurses Spend Their Time?," *The Permanente Journal*, Vol. 12, No. 3, pp. 25-34, 2008. Available at https://pmc.ncbi.nlm.nih.gov/articles/PMC3037121/

[4] World Health Organization, *State of the World's Nursing 2020: Investing in Education, Jobs and Leadership*, Geneva: WHO, 6 April 2020. Available at https://www.who.int/publications/i/item/9789240003279.

[5] World Health Organization, "Ageing and health," fact sheet, 1 October 2025. Available at https://www.who.int/news-room/fact-sheets/detail/ageing-and-health

[6] Hiwonder, *JetArm and Mecanum Chassis Integration Course*, ROS 2 Course, Chapter 11, course documentation supplied with the JetArm Pro.

[7] Hero Health, "Hero Smart Pill Dispenser and Medication Management Service." Available at https://herohealth.com/our-product, accessed 5 November 2026.

[8] Aethon, "Aethon Installs the TUG Autonomous Smart Mobile Robot at 21 Hospitals in 2013," press release, 14 January 2014. Available at https://aethon.com/press-release-aethon-installs-tug-autonomous-smart-mobile-robot-21-hospitals-2013/

[9] Diligent Robotics, "Diligent Robotics Leads U.S. Adoption of Hospital Pharmacy Robotics, Redefines the Last Mile," 16 July 2025. Available at https://www.diligentrobots.com/blog/diligent-robotics-leads-us-adoption-of-hospital-pharmacy-robotics-redefines-the-last-mile

[10] Kyrarini M., Lygerakis F., Rajavenkatanarayanan A., Sevastopoulos C., Nambiappan H. R., Chaitanya K. K., Babu A. R., Mathew J. and Makedon F., "A Survey of Robots in Healthcare," *Technologies*, Vol. 9, No. 1, Article 8, 2021. DOI: 10.3390/technologies9010008

[11] Hiwonder, *JetArm User Manual*, course documentation supplied with the JetArm Pro.

[12] Macenski S., Foote T., Gerkey B., Lalancette C. and Woodall W., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," *Science Robotics*, Vol. 7, No. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074

[13] ros2_control Development Team, *ros2_control documentation*. Available at https://control.ros.org

[14] Koenig N. and Howard A., "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sendai, Japan, pp. 2149-2154, September 2004.

[15] Open Robotics, "Detachable Joints," Gazebo Sim 6 (Fortress) documentation. Available at https://gazebosim.org/api/sim/6/detachablejoints.html

[16] Lynch K. M. and Park F. C., *Modern Robotics: Mechanics, Planning, and Control*, Cambridge: Cambridge University Press, 2017.

[17] Bradski G., "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, Vol. 25, No. 11, pp. 120-125, 2000.

[18] Zhao W., Peña Queralta J. and Westerlund T., "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey," *Proc. IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020. DOI: 10.1109/SSCI47803.2020.9308468

[19] International Organization for Standardization, ISO 13482:2014, *Robots and Robotic Devices - Safety Requirements for Personal Care Robots*, Geneva: ISO, 2014.

[20] International Electrotechnical Commission, IEC 60601-1:2005, *Medical Electrical Equipment - Part 1: General Requirements for Basic Safety and Essential Performance*, Geneva: IEC, 2005.

[21] International Organization for Standardization, ISO 13485:2016, *Medical Devices - Quality Management Systems - Requirements for Regulatory Purposes*, Geneva: ISO, 2016.

[22] United Arab Emirates, Federal Law No. 2 of 2019 on the Use of Information and Communication Technology in Health Fields (retention period: Article 20). Official text available at https://elaws.moj.gov.ae/UAE-MOJ_LC-En/00_Health/UAE-LC-En_2019-02-06_00002_Kait.html, accessed 12 November 2026.

[23] United Arab Emirates, Federal Decree-Law No. 45 of 2021 Concerning the Protection of Personal Data. Official text available at https://uaelegislation.gov.ae/en/legislations/1972, accessed 12 November 2026.

[24] Hiwonder, *Robot Arm Advanced Control*, ROS 2 Course, Chapter 2, course documentation supplied with the JetArm Pro.

[25] Hiwonder, *URDF Modeling & Simulation: Gazebo Simulation*, ROS 2 Course, Chapter 7, course documentation and virtual machine image supplied with the JetArm Pro.

APPENDICES

Appendix A: Software Repository

The TrackMed software is published as a public Git repository on GitHub:

https://github.com/Seif-Taha12903478/trackmed-medication-delivery-robot

It has a commit for every working milestone and tagged releases (v1.0 for the delivered system; the latest release, v1.2, holds this report). Its history starts on 4 November 2026, when the simulation work was put under version control; the earlier work, from the proposal and the hardware set-up to the set-up of the simulation environment, is described in Chapters 3 and 6. Its README is the entry point: it summarises the project, shows a preview of the demonstration video, and explains installation, use and testing; the `docs` folder holds the architecture, testing, user and troubleshooting guides and this report. The offline tests (`tools/run_offline_tests.sh`) run automatically on GitHub on every change. The manufacturer's ROS 2 packages are not included, because they declare no licence; the README explains where to obtain them. It is installed into the manufacturer's Gazebo package in the ROS 2 workspace by an install script, so that the manufacturer's own files are never edited. Table A.1 lists its contents.

Path	Contents
install.sh	Copies the robot, world and launch files into the Gazebo package and rebuilds it
README.md, docs/	Entry point; architecture, testing, user and troubleshooting guides; this report
LICENSE, NOTICE	MIT licence for the team's code; terms of the manufacturer-derived files (NOTICE)
tools/run_offline_tests.sh, .github/workflows/offline-tests.yml	All offline tests in one command; automatic run on GitHub on every change
CLAUDE.md	Project notes: design decisions, known issues, next steps
robot_gazebo/urdf/ jetarm_mecanum.gazebo.xacro	Arm on the mecanum chassis, planar base joints, wrist camera, Gazebo systems
robot_gazebo/worlds/hospital.sdf, hospital_video.sdf, hospital_photo.sdf	Hospital world with the patient room (normal, video and photo variants), generated
robot_gazebo/launch/hospital.launch.py	Starts Gazebo, bridge, robot, base node and controllers; gui and video options
scripts/jetarm_pick_place.py	Kinematics, pick-and-place planner and arm/gripper control node
scripts/line_follower.py	Line-following node
scripts/planar_base.py	Converts velocity commands to planar joint velocities; publishes odometry

Path	Contents
scripts/base_moves.py	Closed-loop sideways, forward and rotation moves on odometry
scripts/medicine_weld.py	Confirmed attach and release of the container
scripts/mission.py	Eleven-step delivery mission with checks; writes the result file
scripts/smart_table.py	Smart bedside table controller node and command-line raise, lower and status
tools/test_smart_table.py	Stand-alone test of the smart table (29 checks)
tools/measure_rtf.py	Real-time factor of a fresh headless launch (results/rtf.csv)
scripts/scheduler.py, schedule.yaml	Medication scheduler and example schedule
scripts/medicine_container.sdf	Container model
tools/make_hospital_world.py	World generator: ward, patient room, smart table, colour and clearance checks
tools/evaluate.py, scoring.py, rescore.py	Automated evaluation, shared scoring, re-scoring
tools/sim_screenshot.py, record_video.py	Screenshots and video from the overview camera
results/	Evaluation results (runs.csv, summary.txt, per-run JSON)
media/mission_overview.mp4, mission_confirmed.mp4	Demonstration videos of scheduled deliveries before the patient room
media/mission_room.mp4, room_*.png	Demonstration video and pictures of a delivery in the patient room
tools/system_layout.py, make_system_world.py	Hospital system layout (with clearance checks) and world generator: corridor, pharmacy, 4 rooms, line network
robot_gazebo/launch/ hospital_system.launch.py	Starts the hospital system (normal, video and photo worlds)
config/hospital_map.yaml, inventory.yaml, doses.yaml	Line network graph, medicine storage inventory, medication schedule
scripts/route_planner.py, route_follower.py	Dijkstra route planner; line follower with junction handling
scripts/pharmacy.py, delivery.py	Pharmacy set-up, slot line-up and cap check; 15-step hospital delivery
scripts/hospital_scheduler.py	Scheduler with queueing, stock, lateness and the hospital clock
tools/offline_route_sim.py, test_system_offline.py, test_arm_wait.py	Offline tests of routes, planner, scheduler and arm wait
tools/test_route_gazebo.py, test_pharmacy_pick.py, run_in_sim.py, run_demo.sh	Gazebo tests, test runner with deadline, self-contained demonstration
media/system_demo.mp4, system_*.png	Demonstration video of four scheduled deliveries and pictures of the hospital system

Table A.1: Contents of the TrackMed repository

Appendix B: User Guide

B.1 Installation

The simulation runs on Ubuntu 22.04 with ROS 2 Humble, Gazebo Fortress and the manufacturer's simulation packages installed in `~/ros2_ws`. Clone the repository to `~/sdp_sim` (`git clone https://github.com/Seif-Taha12903478/trackmed-medication-delivery-robot.git ~/sdp_sim`) and run the install script, then open a new terminal so that the rebuilt package is found:

```
bash ~/sdp_sim/install.sh
```

The script copies the robot description, world and launch file into the `robot_gazebo` package, checks that the required Gazebo systems and OpenCV are present, and rebuilds the package.

B.2 Running a Single Delivery

Start the simulation (add `gui:=false` to run without the Gazebo window), wait until the three controllers are active, then run the mission:

```
ros2 launch robot_gazebo hospital.launch.py
ros2 control list_controllers        # expect three controllers "active"
python3 ~/sdp_sim/scripts/mission.py
```

The mission prints each step as it starts and finishes with the placement error and the robot's final pose. A mission can be resumed from a given step with `--from N`.

B.3 Running the Scheduler

Edit `~/sdp_sim/schedule.yaml` (Section 5.9.1), then start the scheduler. To test without waiting, run one entry one minute from now:

```
python3 ~/sdp_sim/scripts/scheduler.py --list
python3 ~/sdp_sim/scripts/scheduler.py
python3 ~/sdp_sim/scripts/scheduler.py --in-minutes 1 --entry 1 --once
```

Each delivery is appended to `~/sdp_sim/results/deliveries.csv`.

B.4 Running an Evaluation

The evaluation starts and stops its own simulations, so no simulation may be running when it starts:

```
python3 ~/sdp_sim/tools/evaluate.py --runs 5 --out ~/sdp_sim/results/eval_new
python3 ~/sdp_sim/tools/evaluate.py --out ~/sdp_sim/results/eval_new --append --total 5
python3 ~/sdp_sim/tools/rescore.py ~/sdp_sim/results/eval_new
```

B.5 Useful Commands

Table B.1 lists the commands used most often during development and testing.

Command	Purpose
python3 scripts/jetarm_pick_place.py state	Show arm and gripper joint positions
python3 scripts/jetarm_pick_place.py home	Move the arm to the ready pose
python3 scripts/jetarm_pick_place.py plan	Print the planned pick-and-place waypoints
python3 scripts/line_follower.py --show	Follow the line with a camera window
python3 scripts/base_moves.py rotate 180	Rotate the base 180 degrees on odometry
python3 tools/sim_screenshot.py out.png	Save an overview screenshot
ros2 topic hz /depth_cam/depth_cam	Check the camera rate
python3 scripts/smart_table.py status	Show the table state, platform height and contact
python3 scripts/smart_table.py raise / lower	Raise or lower the table by hand (refused while the gripper is near it)
ros2 topic echo /smart_table/state	Watch the table state
python3 scripts/route_planner.py --all	Print the planned routes of all rooms
bash tools/run_demo.sh	Run the hospital demonstration (four deliveries, video and summary in the demonstration's results folder)
touch results/system/demo_<date>/STOP	End the demonstration after the current delivery
grep END results/system/demo_<date>/scheduler.log	Show the result of each finished delivery

Table B.1: Useful commands

B.6 Troubleshooting

- **Blank Gazebo view:** make sure Gazebo is started with `--render-engine ogre` (already set in the launch file).
- **Two Gazebo windows or duplicated topics:** a previous launch is still running; stop all simulation processes before relaunching.
- **Container floats with the gripper after spawning:** the DetachableJoint welds it at spawn; run `python3 scripts/jetarm_pick_place.py detach` (the mission does this automatically).
- **Very slow simulation:** run headless (`gui:=false`) and keep MoveIt closed.

- **Smart table does not lift:** check `smart_table.py status`; the table lifts only when the container has been in steady contact for 2 s, the weld state is "detached" and the arm is at home.
- **Spawned model does not appear:** give the spawn tool an absolute file path; a relative path is resolved in the simulator's own working directory.

Appendix C: Selected Source Code

The listings below are of the main algorithms as implemented at the stage described in Chapters 5 and 6. The complete and current code is in the repository (Appendix A).

C.1 Arm Kinematics and Pick-and-Place Planner (jetarm_pick_place.py, excerpt)

```
# ---------------------------------------------------------- arm geometry
# From jetarm_6dof_description/urdf/{connect,arm}.urdf.xacro
BASE_Z = 0.05                       # joint1 height above base origin
H2 = 0.054048012164686              # joint1 -> joint2
L2 = 0.134416446394797              # joint2 -> joint3
L3 = 0.134444631186569              # joint3 -> joint4
D4 = 0.0544833339503674             # joint4 -> servo_link2 (wrist)
D_EE = 0.08                         # link5 -> end_effector_link
Z2 = BASE_Z + H2                    # joint2 pivot height
LT = D4 + D_EE                      # joint4 pivot -> tool point
JOINT_LIMIT = 2.09                  # +/- rad on joint1..joint5
MARGIN = 0.10                       # keep this far from the limits
INIT_POSE = [0.0, 0.7309, -1.6127, -1.6591, 0.0]     # SRDF 'init_pose'

ARM_TOPIC = '/arm_controller/joint_trajectory'
GRIPPER_TOPIC = '/gripper_controller/joint_trajectory'
JOINT_STATES_TOPIC = '/joint_states'
ARM_JOINTS = ['joint1', 'joint2', 'joint3', 'joint4', 'joint5']
GRIP_JOINTS = ['r_joint']


def fk(q):
    """Tool-point position and tool direction for q = [j1..j5] (all about
    axes (0,-1,0) for j2..j4)."""
    j1, j2, j3, j4, _ = q
    a = j2
    zx = [(H2, a)]
    # planar chain in the arm plane: angle from vertical toward +x is -(sum)
    ang = -j2
    r, z = 0.0, Z2
    r += L2 * math.sin(ang); z += L2 * math.cos(ang)
    ang = -(j2 + j3)
    r += L3 * math.sin(ang); z += L3 * math.cos(ang)
    ang = -(j2 + j3 + j4)
    r += LT * math.sin(ang); z += LT * math.cos(ang)
    return (r * math.cos(j1), r * math.sin(j1), z), ang
```

```
def ik_down(x, y, z, max_tilt_deg=60):
    """Joint angles that put the tool point at (x, y, z) with the tool
    pointing down (tilted outward as little as needed)."""
    j1 = math.atan2(y, x)
    r = math.hypot(x, y)
    for tilt in range(0, max_tilt_deg + 1, 5):
        t = math.radians(tilt)
        phit = math.pi - t                      # tool direction from vertical
        wr = r - LT * math.sin(phit)
        wz = z - LT * math.cos(phit)
        dx, dz = wr, wz - Z2
        d = math.hypot(dx, dz)
        if d > L2 + L3 - 1e-6 or d < abs(L2 - L3) + 1e-6:
            continue
        base = math.atan2(dx, dz)
        a = math.acos((d * d + L2 * L2 - L3 * L3) / (2 * d * L2))
        for sgn in (1, -1):
            pa = base + sgn * a
            v = (dx - L2 * math.sin(pa), dz - L2 * math.cos(pa))
            pb = math.atan2(v[0], v[1])
            wrap = lambda u: (u + math.pi) % (2 * math.pi) - math.pi
            q = [j1, wrap(-pa), wrap(-(pb - pa)), wrap(-(phit - pb)), 0.0]
            if all(abs(v_) <= JOINT_LIMIT - MARGIN for v_ in q):
                (ex, ey, ez), _ = fk(q)
                if abs(ex - x) + abs(ey - y) + abs(ez - z) < 1e-3:
                    return q, tilt
    raise ValueError('Position (%.3f, %.3f, %.3f) is out of reach' % (x, y, z))


def build_plan(a):
    """Named waypoints; descents/ascents are several points so the tool goes
    (almost) straight down/up."""
    def pose(xy, z):
        q, tilt = ik_down(xy[0], xy[1], z)
        return q

    def col(xy, z_start, z_end, n=4):
        zs = [z_start + (z_end - z_start) * i / (n - 1) for i in range(n)]
        return [pose(xy, z) for z in zs]

    plan = {
        'pick_above': pose(a.pick, a.above_z),
        'pick_descent': col(a.pick, a.above_z, a.grasp_z)[1:],
        'pick_lift': col(a.pick, a.grasp_z, a.above_z)[1:],
        'place_above': pose(a.place, a.above_z),
        'place_descent': col(a.place, a.above_z, a.release_z)[1:],
        'place_retreat': col(a.place, a.release_z, a.above_z)[1:],
        'home': INIT_POSE,
    }
    return plan
```

C.2 Line Follower (line_follower.py)

```
#!/usr/bin/env python3
"""Blue-line follower for the simulated JetArm mecanum robot (ROS 2 Humble).

Port of Hiwonder's line_following node ("JetArm and Mecanum Chassis
Integration Course", section 6) to the simulation:
  * same camera (arm wrist camera, arm in its ready pose looking down),
  * same three image strips near the bottom of the image with weights
    0.7 / 0.2 / 0.1 (Hiwonder's GEMINI ROIs),
  * deflection angle = -atan((cx - w/2) / (h/2)), PD steering,
  * constant forward speed 0.08 m/s (Hiwonder uses 80 mm/s).
Differences: colour is a fixed HSV range instead of click-to-pick, the
weighted centre is normalised by the strips actually found (Hiwonder divides
by the full weight sum), commands are geometry_msgs/Twist on
/controller/cmd_vel, and the robot stops on a red floor marker.

Usage:
  python3 line_follower.py             # follow until the red marker, then stop
  python3 line_follower.py --show      # also open a camera window
Exit code 0 = arrived at marker, 2 = line lost, 1 = error / aborted.
A debug image is written to /tmp/line_follower_debug.jpg about once a second.
"""
import argparse
import math
import sys
import time

import cv2
import numpy as np
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from sensor_msgs.msg import Image

IMAGE_TOPIC = '/depth_cam/depth_cam'
CMD_TOPIC = '/controller/cmd_vel'
# (y_start, y_end, x_start, x_end, weight) as fractions of the image (Hiwonder GEMINI)
ROIS = ((0.90, 0.95, 0.0, 1.0, 0.7), (0.80, 0.85, 0.0, 1.0, 0.2), (0.70, 0.75, 0.0, 1.0, 0.1))
BLUE_HSV = ((100, 120, 60), (130, 255, 255))
RED_HSV = (((0, 120, 60), (10, 255, 255)), ((170, 120, 60), (180, 255, 255)))


def image_to_bgr(msg):
    buf = np.frombuffer(msg.data, dtype=np.uint8)
    if msg.encoding in ('rgb8', 'bgr8'):
        img = buf.reshape(msg.height, msg.step)[:, :msg.width * 3].reshape(msg.height,
msg.width, 3)
        return cv2.cvtColor(img, cv2.COLOR_RGB2BGR) if msg.encoding == 'rgb8' else img.copy()
    raise ValueError('unsupported image encoding %s' % msg.encoding)
```

```
class LineFollower(Node):
    def __init__(self, a):
        super().__init__('line_follower')
        self.a = a
        self.pub = self.create_publisher(Twist, CMD_TOPIC, 10)
        self.create_subscription(Image, IMAGE_TOPIC, self.on_image, 5)
        self.result = None          # 'arrived' | 'lost'
        self.prev_t = None
        self.prev_err = 0.0
        self.lost_since = None
        self.red_frames = 0
        self.frames = 0
        self.t_start = None
        self.last_debug = 0.0

    @staticmethod
    def stamp(msg):
        return msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9

    def analyse(self, bgr):
        h, w = bgr.shape[:2]
        hsv = cv2.cvtColor(cv2.GaussianBlur(bgr, (3, 3), 0), cv2.COLOR_BGR2HSV)
        kernel = np.ones((3, 3), np.uint8)
        centre_sum, weight_found, red_px, band_px = 0.0, 0.0, 0, 0
        for (y0, y1, x0, x1, wt) in ROIS:
            ys, ye, xs, xe = int(y0 * h), int(y1 * h), int(x0 * w), int(x1 * w)
            roi = hsv[ys:ye, xs:xe]
            band_px += roi.shape[0] * roi.shape[1]
            red = sum(cv2.inRange(roi, np.array(lo), np.array(hi)) for lo, hi in RED_HSV)
            red_px += int(np.count_nonzero(red))
            mask = cv2.inRange(roi, np.array(BLUE_HSV[0]), np.array(BLUE_HSV[1]))
            mask = cv2.dilate(cv2.erode(mask, kernel), kernel)
            contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
            contours = [c for c in contours if cv2.contourArea(c) > self.a.min_area]
            if contours:
                c = max(contours, key=cv2.contourArea)
                (cx, cy), _, _ = cv2.minAreaRect(c)
                cx += xs
                centre_sum += cx * wt
                weight_found += wt
                cv2.circle(bgr, (int(cx), int(cy + ys)), 4, (0, 255, 255), -1)
            cv2.rectangle(bgr, (xs, ys), (xe - 1, ye - 1), (0, 255, 0), 1)
        red_frac = red_px / max(band_px, 1)
        if weight_found == 0:
            return None, red_frac
        centre = centre_sum / weight_found
        cv2.line(bgr, (int(centre), 0), (int(centre), h - 1), (255, 0, 255), 1)
        return -math.atan((centre - w / 2.0) / (h / 2.0)), red_frac

    def send(self, vx=0.0, wz=0.0):
        t = Twist()
        t.linear.x = float(vx)
        t.angular.z = float(wz)
        self.pub.publish(t)
```

```
    def on_image(self, msg):
        if self.result:
            return
        try:
            bgr = image_to_bgr(msg)
        except ValueError as e:
            self.get_logger().error(str(e))
            return
        t = self.stamp(msg)
        if self.t_start is None:
            self.t_start = t
        self.frames += 1
        angle, red_frac = self.analyse(bgr)

        if t - self.t_start > self.a.ignore_stop and red_frac > self.a.stop_frac:
            self.red_frames += 1
        else:
            self.red_frames = 0
        if self.red_frames >= 2:
            self.send()
            self.result = 'arrived'
            self.get_logger().info('Red stop marker reached.')
        elif angle is None:
            self.send()
            self.lost_since = self.lost_since if self.lost_since is not None else t
            if t - self.lost_since > self.a.lost_timeout:
                self.result = 'lost'
                self.get_logger().warn('Line lost for %.1f s - stopping.' %
self.a.lost_timeout)
        else:
            self.lost_since = None
            dt = (t - self.prev_t) if self.prev_t is not None else 0.0
            d = (angle - self.prev_err) / dt if dt > 1e-6 else 0.0
            self.prev_t, self.prev_err = t, angle
            wz = max(-self.a.max_turn, min(self.a.max_turn, self.a.kp * angle + self.a.kd * d))
            vx = self.a.speed * max(0.4, 1.0 - abs(angle) / 0.6)
            self.send(vx, wz)

        now = time.monotonic()
        if now - self.last_debug > 1.0:
            self.last_debug = now
            txt = 'angle=%s red=%.2f %s' % ('none' if angle is None else '%.3f' % angle,
                                            red_frac, self.result or '')
            cv2.putText(bgr, txt, (5, 15), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 0, 0), 1)
            cv2.imwrite('/tmp/line_follower_debug.jpg', bgr)
            self.get_logger().info('sim t=%.1f  %s' % (t - self.t_start, txt))
        if self.a.show:
            cv2.imshow('line follower', bgr)
            cv2.waitKey(1)


def main():
```

```
    p = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument('--speed', type=float, default=0.08, help='forward speed m/s (default
0.08)')
    p.add_argument('--kp', type=float, default=1.2)
    p.add_argument('--kd', type=float, default=0.1)
    p.add_argument('--max-turn', type=float, default=0.6, help='max yaw rate rad/s')
    p.add_argument('--min-area', type=float, default=15, help='min blob area in pixels')
    p.add_argument('--stop-frac', type=float, default=0.15, help='red fraction of the strips
that means stop')
    p.add_argument('--ignore-stop', type=float, default=0.0, help='ignore red for the first S
sim seconds')
    p.add_argument('--lost-timeout', type=float, default=2.0, help='sim seconds without line
before giving up')
    p.add_argument('--show', action='store_true', help='open a camera window')
    a = p.parse_args()

    rclpy.init()
    node = LineFollower(a)
    node.get_logger().info('Waiting for camera images on %s ...' % IMAGE_TOPIC)
    code = 1
    try:
        while rclpy.ok() and node.result is None:
            rclpy.spin_once(node, timeout_sec=0.2)
        code = 0 if node.result == 'arrived' else 2
    except KeyboardInterrupt:
        print('\nAborted.')
    finally:
        for _ in range(3):
            node.send()
            time.sleep(0.05)
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()
    print('Result:', node.result or 'aborted')
    sys.exit(code)


if __name__ == '__main__':
    main()
```

C.3 World Generator (make_hospital_world.py)

```
#!/usr/bin/env python3
"""Generates robot_gazebo/worlds/hospital.sdf (run again after editing the layout).

Layout (metres, robot starts at the origin facing +x):
  - medicine station: low table on the robot's LEFT at the start
  - blue line: straight along +x, 90-degree left turn, straight along +y
  - red stop marker at the patient end, and one behind the start (used when
    the robot turns around and follows the line home)
  - patient bed with a low bedside table on the robot's LEFT at the stop
```

```
Table tops are at the arm-plate height (0.10 m) so the arm reaches them with
the same joint angles as the tested pick-and-place.
"""
import math, os, sys

PLATE_H   = 0.10    # chassis plate height = table top height
LINE_W    = 0.03    # tape width
LINE_T    = 0.001   # tape thickness (visual only)
R_TURN    = 0.50    # turn radius
X_TURN    = 1.20    # where the turn starts
Y_END     = 1.75    # patient stop marker (centre)
REACH     = 0.20    # sideways reach used for pick / place (m from arm joint1)
ROBOT_HALF_W = 0.13
HOME_X    = -0.22   # home stop marker (centre)
STOP_AHEAD = 0.29   # robot stops when the marker is this far ahead of joint1 (offline test)

def box_visual(name, x, y, z, yaw, sx, sy, sz, rgba, emissive=None):
    e = emissive or (0, 0, 0, 1)
    return f"""
      <visual name="{name}">
        <pose>{x:.4f} {y:.4f} {z:.4f} 0 0 {yaw:.4f}</pose>
        <geometry><box><size>{sx:.4f} {sy:.4f} {sz:.4f}</size></box></geometry>
        <material>
          <ambient>{rgba[0]} {rgba[1]} {rgba[2]} 1</ambient>
          <diffuse>{rgba[0]} {rgba[1]} {rgba[2]} 1</diffuse>
          <specular>0.1 0.1 0.1 1</specular>
          <emissive>{e[0]} {e[1]} {e[2]} 1</emissive>
        </material>
      </visual>"""

def static_box_model(name, x, y, sx, sy, sz, rgba):
    return f"""
    <model name="{name}">
      <static>true</static>
      <pose>{x:.4f} {y:.4f} {sz/2:.4f} 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><box><size>{sx} {sy} {sz}</size></box></geometry>
          <surface><friction><ode><mu>1.0</mu><mu2>1.0</mu2></ode></friction></surface>
        </collision>
        {box_visual('v', 0, 0, 0, 0, sx, sy, sz, rgba)}
      </link>
    </model>"""

def line_model():
    blue, blue_e = (0.0, 0.15, 0.9), (0.0, 0.05, 0.35)
    red, red_e = (0.9, 0.05, 0.05), (0.35, 0.0, 0.0)
    z = LINE_T / 2 + 0.0005
    parts = []
    # straight 1: from the home marker to the turn
    x0, x1 = HOME_X, X_TURN
    parts.append(box_visual('s1', (x0 + x1) / 2, 0, z, 0, x1 - x0, LINE_W, LINE_T, blue,
blue_e))
    # 90-degree left arc, centre (X_TURN, R_TURN)
```

```
    n = 9
    for i in range(n):
        a0 = -math.pi / 2 + (math.pi / 2) * i / n
        a1 = -math.pi / 2 + (math.pi / 2) * (i + 1) / n
        am = (a0 + a1) / 2
        cx = X_TURN + R_TURN * math.cos(am)
        cy = R_TURN + R_TURN * math.sin(am)
        chord = 2 * R_TURN * math.sin((a1 - a0) / 2) + 0.012
        parts.append(box_visual(f'arc{i}', cx, cy, z, am + math.pi / 2, chord, LINE_W, LINE_T,
blue, blue_e))
    # straight 2: up to the patient marker
    xs = X_TURN + R_TURN
    y0, y1 = R_TURN, Y_END
    parts.append(box_visual('s2', xs, (y0 + y1) / 2, z, math.pi / 2, y1 - y0, LINE_W, LINE_T,
blue, blue_e))
    # stop markers
    parts.append(box_visual('stop_patient', xs, Y_END, z + 0.0002, 0, 0.15, 0.15, LINE_T, red,
red_e))
    parts.append(box_visual('stop_home', HOME_X, 0, z + 0.0002, 0, 0.15, 0.15, LINE_T, red,
red_e))
    return f"""
    <model name="floor_line">
      <static>true</static>
      <link name="link">{''.join(parts)}
      </link>
    </model>"""

def world():
    xs = X_TURN + R_TURN
    # medicine station: robot's left at the start (container at y = REACH)
    st_edge = REACH - 0.03                    # table edge nearest the robot
    st = static_box_model('medicine_station', 0.0, st_edge + 0.15, 0.30, 0.30, PLATE_H, (0.85,
0.85, 0.95))
    # at the patient stop the robot faces +y, its left is -x
    stop_y = Y_END - STOP_AHEAD               # arm joint1 when the robot stops
    bt_edge = xs - (REACH - 0.03)
    bedside = static_box_model('bedside_table', bt_edge - 0.15, stop_y, 0.30, 0.30, PLATE_H,
(0.95, 0.9, 0.8))
    bed = static_box_model('patient_bed', bt_edge - 0.30 - 0.50, stop_y + 0.4, 0.95, 1.95,
0.50, (0.9, 0.95, 1.0))
    assert st_edge - ROBOT_HALF_W >= 0.035 and xs - ROBOT_HALF_W - bt_edge >= 0.035
    return f"""<?xml version="1.0" ?>
<!-- generated by tools/make_hospital_world.py - edit that file, not this one -->
<sdf version="1.6">
  <world name="robot_world">
    <physics name="default" type="ignored">
      <max_step_size>0.02</max_step_size>
      <real_time_factor>1</real_time_factor>
    </physics>
    <plugin filename="ignition-gazebo-physics-system"
name="ignition::gazebo::systems::Physics"/>
    <plugin filename="ignition-gazebo-user-commands-system"
name="ignition::gazebo::systems::UserCommands"/>
```

```
    <plugin filename="ignition-gazebo-scene-broadcaster-system"
name="ignition::gazebo::systems::SceneBroadcaster"/>
    <!-- renders the robot's camera; ogre2 does not work in this VMware VM -->
    <plugin filename="ignition-gazebo-sensors-system"
name="ignition::gazebo::systems::Sensors">
      <render_engine>ogre</render_engine>
    </plugin>
    <scene>
      <ambient>0.8 0.8 0.8 1</ambient>
      <background>0.8 0.8 0.8 1</background>
      <shadows>false</shadows>
    </scene>
    <light type="directional" name="sun">
      <cast_shadows>false</cast_shadows>
      <pose>0 0 10 0 0 0</pose>
      <diffuse>0.9 0.9 0.9 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <direction>-0.3 0.2 -0.9</direction>
    </light>
    <model name="ground_plane">
      <static>true</static>
      <link name="link">
        <collision name="collision">
          <geometry><plane><normal>0 0 1</normal><size>20 20</size></plane></geometry>
        </collision>
        <visual name="visual">
          <geometry><plane><normal>0 0 1</normal><size>20 20</size></plane></geometry>
          <material>
            <ambient>0.75 0.75 0.72 1</ambient>
            <diffuse>0.75 0.75 0.72 1</diffuse>
            <specular>0.05 0.05 0.05 1</specular>
          </material>
        </visual>
      </link>
    </model>{line_model()}{st}{bedside}{bed}
  </world>
</sdf>
"""

if __name__ == '__main__':
    out = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
        os.path.dirname(os.path.abspath(__file__)), '..', 'robot_gazebo', 'worlds',
'hospital.sdf')
    open(out, 'w').write(world())
    xs = X_TURN + R_TURN
    print('wrote', os.path.abspath(out))
    print('medicine container spawn  : x=0.000 y=%.3f z=%.3f' % (REACH, PLATE_H + 0.031))
    print('patient stop (approx)     : joint1 at x=%.3f y=%.3f, facing +y' % (xs, Y_END -
STOP_AHEAD))
    print('place point (robot left)  : x=%.3f y=%.3f' % (xs - REACH, Y_END - STOP_AHEAD))
    print('home stop (approx)        : joint1 at x=%.3f y=0, facing -x' % (HOME_X +
STOP_AHEAD))
```

C.4 Medication Container Model (medicine_container.sdf)

```
<?xml version="1.0"?>
<sdf version="1.7">
  <model name="medicine_container">
    <link name="body">
      <!-- 30 g, 30 mm diameter x 60 mm tall cylinder.
           Ixx=Iyy=m(3r^2+h^2)/12, Izz=m r^2/2 -->
      <inertial>
        <mass>0.03</mass>
        <inertia>
          <ixx>1.07e-5</ixx><iyy>1.07e-5</iyy><izz>3.4e-6</izz>
          <ixy>0</ixy><ixz>0</ixz><iyz>0</iyz>
        </inertia>
      </inertial>
      <collision name="collision">
        <geometry>
          <cylinder><radius>0.015</radius><length>0.06</length></cylinder>
        </geometry>
        <surface>
          <friction><ode><mu>1.5</mu><mu2>1.5</mu2></ode></friction>
          <bounce><restitution_coefficient>0.0</restitution_coefficient></bounce>
          <contact><ode><kp>1e6</kp><kd>1</kd><min_depth>0.001</min_depth></ode></contact>
        </surface>
      </collision>
      <visual name="bottle">
        <geometry>
          <cylinder><radius>0.015</radius><length>0.06</length></cylinder>
        </geometry>
        <material>
          <ambient>0.9 0.9 0.9 1</ambient>
          <diffuse>0.9 0.9 0.9 1</diffuse>
          <specular>0.3 0.3 0.3 1</specular>
        </material>
      </visual>
      <visual name="cap">
        <pose>0 0 0.036 0 0 0</pose>
        <geometry>
          <cylinder><radius>0.017</radius><length>0.012</length></cylinder>
        </geometry>
        <material>
          <ambient>0.1 0.3 0.9 1</ambient>
          <diffuse>0.1 0.3 0.9 1</diffuse>
          <specular>0.3 0.3 0.3 1</specular>
        </material>
      </visual>
    </link>
  </model>
</sdf>
```

C.5 Wrist Camera in the Robot Description (excerpt)

```
<!-- ===== simulated wrist camera =====
     Mounted on link4 like the real Gemini camera (above the gripper,
     looking along the gripper). Sensor +x = link4 +z (view direction),
     sensor +z = -link4 x (image "up"), so the image is upright.
     Offset 5 cm "above" the gripper keeps the fingers out of view.
     Low resolution / rate on purpose: the VM is slow.                    -->
<link name="sim_cam_link"/>
<joint name="sim_cam_joint" type="fixed">
  <parent link="link4"/>
  <child link="sim_cam_link"/>
  <origin xyz="-0.05 0 0.03" rpy="0 ${-PI/2} 0"/>
</joint>
<gazebo reference="sim_cam_link">
  <sensor name="wrist_cam" type="camera">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image>
        <width>320</width>
        <height>200</height>
        <format>R8G8B8</format>
      </image>
      <clip><near>0.03</near><far>5.0</far></clip>
    </camera>
    <always_on>1</always_on>
    <update_rate>10</update_rate>
    <visualize>false</visualize>
    <topic>/depth_cam/depth_cam</topic>
  </sensor>
</gazebo>
```

C.6 Smart Bedside Table Controller (smart_table.py, excerpt)

```
# The platform's ROBOT-side edge stays where the old bedside table's edge was
# (x = 1.53: the robot keeps its ~4 cm side clearance at the patient stop and stays
# out of the 0.27 m turning circle). Centred in y on the place target (patient stop
# y 1.4723 + the arm's reach to the left); the target (1.499, 1.472) is 3 cm inside
# the robot-side edge. A 25 cm platform centred on the target in x would reach
# x = 1.62, into the robot's body (collision box from x = 1.61).
N_STAGES = 3                      # see the module docstring: a 4th joint in series fails
LOWERED_TOP = 0.10                # platform top when lowered = arm plate height (place
height)
RAISED_TOP = 0.80                 # platform top when raised
TRAVEL = RAISED_TOP - LOWERED_TOP  # 0.70 m, TRAVEL / N_STAGES per stage
PLATFORM_SIZE = (0.25, 0.25)      # x, y (m)
PLATFORM_ROBOT_EDGE_X = 1.53
PLATFORM_CENTER = (PLATFORM_ROBOT_EDGE_X - PLATFORM_SIZE[0] / 2, 1.472)
CONTAINER_HALF_H = 0.03           # container centre above the surface it stands on

# ------------------------------------------------------------------ behaviour
CONTAINER = 'medicine'            # model name of the container (contact partner)
```

```
CONFIRM_S = 2.0                    # steady contact before lifting (sim s)
LOST_S = 3.0                       # no contact on the raised table -> lower (sim s)
GAP_S = 0.3                        # contact messages further apart = contact interrupted
SPEED = 0.04                       # m/s platform speed (cruise)
ACCEL = 0.04                       # m/s^2 at start and stop
STAGE_CMD_MAX = 0.02               # m/s per stage, Gazebo-side cap (world file cmd_max), 1.5x
cruise
HOME_TOL = 0.06                    # rad: arm "at home"


def profile(h0, h1, tau, v=SPEED, a=ACCEL):
    """Trapezoidal (or triangular) move h0 -> h1 starting and ending at rest.
    Returns (position at tau sim s after the start, total duration)."""
    d = abs(h1 - h0)
    if d < 1e-9:
        return h1, 0.0
    ta = v / a
    if d < v * ta:                      # too short to reach full speed
        ta = math.sqrt(d / a)
        v = a * ta
    tc = (d - v * ta) / v
    T = 2 * ta + tc
    if tau <= 0:
        s = 0.0
    elif tau < ta:
        s = 0.5 * a * tau * tau
    elif tau < ta + tc:
        s = 0.5 * a * ta * ta + v * (tau - ta)
    elif tau < T:
        r = T - tau
        s = d - 0.5 * a * r * r
    else:
        s = d
    return h0 + math.copysign(s, h1 - h0), T




    # ... (SmartTable node: subscriptions, contact tracking, publishers)

    def update(self):
        touching, steady, since = self.contact()
        cs = 'steady' if steady else ('touching' if touching else 'none')
        if cs != self.contact_state:
            self.contact_state = cs
            self.get_logger().info('[sim %.2f] container contact: %s' % (self.now, cs))
            self.publish_contact()
        if self.state == 'lowered':
            fresh = steady and self.streak_start >= (self.lowered_since or 0.0) - 1e-6
            if fresh:
                missing = [w for w, ok in (('weld released', self.weld == 'detached'),
                                           ('arm at home', self.arm_home())) if not ok]
                if not missing:
                    self.wait_reason = None
```

```
                    self.start(RAISED_TOP, 'container steady on the platform for %.1f s, weld
released, '
                                           'arm at home' % (self.now - self.streak_start))
                elif missing != self.wait_reason:
                    self.wait_reason = missing
                    self.get_logger().info('[sim %.2f] container on the platform; waiting for:
%s'
                                           % (self.now, ', '.join(missing)))
        elif self.state == 'raised':
            if touching:
                self.raised_had_contact = True
            elif self.raised_had_contact and since is not None and since >= LOST_S:
                if self.gripper_clear():
                    self.start(LOWERED_TOP, 'container removed: no contact for %.1f s' % since)
                elif self.wait_reason != ['gripper clear']:
                    self.wait_reason = ['gripper clear']
                    self.get_logger().warn('container removed, but the gripper is near the
table: not lowering')
        # ... (end of a move: lowered / raised; set-points sent to the three stages)
```

C.7 Hospital Route Planner (route_planner.py, excerpt)

```
class Network:
    def __init__(self, m=None):
        self.m = m or load_map()
        self.nodes = self.m['nodes']
        self.adj = {n: [] for n in self.nodes}
        for e in self.m['edges']:
            a, b, l = e['from'], e['to'], float(e['length'])
            self.adj[a].append((b, l))
            self.adj[b].append((a, l))

    def heading(self, a, b):
        na, nb = self.nodes[a], self.nodes[b]
        return math.degrees(math.atan2(nb['y'] - na['y'], nb['x'] - na['x']))

    def shortest(self, src, dst):
        """Dijkstra: (length, [src, ..., dst])."""
        if src not in self.nodes or dst not in self.nodes:
            raise KeyError('unknown node %s / %s (nodes: %s)' % (src, dst, ',
'.join(self.nodes)))
        dist, prev, todo = {src: 0.0}, {}, [(0.0, src)]
        while todo:
            d, n = heapq.heappop(todo)
            if n == dst:
                break
            if d > dist.get(n, 1e18):
                continue
            for nb, l in self.adj[n]:
                nd = d + l
                if nd < dist.get(nb, 1e18):
                    dist[nb], prev[nb] = nd, n
                    heapq.heappush(todo, (nd, nb))
```

```
        if dst not in dist:
            raise ValueError('no route %s -> %s' % (src, dst))
        path = [dst]
        while path[-1] != src:
            path.append(prev[path[-1]])
        return dist[dst], path[::-1]

    def plan(self, src, dst, heading_deg=None):
        """heading_deg: the robot's heading at src (None = facing along the first edge)."""
        length, path = self.shortest(src, dst)
        if len(path) < 2:
            raise ValueError('already at %s' % dst)
        h0 = self.heading(path[0], path[1])
        turn_round = heading_deg is not None and abs(wrap_deg(h0 - heading_deg)) > 90.0
        events = []
        for i in range(1, len(path)):
            n = path[i]
            nd = self.nodes[n]
            h_in = self.heading(path[i - 1], n)
            ev = {'node': n, 'colour': nd['marker'], 'x': nd['x'], 'y': nd['y'],
'edge_heading': round(h_in, 3)}
            if i == len(path) - 1:
                ev['action'], ev['out_heading'] = 'stop', round(h_in, 3)
            else:
                h_out = self.heading(n, path[i + 1])
                d = wrap_deg(h_out - h_in)
                if nd['type'] == 'junction':
                    ev['action'] = 'straight' if abs(d) < 1.0 else ('left' if d > 0 else
'right')
                    tt = (self.m.get('turns') or {}).get(n, {}).get('%s->%s' % (path[i - 1],
path[i + 1]))
                    if tt and tt != ev['action']:
                        raise ValueError('map turn table says %s at %s, headings say %s' % (tt,
n, ev['action']))
                else:
                    if abs(d) > 1.0:
                        raise ValueError('route turns at the non-junction node %s' % n)
                    ev['action'] = 'pass'
                ev['out_heading'] = round(h_out, 3)
            events.append(ev)
        return {'from': src, 'to': dst, 'path': path, 'length': round(length, 4),
                'start_heading': round(h0, 3), 'turn_round': bool(turn_round), 'events':
events}
```

Appendix D: Evaluation Data

Table D.1 lists the scored result of every evaluation run, as written by the evaluation tool to `runs.csv`; evaluation 1 is the folder `results/eval_5runs` and evaluation 2 the folder

`results/eval_5runs_confirmed`. Raw container and robot poses for each run are stored in the per-run JSON files in the repository.

Eval.	Run	Success	Placement error (mm)	Tilt (deg)	On table	Sim. time (s)	Note
1	1	False	199.94	180.0	False	162.68	Container on the floor (z 0.030 m)
1	2	True	0.09	0.0	True	160.54	
1	3	False	2034.83	34.3	False	162.26	Still held by the gripper (z 0.272 m)
1	4	False	1966.71	0.0	True	161.76	On the station table, not at the target
1	5	True	0.07	0.0	True	161.64	
2	1	True	0.19	0.0	True	160.24	Attempts 1/1/1
2	2	True	0.20	0.0	True	160.08	Attempts 1/1/1
2	3	True	0.20	0.0	True	160.30	Attempts 1/1/1
2	4	True	0.19	0.0	True	160.22	Attempts 1/1/1; wall time invalid (computer asleep)
2	5	True	0.19	0.0	True	160.08	Attempts 1/1/1

Table D.1: Per-run evaluation data

Table D.2 lists the two scheduled missions in the patient room with the smart bedside table, from their result files.

Mission	Success	Placement error (mm)	Tilt (deg)	Lift time (s)	Platform after lift (m)	Container z after lift (m)	On platform	Sim. time (s)
20261105_173406	True	0.27	0.13	18.56	0.7998	0.830	True	178.34
20261105_183552	True	0.25	0.13	18.68	0.7997	0.830	True	178.38

Table D.2: Missions in the patient room with the smart table

Table D.3 lists the four deliveries of the hospital-system demonstration (attempt 2), from the delivery log.

Room	Patient	Medicine	Slot	Lateness (min)	Placement (mm)	Lift time (s)	Right container	Others in place	Sim. time (s)
1	A	Paracetamol	S1	0.0	2.01	18.36	True	True	185.2
3	C	Ibuprofen	S3	3.08	1.44	18.48	True	True	237.7
4	D	Metformin	S4	5.05	0.56	18.68	True	True	238.8
2	B	Amoxicillin	S2	7.03	1.71	18.48	True	True	181.4

Table D.3: Deliveries of the hospital-system demonstration